SENAI Limeira/SP
Curso Técnico em Desenvolvimento de Sistemas — 3º DEV-A



SENTINEL

Interface Conversacional Agêntica sobre Sistema MVC

Documentação técnica — versão 1.2

Arquitetura, agente, fluxos, segurança, código, dados, testes, rastreabilidade e guia do código-fonte do sistema construído nas Fases 0 a 6

Equipe: Lucas Terminiello, Matheus Malaman, Jefferson Bruno C. Miguel, Júlio Cena

Limeira, novembro de 2026

Controle do documento

Campo	Valor
Documento	Sentinel — Documentação técnica
Versão	1.2 — 4 de novembro de 2026
Base do código	Repositório SENTINEL, branch `main`, revisão `c641002`
Situação do projeto	Construção concluída; Fase 6 em andamento (teste manual, ajustes visuais e preparação da apresentação); entrega final em 25/11/2026
Substitui	Documentação 1.1 (28 de outubro de 2026), revisão `ae79781`
Fontes	Código-fonte e suíte de testes na revisão `c641002`, relatórios de teste em `documentacao/testes/` e Notion (projeto SENTINEL)

O que mudou em relação à 1.1. Nenhum diagrama compatível com o sistema foi removido; os que mudaram foram atualizados e outros foram acrescentados. Em resumo:

- **Sistema:** conciliação em lote (RF03) implementada, intenção explícita com conferência da tool escolhida (`AI_INTENT_MISMATCH`), exportação da auditoria em CSV, coluna `lote_id`, catálogo com 38 tools e os números da suíte em `c641002`.
- **Diagramas novos:** casos de uso por papel, relações entre os componentes, estados de um lote e três diagramas de sequência (comando, confirmação e restauração com senha de administrador). Arquitetura, fluxo F1, modelo de dados, fronteira da IA e plano do lote foram redesenhados.
- **Seção 12 reescrita:** tipos de teste e técnicas de teste, cada um explicado e ligado a exemplos reais da suíte.
- **Apêndice B novo:** guia do código-fonte, arquivo por arquivo.

Como ler este documento. As seções 1 a 3 apresentam o problema, o conceito e o público. As seções 4 a 7 descrevem o sistema construído: visão geral, arquitetura, o agente e os fluxos. As seções 8 a 10 tratam de segurança, do código e dos dados. As seções 11 a 13 cobrem interface, testes e a rastreabilidade de cada requisito. As seções 14 e 15 registram limitações e próximos passos. O Apêndice A é o glossário e o Apêndice B percorre o código-fonte arquivo por arquivo. Os diagramas largos estão em páginas deitadas.

Sumário

1. Contexto e motivação 5
1.1 Inviabilidade técnica do caminho original 5
1.2 Uma oportunidade mais forte 5
2. Conceito central 6
2.1 Exemplo de uso 6
2.2 Fundamentação teórica 6
2.3 Os quatro padrões de interação 6
3. Público-alvo e mercado 8
3.1 O tamanho e a natureza do mercado 8
3.2 O gargalo é a interface, não o acesso 8
3.3 Definição do público-alvo 8
3.4 Predições sobre o público-alvo 9
4. Visão geral do sistema construído 11
4.1 Stack tecnológica 11
4.2 Papéis e permissões 12
4.3 Casos de uso por papel 12
4.4 Telas 13
5. Arquitetura 15
5.1 As camadas 15
5.2 Componentes principais 16
5.3 Rotas 17
5.4 Decisões arquiteturais 18
5.5 Resiliência quando a IA falha 19
5.6 Relações entre os componentes 19
6. O agente 21
6.1 O catálogo de ferramentas 21
6.2 O que vai e o que nunca vai para a IA 21
6.3 Memória da conversa 22
6.4 Como o pedido e a resposta do modelo são conferidos 22
6.5 Tempo de resposta 23
7. Fluxos da aplicação 25
7.1 F1: do comando à gravação 25
7.2 Estados de uma proposta 27
7.3 Segurar para confirmar 28
7.4 F2: atualizar um registro 28
7.5 F3: lixeira, arquivamento e exclusão definitiva 29
7.6 F4: consultar e exportar 31
7.7 F5: modo manual, sem IA 32
7.8 F6: conciliação em lote (RF03) 33
7.9 Diagramas de sequência 35
8. Segurança 41
8.1 Princípios 41
8.2 Ameaças e proteções 42
8.3 Dados pessoais (LGPD) 43
8.4 Autenticação, sessão e limites 44
8.5 Ambiente e dependências 45

9. O código em detalhe 46
9.1 Uma tool no catálogo 46
9.2 A intenção explícita, antes da IA 47
9.3 A chamada ao provedor e a medição do tempo 48
9.4 Conferindo o que o modelo respondeu 49
9.5 Revisar: validar sem gravar 50
9.6 Confirmar: sem segredo antes da hora 51
9.7 Uso único do token 51
9.8 Execução e auditoria na mesma transação 52
9.9 RBAC com falha fechada 52
9.10 Regras exists presas ao tenant 53
9.11 Máscara LGPD 53
9.12 Higiene do histórico da conversa 54
9.13 Limite atômico de tentativas de senha 54
9.14 Bloqueio por dependência 55
9.15 Lote: tudo ou nada 56
9.16 CSV sem injeção de fórmula 56
9.17 Segurar para confirmar, no navegador 57
10. Modelo de dados e auditoria 58
10.1 A tabela de auditoria 59
10.2 Migrations 60
11. Interface e acessibilidade 61
11.1 Sinalização de risco 61
11.2 Área de trabalho redesenhada 61
11.3 Acessibilidade 61
12. Testes do projeto 63
12.1 Tipos de teste realizados 63
12.2 Técnicas usadas para escolher os casos 66
12.3 Infraestrutura de testes 69
12.4 Testes automatizados por arquivo 69
12.5 Testes manuais e homologação 71
13. Requisitos e rastreabilidade 72
13.1 Requisitos funcionais 72
13.2 Requisitos não funcionais 73
14. Limitações reconhecidas 74
15. Próximos passos 76
15.1 Antes da entrega (25/11) 76
15.2 Depois do TCC 76
Referências 77
Apêndice A. Glossário 78
Apêndice B. Guia do código-fonte 79
B.1 Mapa das pastas 79
B.2 Arquivos centrais 80
B.3 Fichas curtas 92
B.4 Arquivos de mesmo formato 97

1. Contexto e motivação

O Sentinel nasceu, no anteprojeto, como uma plataforma SaaS multi-tenant de detecção de anomalias e manutenção preditiva, apoiada em serviços de IA do Azure. No aprofundamento técnico da proposta, dois problemas concretos motivaram a revisão do conceito. Este documento registra o conceito revisado e, principalmente, **o sistema efetivamente construído**: arquitetura, agente, fluxos, segurança, código, dados, testes e rastreabilidade dos requisitos.

1.1 Inviabilidade técnica do caminho original

O Azure AI Anomaly Detector, serviço que embasava a camada preditiva do anteprojeto, está sendo descontinuado pela Microsoft. Desde setembro de 2023 não é possível criar novos recursos do serviço, e a aposentadoria definitiva ocorre em 1º de outubro de 2026, junto com o Metrics Advisor, também cogitado.

Além disso, manutenção preditiva de verdade exige histórico de falhas e treinamento de modelo, e um TCC de poucos meses não comporta, de forma realista, nem o tempo nem os dados que isso demanda.

1.2 Uma oportunidade mais forte

Da análise desses limites surgiu um conceito mais maduro e mais atual. Em vez de prever falhas em sensores, o sistema ataca um problema de usabilidade comum a todo software de gestão: a distância entre o que o usuário quer fazer e a navegação por menus, telas e cadastros necessária para fazê-lo. A camada preditiva foi substituída por uma **camada de interface conversacional agêntica**: o usuário declara a intenção em linguagem natural, e um agente de IA orquestra as operações de um sistema MVC tradicional, mantendo o humano no controle de toda decisão de escrita.

O domínio de aplicação escolhido é o de um **escritório de contabilidade** atendendo micro e pequenas empresas: lançamentos financeiros, categorias, clientes, fornecedores, funcionários e notas fiscais. O nome Sentinel foi mantido.

2. Conceito central

A ideia central cabe em uma frase: **um sistema de gestão em que o usuário declara a intenção em linguagem natural e um agente de IA orquestra as operações do sistema, mantendo o humano no controle das decisões.**

2.1 Exemplo de uso

Em vez de navegar por Cadastros → Lançamentos → Filtros, o usuário digita "quais lançamentos ainda estão pendentes?". O sistema responde com uma tabela que pode ser filtrada, editada linha a linha e exportada. Em seguida, o usuário pede "concilie os lançamentos pendentes de setembro". O sistema não grava nada ainda: abre um card com o **plano** (os lançamentos que vão mudar, um por linha, e o valor total), o usuário confere, clica em **Revisar**, e só depois de **segurar o botão Confirmar por 2 segundos** a alteração é gravada, de uma vez, e registrada na auditoria.

2.2 Fundamentação teórica

Jakob Nielsen descreveu a IA generativa como a primeira mudança de paradigma de interface em 60 anos: a transição da manipulação direta (o usuário especifica como fazer) para a especificação de intenção (o usuário declara o que quer, e o sistema resolve o como).

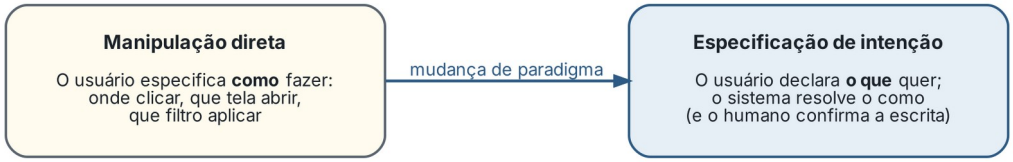


Figura 1 — Mudança de paradigma de interface (Nielsen, 2023).

O mecanismo técnico que viabiliza essa transição é o **tool calling** (chamada de ferramentas): o modelo de linguagem recebe uma lista de ferramentas disponíveis e decide qual invocar e com quais argumentos, mas **não executa** a operação. O padrão de raciocínio e ação foi formalizado no artigo ReAct (Yao et al., 2022) e é a base da documentação oficial de tool use dos principais provedores. O modelo de supervisão adotado, em que toda ação de escrita depende de confirmação humana, é conhecido como **human-in-the-loop**. A equipe resume o argumento como "um Jarvis com freio": o agente entende, sugere e prepara; quem aperta o botão é o humano. Numa empresa de contabilidade, um erro do modelo vira multa, e por isso a confirmação é o diferencial do sistema, não uma limitação aceita.

2.3 Os quatro padrões de interação

Tabela 2 — Os quatro padrões de interação e sua situação nesta versão

Padrão	Como funciona no Sentinel	Situação
Consulta com resultado tabular	O usuário pergunta; o agente escolhe uma tool `consultar_*`; o servidor executa e devolve uma tabela mascarada, com botão Editar por linha e botão Exportar (CSV).	Implementado

Padrão	Como funciona no Sentinel	Situação
Cadastro ou atualização por formulário pré-preenchido	O agente escolhe `criar_*` ou `atualizar_*`; o card abre o formulário com o que a IA entendeu; o humano edita, revisa e segura para confirmar. A validação é do backend (Form Request).	Implementado
Ação em massa com confirmação	O agente pede `conciliar_lancamentos_em_lote` com um período; o servidor monta um **plano** com até 25 lançamentos; o humano revisa o plano inteiro e confirma uma vez; a gravação é tudo ou nada.	Implementado (seção 7.8)
Log de auditoria	Toda decisão (sucesso, erro, recusa, negação, cancelamento, expiração) vira uma linha em `audit_logs`, consultável e exportável por administradores, com o que a IA propôs e o que o humano confirmou.	Implementado

3. Público-alvo e mercado

A definição do público-alvo segue o mesmo rigor exigido do restante do projeto e se apoia em dados oficiais e recentes (Sebrae, 2025).

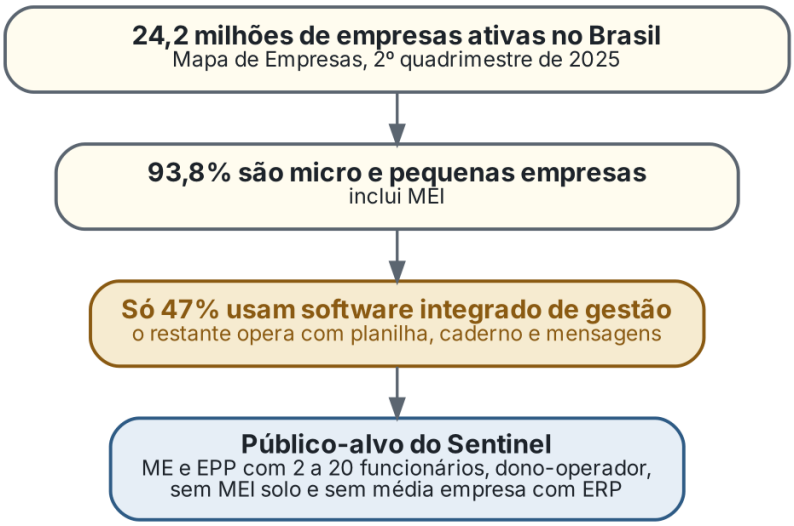


Figura 2 — Funil de definição do público-alvo com base em dados do Sebrae.

3.1 O tamanho e a natureza do mercado

O Brasil encerrou o segundo quadrimestre de 2025 com 24,2 milhões de empresas ativas, das quais 93,8% são micro e pequenas empresas. Apenas 47% dos pequenos negócios utilizam softwares ou aplicativos integrados de gestão.

3.2 O gargalo é a interface, não o acesso

76% dos empreendedores já usam computador nas atividades, e 88% usam internet no dia a dia, concentrada em redes sociais e serviços bancários; menos de 13% usam plataformas de gestão. O padrão é o de um empreendedor fluente em interface de conversa e pouco familiarizado com interface de sistema, exatamente a lacuna que a interface conversacional agêntica ataca.

3.3 Definição do público-alvo

Micro e pequenas empresas brasileiras com aproximadamente 2 a 20 funcionários, geridas por um dono-operador sem departamento administrativo dedicado, excluindo o MEI que trabalha sozinho e a média empresa já madura em ERP. No domínio escolhido, o usuário típico é o contador ou o assistente de um escritório que atende essas empresas.

> **Nota metodológica**
>
> As seções 3.1 a 3.3 apresentam estatísticas de fontes oficiais. A seção 3.4 apresenta inferências e projeções da equipe construídas sobre esses dados: hipóteses argumentadas, não fatos estabelecidos.

3.4 Predições sobre o público-alvo

O público cresce por substituição geracional

Entre empreendedores de até 34 anos, o uso de aplicativos de gestão já chega a 57%, contra 47% na média geral. A janela de encaixe do produto não se fecha: abre-se progressivamente.

O mercado tende a pular a era do ERP tradicional

Os 53% de pequenos negócios que nunca adotaram software de gestão dificilmente passarão por uma fase intermediária de ERP de menus. O concorrente direto não é um ERP estabelecido: é a planilha e a conversa informal.

A adoção será decidida por confiança, não por capacidade técnica

A retenção deve se correlacionar mais com a qualidade do fluxo de confirmação do que com a quantidade de funcionalidades. Por isso o card de confirmação (Revisar → segurar) recebeu a maior parte do esforço de engenharia e de testes do projeto.

O uso tenderá a ser móvel, fragmentado e falado

Um público que já administra fornecedores e clientes por mensagem de voz tende a tratar a entrada por voz como expectativa natural. A interface foi construída responsiva (360 px sem rolagem horizontal); a voz fica como trabalho futuro.

A interface estrutura o dado que hoje não existe

Cada intenção do usuário vira uma chamada de ferramenta tipada e auditada. O sistema estrutura o dado como efeito colateral do uso comum, o que reabre, no futuro, a camada preditiva descartada no início.

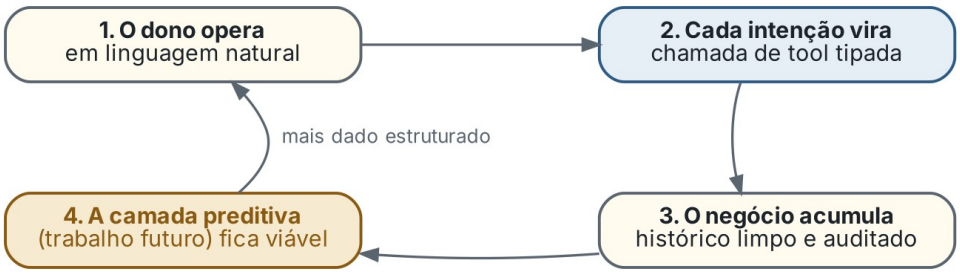


Figura 3 — Ciclo pelo qual a interface conversacional viabiliza, no futuro, a camada preditiva do anteprojeto.

Risco previsto: fadiga de confirmação

É esperado que o usuário passe a confirmar sem revisar. O sistema responde com fricção proporcional ao risco: a etapa **Revisar** obriga a olhar o formulário antes de o botão de segurar existir; a borda do card muda de cor conforme o risco (escrita ou exclusão); restaurar ou excluir de vez exige a senha de um administrador; e a ação em massa tem teto de 25 itens por plano.

Síntese

O público-alvo não se define apenas pelo porte da empresa, mas por um perfil cognitivo: gestores cuja alfabetização digital ocorreu em interfaces de conversa, administrando negócios complexos demais para a planilha e simples demais para um ERP.

4. Visão geral do sistema construído

Esta seção resume o que existe no repositório ao fim da construção (Fases 0 a 5) e da reconciliação dos testes manuais (Fase 6), na revisão `c641002` da branch `main`.

Tabela 3 — O sistema em números

Indicador	Valor	Observação
Tools (ferramentas) no catálogo do agente	38	7 consultas, 6 cadastros, 6 atualizações, 1 conciliação em lote, 6 exclusões, 6 restaurações, 6 exclusões definitivas
Entidades de domínio	6	Lançamento, categoria de lançamento, cliente, fornecedor, funcionário, nota fiscal
Papéis de acesso	3	leitura < operador < admin
Migrations	16	MySQL em desenvolvimento; as 13 primeiras compiladas também para PostgreSQL
Testes automatizados	475 casos	39 arquivos, 397 métodos, 4.335 asserções, todos verdes (seção 12)
Rotas autenticadas	16	Todas atrás de `auth` e CSRF; 15 também passam pela varredura de propostas vencidas
Código de aplicação (app/)	≈ 6.300 linhas	O `AiAgentService` sozinho tem ≈ 2.240 linhas; os testes somam ≈ 7.900

4.1 Stack tecnológica

Tabela 4 — Stack tecnológica

Camada	Tecnologia	Por quê
Backend	Laravel 13 sobre PHP 8.4 (8.4.27 ou mais)	Framework MVC maduro, com Form Requests, Eloquent, rate limiting e autenticação nativos; regra do projeto: 100% PHP
Interface	Blade, Tailwind CSS 4 e `workspace.css` (compilados pelo Vite), JavaScript puro	Sem framework de front-end: o HTML é sempre escrito pela equipe, nunca gerado pela IA. Fontes (IBM Plex, Sora) servidas do próprio projeto
Banco de dados	MySQL (Laragon) em desenvolvimento e na demonstração	A rede do SENAI bloqueia o protocolo PostgreSQL; a migração para Supabase (PostgreSQL, região de São Paulo) fica para quando o projeto sair de protótipo
IA	Microsoft Foundry, Chat Completions API com `tools`/`tool_choice`	Chamada HTTP REST única e stateless, sem SDK e sem Agent Service (threads/runs): menos complexidade e menos exposição a uma API em transição
Testes	PHPUnit, SQLite em memória, provedor de IA simulado (`Http::fake`)	A suíte inteira roda offline, sem custo de API e de forma determinística

Camada	Tecnologia	Por quê
Vitrine	Three.js (apenas na página pública, antes do login)	Farol 3D que gira com a rolagem; a área de trabalho não tem animação

4.2 Papéis e permissões

O controle de acesso é **por ferramenta**, não só por tela: cada tool declara um papel mínimo, e a hierarquia é `leitura < operador < admin`. Papel desconhecido ou ausente nega tudo (falha fechada). O papel e o tenant (a empresa) vêm sempre do usuário logado.

Tabela 5 — Matriz de permissões (RBAC por ferramenta)

Ação	leitura	operador	admin
Consultar lançamentos, categorias, clientes, fornecedores, notas e funcionários	Sim	Sim	Sim
Ver salário (em R$) no card de funcionário	Não	Não	Sim (só no card)
Criar e atualizar lançamento, categoria, cliente, fornecedor e nota fiscal	Não	Sim	Sim
Conciliar lançamentos em lote (até 25 por plano)	Não	Sim	Sim
Criar e atualizar funcionário	Não	Não	Sim
Consultar a lixeira	Não	Sim	Sim
Excluir (mover para a lixeira)	Não	Não	Sim
Restaurar da lixeira (com senha de um admin)	Não	Sim (funcionário: só admin)	Sim
Excluir de vez um registro arquivado (com senha)	Não	Não	Sim
Consultar e exportar a auditoria	Não	Não	Sim
Exportar CSV de uma consulta	Sim	Sim	Sim

4.3 Casos de uso por papel

O diagrama de casos de uso abaixo é a mesma matriz vista do lado de quem usa: cada papel herda o que o papel abaixo dele pode fazer e acrescenta as suas ações. As cores seguem a sinalização de risco da interface: âmbar para escrita, vermelho para exclusão e azul para a auditoria, que é só leitura mas exclusiva do administrador.

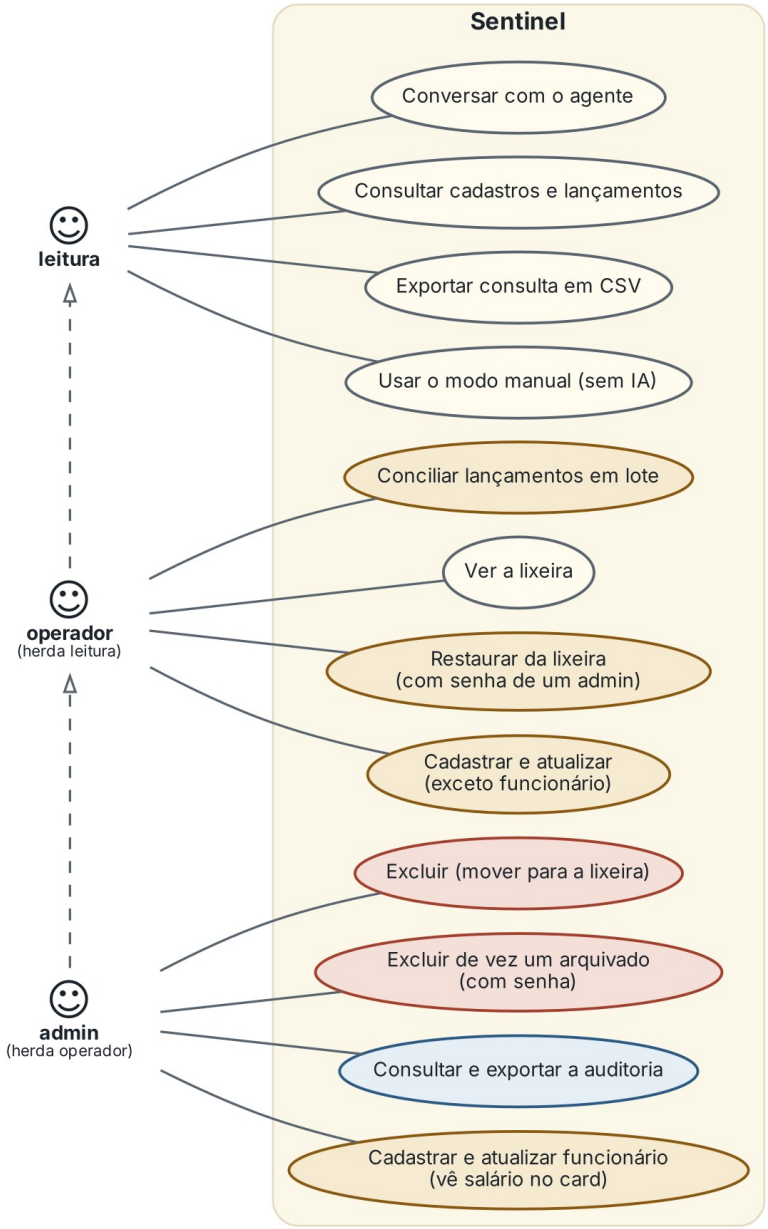


Figura 4 — Casos de uso do Sentinel por papel (a seta tracejada indica herança de permissões).

4.4 Telas

Tabela 6 — Telas do sistema

Tela	Quem vê	Para que serve
`/` (vitrine)	Público	Apresentação do produto; único botão leva ao login
`/login`	Público	Login por sessão; 5 tentativas por minuto por e-mail + IP
`/agente`	Todos os papéis	Conversa com o agente, tabelas de consulta, card de confirmação e card de plano (lote)
`/cadastros`	Todos os papéis	Modo manual sem IA: listar, buscar, cadastrar, editar e exportar

Tela	Quem vê	Para que serve
`/lixeira`	operador e admin	Itens na lixeira e arquivados; restaurar e excluir de vez
`/auditoria`	admin	Histórico filtrável e paginado; detalhe com proposto × confirmado; exportação em CSV
Telas de erro	Todos	401, 403, 404, 419, 429, 500 e 503 em português, com o que fazer

5. Arquitetura

O princípio arquitetural inegociável do projeto é que **o modelo de linguagem nunca toca o banco de dados**. O LLM é uma camada de decisão: escolhe, entre ferramentas pré-registradas pelo backend, qual invocar e com quais argumentos. Quem executa (valida, grava, audita) é o próprio Laravel, de forma inteiramente determinística. A mesma regra vale para a tela: a IA nunca gera HTML; ela devolve dados, e a interface monta componentes que a equipe escreveu.

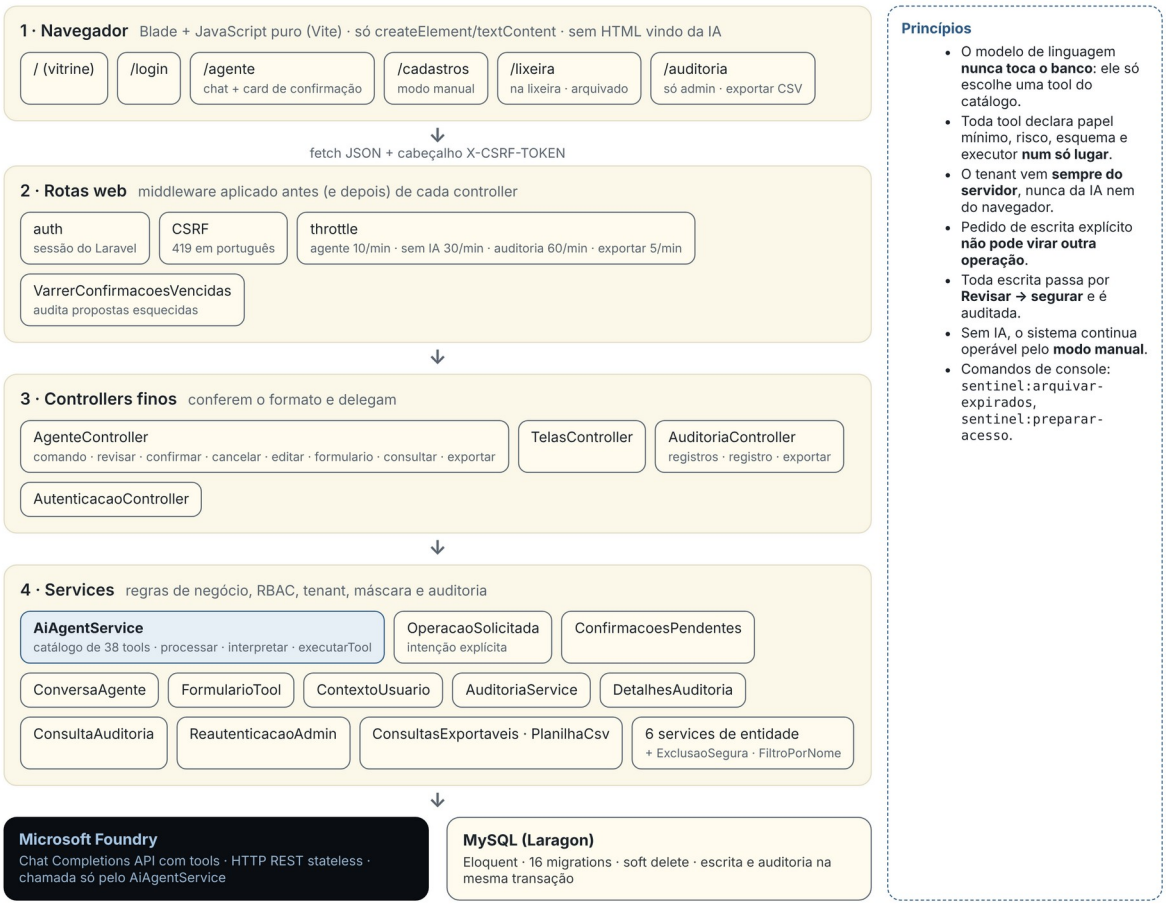


Figura 5 — Arquitetura em camadas do Sentinel e os princípios que a regem.

5.1 As camadas

Tabela 7 — Camadas e responsabilidades

Camada	Responsabilidade	Determinística?
Intenção (LLM no Microsoft Foundry)	Interpretar a linguagem natural e escolher uma tool do catálogo, com argumentos	Não: é a única camada não determinística, e por isso tudo o que sai dela é tratado como dado não confiável

Camada	Responsabilidade	Determinística?
Orquestração (`AiAgentService` + `OperacaoSolicitada`)	Reconhecer pedidos de escrita explícitos antes da IA; montar o prompt, o histórico e a lista de tools; validar a resposta do modelo contra o catálogo, o esquema e a operação pedida; decidir se executa (leitura) ou propõe (escrita)	Sim
Ferramentas (catálogo de 38 tools)	Cada tool declara papel mínimo, risco, esquema JSON, regras de validação e executor	Sim
Execução (services, Form Requests, Eloquent)	Regras de negócio, validação, tenant, transação e auditoria	Sim
Apresentação (Blade + JS)	Montar tabela, formulário, card, plano e mensagens a partir do JSON; só `textContent`	Sim

5.2 Componentes principais

Tabela 8 — Componentes e responsabilidades

Classe	Papel no sistema
`AiAgentService`	Única porta para a IA. Contém o catálogo de tools (fonte única: o que não está nele não é oferecido nem executável), `processar`/`interpretar`, `executarTool` (RBAC + tenant + transação + auditoria), `revisar`, `abrirFormulario`, `consultarManual`, `exportar` e `auditarDesfecho`
`OperacaoSolicitada`	Reconhece, no começo da mensagem, um verbo de escrita (cadastrar, alterar, excluir, excluir de vez, restaurar) e a entidade. Com isso o servidor nega antes da IA quem não tem permissão e força a IA a usar só a tool pedida
`ConfirmacoesPendentes`	Estado das propostas na sessão: rascunho (5 min), token de segurar (válido de 2 a 12 s), uso único atômico (`Cache::add`), ids congelados do lote, varredura de vencidas e tempo da IA
`FormularioTool`	Converte as regras do Form Request nos campos do formulário do card (tipo, obrigatório, opções, vínculos, condições, máscara)
`ConversaAgente`	Memória curta da conversa: até 16 mensagens e 8.000 caracteres, só texto, isolada por usuário e tenant, higienizada; marcadores internos da interface não voltam para a IA
`ContextoUsuario`	Usuário, papel e tenant do usuário logado; lança exceção se faltar qualquer um (falha fechada)
`AuditoriaService`	Ponto único de gravação em `audit_logs` e fonte única da máscara LGPD (CPF, salário, documento)
`DetalhesAuditoria`	Escopo por requisição para campos extras da auditoria: origem, proposto, quem autorizou, tempo da IA, valores anteriores e id do lote
`ConsultaAuditoria`	Leitura da auditoria: só admin (a negação também é auditada), por tenant, filtros, 25 por página, detalhe mascarado e exportação em CSV com teto de 5.000 linhas

Classe	Papel no sistema
`ReautenticacaoAdmin`	Confere a senha de um admin do mesmo tenant, com limites atômicos de tentativas
`ConsultasExportaveis` e `PlanilhaCsv`	Guardam a consulta na sessão (o navegador só conhece um id) e geram o CSV seguro
`ExclusaoSegura` e `FiltroPorNome` (traits)	Excluir com bloqueio por dependência, restaurar e excluir de vez; busca por nome em que `%` e `_` digitados são literais
Services de entidade	`LancamentoService`, `CategoriaLancamentoService`, `ClienteService`, `FornecedorService`, `FuncionarioService`, `NotaFiscalService`, todos com `int $tenantId` obrigatório
`AgenteController`	Rotas do agente: comando, revisar, confirmar, editar, cancelar, formulário, consultar e exportar
`TelasController`, `AuditoriaController`, `AutenticacaoController`	Telas de Cadastros, Lixeira e Auditoria; JSON e CSV da auditoria; login e logout (o logout audita as propostas abertas)
`VarrerConfirmacoesVencidas` (middleware)	Depois de cada requisição autenticada, audita como "expirado" as propostas esquecidas
Comandos Artisan	`sentinel:arquivar-expirados` (RF11, agendado diariamente) e `sentinel:preparar-acesso` (usuários locais)

5.3 Rotas

Tabela 9 — Rotas autenticadas e seus limites

Método e rota	Limite	Função
POST `/login`	5/min por e-mail + IP	Autenticação por sessão; regenera a sessão
POST `/agente/comando`	10/min por usuário	Envia a mensagem ao agente (única rota que chama a IA)
POST `/agente/revisar`	30/min	Valida o formulário editado sem gravar e emite o token de segurar
POST `/agente/confirmar`	uso único do token	Executa a proposta revisada (valores vindos da sessão)
POST `/agente/cancelar` e `/agente/editar`	30/min	Cancela (auditado) ou volta ao formulário (anula o token)
POST `/agente/formulario`	30/min	Abre o formulário de criar ou atualizar sem IA
POST `/agente/consultar`	30/min	Modo manual: consulta sem IA
POST `/agente/exportar`	5/min	Gera o CSV de uma consulta, refeita no servidor
GET `/auditoria/registros[/{id}]`	60/min	Lista e detalhe da auditoria em JSON (só admin)

Método e rota	Limite	Função
GET `/auditoria/exportar`	5/min (mesmo balde do exportar)	CSV da auditoria com os filtros da tela, até 5.000 linhas (só admin)
GET `/agente`, `/cadastros`, `/lixeira`, `/auditoria`	—	Telas (os dados vêm das rotas JSON)

5.4 Decisões arquiteturais

Tabela 10 — Decisões de arquitetura e alternativas avaliadas

Decisão	Alternativa descartada	Motivo
Chat Completions API, chamada stateless	Agent Service / Assistants API (threads e runs)	Menor complexidade no prazo do TCC e menor exposição a uma API em transição. A confirmação humana é implementada pela equipe, não por recurso do provedor. Toda a integração fica em uma classe: trocar de provedor é mexer em um ponto só
Catálogo estático de tools (whitelist)	MCP para as ferramentas internas	MCP favorece a descoberta dinâmica de ferramentas, o oposto da whitelist que sustenta a governança. Fica como candidato para operar serviços externos no futuro
Intenção explícita reconhecida no servidor	Confiar só na escolha do modelo	Nos testes manuais o modelo trocava "exclua" por uma consulta e "excluir de vez" por exclusão lógica. Um vocabulário fixo de verbos e entidades, conferido antes e depois da chamada, impede que a operação pedida vire outra
Engenharia de prompt + tool calling	Fine-tuning de modelo	Exigiria dados rotulados e tempo de treino incompatíveis com o prazo; o tool calling já entrega a precisão necessária
Consultas estruturadas ao banco	RAG (busca semântica)	As operações centrais são consultas estruturadas, não busca em texto. RAG fica como extensão possível para um modo de ajuda
Catálogo fechado de componentes de tela	IA gerando HTML	Previsibilidade visual, auditabilidade, acessibilidade e eliminação de XSS vindo do modelo
Ação em massa como plano com teto	Lote livre, executado item a item	Um plano fechado (ids congelados, no máximo 25, uma transação) mantém as garantias da escrita unitária e combate a fadiga de confirmação
MySQL local	Supabase PostgreSQL já no TCC	A rede do SENAI descarta o protocolo PostgreSQL. As migrations foram compiladas para PostgreSQL; o checklist de migração está no `CLAUDE.md`

5.5 Resiliência quando a IA falha

O cenário mais frequente não é a falta total de internet, mas a API de IA lenta, fora do ar ou recusando a chamada. Nesse caso o Sentinel **degrada sem travar**:

- a chamada tem timeout de 30 s no servidor (o navegador desiste em 35 s), e toda falha vira uma mensagem amigável no contrato de erro, com um `correlation_id` para localizar o log;
- o **modo manual** (`/cadastros` e `/lixeira`) faz todo o CRUD sem chamar a IA: listar, buscar, cadastrar, editar, exportar, restaurar e excluir de vez;
- a governança é a mesma nos dois caminhos: o modo manual usa o mesmo `executarTool()` e o mesmo fluxo Revisar → segurar, e a auditoria registra a origem (`agente`, `formulario` ou `manual`).

A fila local de comandos prevista na versão 1.0, que guardaria a intenção para reapresentá-la quando a IA voltasse, **não foi implementada**: o modo manual cobre o requisito de continuidade (RNF05) com menos complexidade. A ausência total de conectividade (PWA ou aplicativo com banco local) continua como trabalho futuro.

5.6 Relações entre os componentes

O diagrama de camadas mostra **onde** cada peça fica; o diagrama de relações a seguir mostra **quem chama quem**. Ele é um diagrama de dependências no estilo de um diagrama de classes da UML: cada caixa é uma classe (ou um grupo de classes do mesmo tipo), e cada seta parte de quem usa e aponta para quem é usado. Os métodos listados dentro das caixas são os públicos mais importantes.

Quatro regras de arquitetura ficam visíveis no desenho:

- **Uma só seta chega à IA.** Só o `AiAgentService` fala com o Microsoft Foundry. Nenhum controller, view ou outro service monta uma chamada ao provedor.
- **Uma só fonte de "quem está agindo".** Todas as setas que precisam de usuário, papel ou tenant terminam no `ContextoUsuario`. Nenhuma classe lê `Auth::user()` por conta própria para decidir permissão.
- **Um só ponto de gravação da auditoria.** O `AiAgentService` e o `ConsultaAuditoria` gravam sempre pelo `AuditoriaService`, que aplica a máscara LGPD e lê os campos extras do `DetalhesAuditoria`.
- **Os controllers não falam com o banco.** As setas dos controllers vão para services; só services e models chegam ao Eloquent.

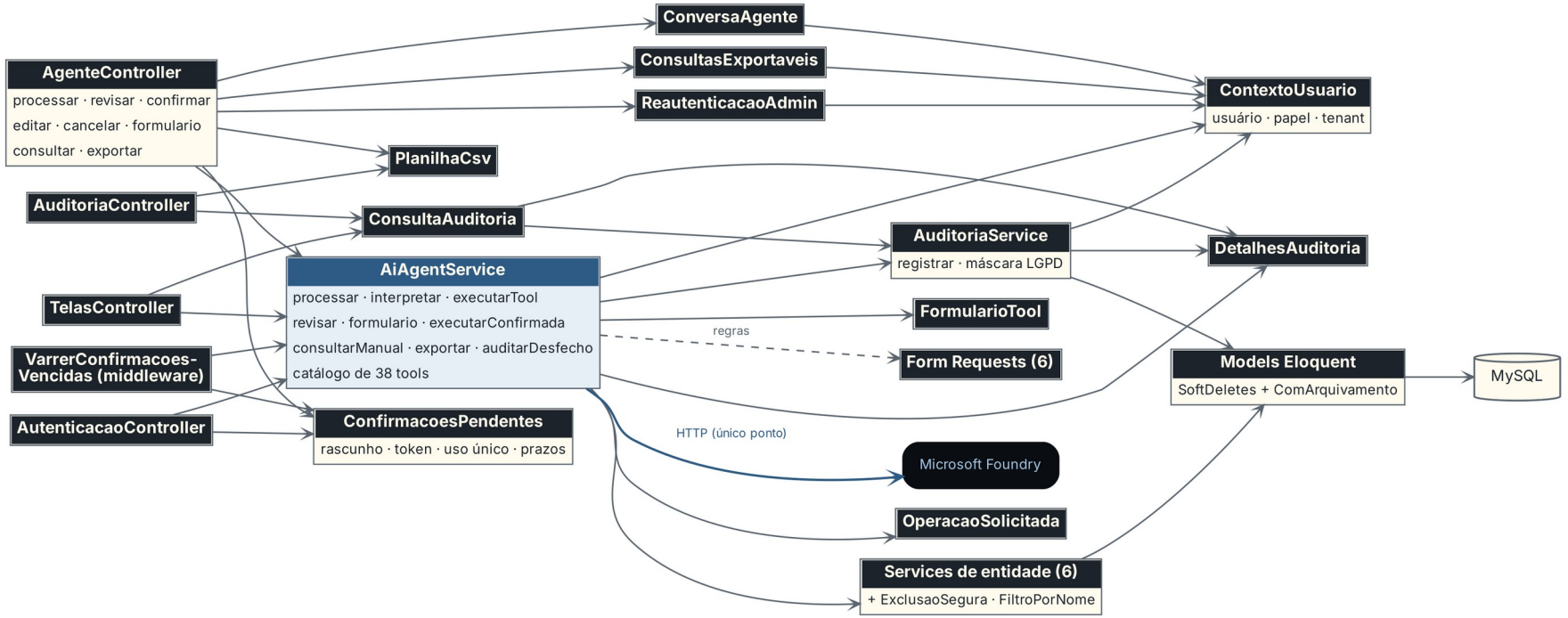


Figura 6 — Relações de dependência entre os componentes do Sentinel (a seta parte de quem usa).

6. O agente

O agente é o conjunto formado pelo modelo de linguagem no Microsoft Foundry e pela camada de orquestração (`AiAgentService` e `OperacaoSolicitada`). O modelo decide; o servidor confere tudo o que o modelo decidiu antes de agir.

6.1 O catálogo de ferramentas

O catálogo é montado por quatro construtores (`definirTool`, `definirEscrita`, `definirAtualizacao` e `definirLixeira`) e é a **fonte única** do que existe: a lista enviada à IA, o RBAC, o formulário do card, as colunas da tabela e o executor saem da mesma definição. Uma tool que não está no catálogo não é oferecida ao modelo e, se o modelo a inventar, não é executada.

Tabela 11 — Catálogo de tools do agente

Grupo	Tools	Risco	Papel mínimo
Consultas (7)	`consultar_lancamentos`, `consultar_categorias_lancamento`, `consultar_clientes`, `consultar_fornecedores`, `consultar_funcionarios`, `consultar_notas_fiscais`, `consultar_lixeira`	leitura	leitura (lixeira: operador)
Cadastros (6)	`criar_lancamento`, `criar_categoria_lancamento`, `criar_cliente`, `criar_fornecedor`, `criar_funcionario`, `criar_nota_fiscal`	escrita	operador (funcionário: admin)
Atualizações (6)	`atualizar_status_lancamento`, `atualizar_categoria_lancamento`, `atualizar_cliente`, `atualizar_fornecedor`, `atualizar_funcionario`, `atualizar_nota_fiscal`	escrita	operador (funcionário: admin)
Ação em massa (1)	`conciliar_lancamentos_em_lote`: período obrigatório e categoria opcional; status fixo pendente → conciliado; até 25 por plano	escrita	operador
Exclusões (6)	`excluir_<entidade>`: move para a lixeira se nada depender do registro	exclusão	admin
Restaurações (6)	`restaurar_<entidade>`: tira da lixeira; exige senha de um admin no card	escrita	operador (funcionário: admin)
Exclusões definitivas (6)	`excluir_definitivo_<entidade>`: só de registro arquivado; exige senha de admin	exclusão	admin

6.2 O que vai e o que nunca vai para a IA

A fronteira de dados com o provedor de IA é tratada como fronteira de segurança. O modelo recebe o mínimo para decidir **qual** ferramenta usar; ele nunca vê os dados que a ferramenta devolve.

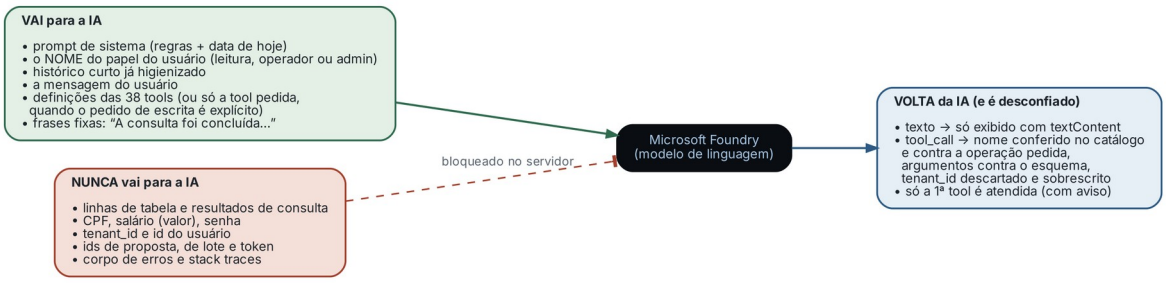


Figura 7 — Fronteira de dados entre o Sentinel e o provedor de IA.

O prompt de sistema fixa o papel do assistente (contabilidade de micro e pequenas empresas), a data de hoje (para resolver "ontem" e "hoje"), o **nome do papel** do usuário (leitura, operador ou admin, para o modelo não sugerir o que o usuário não pode fazer; a decisão continua sendo do servidor) e as regras de conduta: usar só as ferramentas disponíveis, não inventar ferramentas nem dados, preservar a operação pedida (cadastro, alteração e exclusão nunca viram consulta), recusar perguntas de conhecimento geral, perguntar quando faltar um campo obrigatório, não pedir confirmação em texto (a tela já faz isso), lembrar que proposta não é escrita executada, e **nunca pedir nem aceitar senha no chat**. O tenant e o id do usuário continuam fora do prompt.

6.3 Memória da conversa

`ConversaAgente` guarda na sessão as últimas 16 mensagens (no máximo 8.000 caracteres), só texto, ligadas ao usuário e ao tenant: se um deles mudar, a memória é apagada, e o logout a limpa. Antes de guardar, o texto é higienizado: mensagens com termos de credencial são omitidas, CPF e CNPJ viram "[documento omitido]" e o valor que vem depois de "salário" ou "remuneração" vira "[valor omitido]". Resultados de consulta e propostas entram no histórico apenas como frases fixas, sem nenhum dado, e são marcados como **internos**: eles não voltam para a IA como se fossem respostas do assistente, porque, nos testes, o modelo passou a copiar essas frases em vez de chamar a tool no cadastro seguinte.

6.4 Como o pedido e a resposta do modelo são conferidos

1. Antes de chamar a IA, `OperacaoSolicitada` procura no começo da mensagem um verbo de escrita e uma entidade ("exclua o cliente...", "cadastre um fornecedor..."). Se encontrar exatamente uma tool compatível e o papel não permitir, o pedido é **negado e auditado sem chamar o provedor**. Se permitir, a IA recebe **só aquela tool**, com `tool_choice` forçado.
2. A resposta HTTP precisa ser bem-sucedida; caso contrário, vira erro classificado (tabela abaixo).
3. A mensagem precisa ter o formato esperado (objeto, `tool_calls` em lista, `finish_reason` coerente). Resposta truncada ou filtrada pelo provedor vira erro próprio.
4. Se o pedido era explícito e o modelo devolveu outra tool, a resposta é `AI_INTENT_MISMATCH`: nada é executado, e a divergência é auditada como "negado", só com os **nomes** dos argumentos.

5. Se for só texto, é exibida como texto (no navegador, sempre por `textContent`). O log guarda só o tempo, nunca o conteúdo.
6. Se houver chamadas de tool, **todas** são validadas antes de executar a primeira: JSON válido, objeto, e nenhuma chave fora do esquema da tool. `tenant_id` é sempre removido.
7. Só a primeira chamada é atendida; se havia mais, a resposta traz um aviso pedindo que o usuário peça uma de cada vez.
8. Tool de leitura é executada na hora (com RBAC, tenant e auditoria). Tool de escrita passa pelo RBAC e vira **proposta**: nada é gravado nesse momento.

Tabela 12 — Contrato de erro do agente (todas as respostas levam também um correlation_id)

Situação	Categoria	Código	HTTP
Endpoint, chave ou deployment não configurado	`provider_configuration_error`	`AI_PROVIDER_UNAVAILABLE`	503
Falha de conexão	`provider_connection_error`	`AI_PROVIDER_UNAVAILABLE`	503
Tempo esgotado (30 s)	`provider_timeout`	`AI_PROVIDER_TIMEOUT`	504
Provedor recusou a credencial (401/403)	`provider_authentication_error` / `..._authorization_error`	`AI_PROVIDER_UNAVAILABLE`	502
Limite do provedor (429) ou erro 5xx	`provider_rate_limit` / `provider_http_error`	`AI_PROVIDER_UNAVAILABLE`	503
Resposta fora do formato	`provider_response_error`	`AI_PROVIDER_RESPONSE_INVALID`	502
Resposta truncada / filtrada	`provider_response_error`	`AI_RESPONSE_TRUNCATED` / `AI_CONTENT_FILTERED`	502
O modelo trocou a operação pedida	`intent_mismatch`	`AI_INTENT_MISMATCH`	422
Falha técnica ao executar uma tool	`tool_error`	`AI_TOOL_ERROR`	500

6.5 Tempo de resposta

A chamada ao provedor é cronometrada com `hrtime` e o tempo é gravado em `audit_logs.tempo_ia_ms` na linha do desfecho de cada pedido; o detalhe da auditoria mostra "Tempo da IA". Isso torna o RNF04 **mensurável**. O limite de 10 s, porém, **não é imposto** pelo código (o teto técnico é o timeout de 30 s).

Os números disponíveis hoje vêm da bateria de testes manuais de 28/10, que mediu o tempo HTTP de cada pedido (do envio até a resposta no navegador, um pouco acima do `tempo_ia_ms`):

Tabela 13 — Tempo HTTP medido na bateria manual de 28/10 (p90 pelo método nearest-rank)

Grupo de pedidos	n	Média	Mediana	p90	Mín.	Máx.
Papel leitura	17	2,78 s	2,73 s	4,03 s	0,52 s	4,05 s
Papel operador (escritas e negativas)	41	3,15 s	2,77 s	3,70 s	0,84 s	9,07 s

Grupo de pedidos	n	Média	Mediana	p90	Mín.	Máx.
Papel admin	17	2,90 s	2,61 s	4,23 s	2,22 s	4,47 s
Total	77	3,00 s	2,70 s	4,05 s	0,52 s	9,07 s

Nenhum pedido passou de 10 s no tempo HTTP; um passou de 10 s no tempo percebido pelo testador (10,9 s, num pedido de exclusão repetido). A decisão sobre o limite continua em aberto e depende de uma rodada com a Azure real gravando `tempo_ia_ms` no banco da demonstração (seção 15).

7. Fluxos da aplicação

Os fluxogramas desta seção descrevem o comportamento real do código. As cores seguem a sinalização da interface: âmbar para escrita pendente, verde para sucesso, vermelho para negação, recusa, cancelamento ou expiração, azul para decisões e fundo escuro para a IA. A seção termina com três diagramas de sequência, que mostram os mesmos fluxos do ponto de vista das classes.

7.1 F1: do comando à gravação

É o fluxo central do sistema. Toda escrita, venha do agente ou do modo manual, termina nas mesmas etapas de **Revisar** e **segurar**.

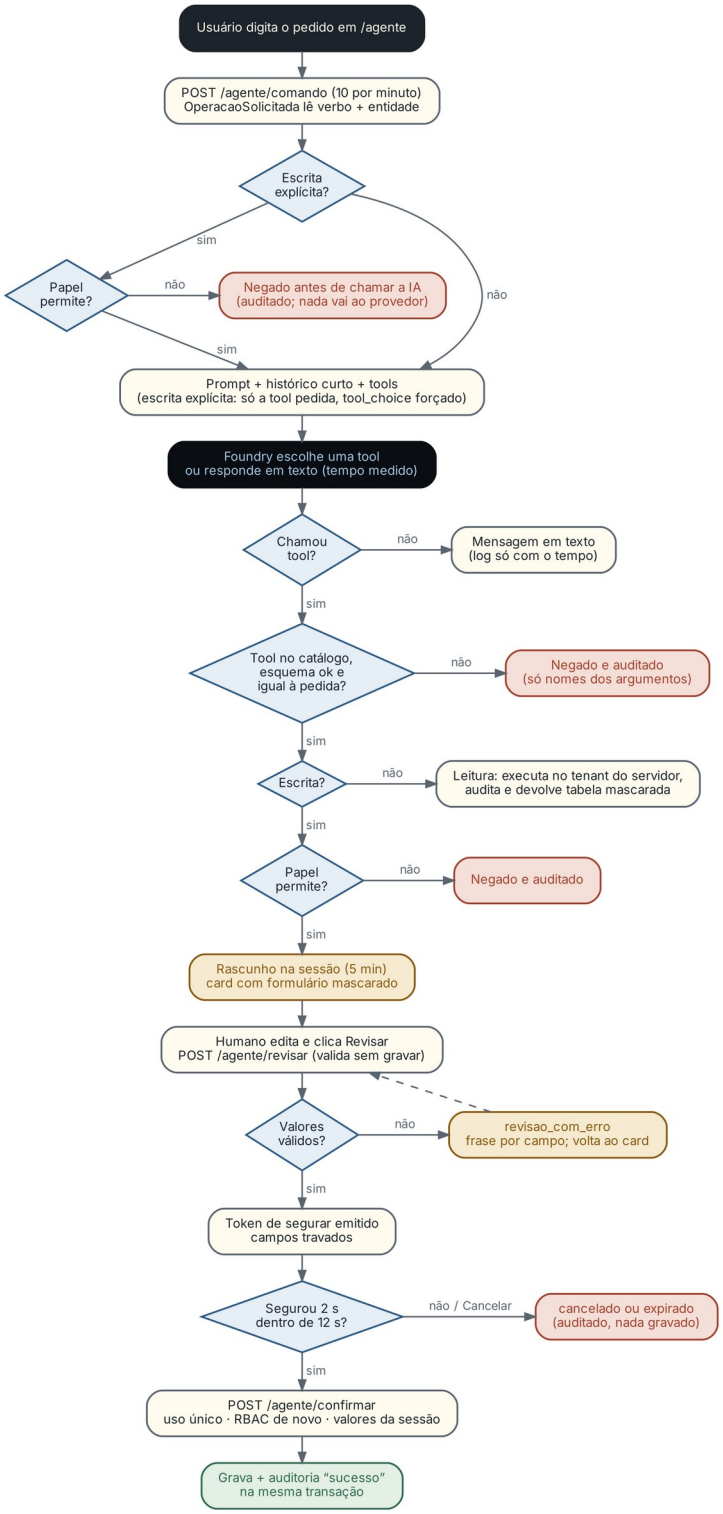


Figura 8 — F1: comando, intenção explícita, proposta, Revisar, segurar e confirmar.

1. O usuário digita o pedido em `/agente`, que chega a `POST /agente/comando` (limite de 10 por minuto).
2. `OperacaoSolicitada` procura um pedido de escrita explícito. Se houver e o papel não permitir, o pedido é negado e auditado ali mesmo, sem chamar a IA.

3. `AiAgentService::processar()` monta o prompt de sistema, o histórico curto e o catálogo de tools (ou só a tool pedida, forçada) e chama a Chat Completions API. O tempo da chamada é medido.
4. `interpretar()` confere a resposta (seção 6.4). Tool fora do catálogo, com argumento fora do esquema ou diferente da pedida é negada e auditada só com os **nomes** dos argumentos.
5. Tool de escrita: o RBAC confere o papel. Sem permissão, "negado" (auditado). Com permissão, a resposta é `confirmacao_pendente` com o formulário. O controller guarda a proposta crua na sessão como **rascunho** (5 minutos) e devolve ao navegador só os valores mascarados e um id de proposta.
6. O humano edita o formulário e clica em **Revisar** (`POST /agente/revisar`): RBAC, tenant e regras da tool são aplicados **sem gravar**. Com erro, o formulário volta com a frase de cada campo (auditado como `revisao_com_erro`, que não é desfecho). Sem erro, sai o **token** de segurar e os campos travam.
7. O humano **segura** Confirmar por 2 s, e o navegador envia `POST /agente/confirmar` com o token. O servidor exige uso único, espera mínima de 2 s e validade de 12 s; a tool e os valores vêm da sessão, nunca do corpo da requisição.
8. `executarTool()` refaz o RBAC, injeta o tenant do servidor e executa a tool e a auditoria "sucesso" **na mesma transação**.
9. Cancelar é auditado como "cancelado" ou "expirado" conforme o horário. Proposta abandonada é auditada como "expirado" pela varredura na requisição seguinte; no logout, as propostas abertas também recebem desfecho.

7.2 Estados de uma proposta

Cada proposta de escrita é uma pequena máquina de estados guardada na sessão. Ela termina em **exatamente um** desfecho, e cada desfecho é uma linha na auditoria.

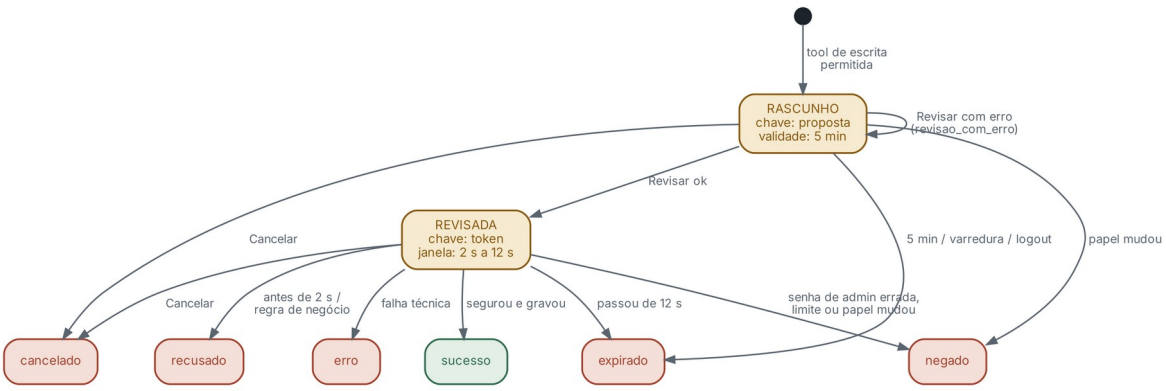


Figura 9 — Máquina de estados da proposta de escrita.

Tabela 14 — Resultados registrados na auditoria

Resultado	Final?	Quando acontece
`sucesso`	Sim	A escrita foi gravada (na mesma transação da auditoria)
`negado`	Sim	Falta permissão (papel); a senha de admin não conferiu ou o limite de tentativas estourou; ou a resposta da IA foi rejeitada (tool fora do catálogo ou operação diferente da pedida)

Resultado	Final?	Quando acontece
`recusado`	Sim	Regra de negócio (ex.: dependência, alteração concorrente, item do lote que mudou) ou confirmação antes da espera mínima de 2 s
`cancelado`	Sim	O humano clicou em Cancelar dentro do prazo
`expirado`	Sim	Passou do prazo (12 s do token ou 5 min do rascunho), por cancelamento tardio, varredura ou logout
`erro`	Sim	Falha técnica durante a execução; nada é gravado
`revisao_com_erro`	Não	O Revisar encontrou erro de validação; a proposta continua em rascunho

7.3 Segurar para confirmar

A confirmação usa o padrão **hold-to-confirm**: o botão Confirmar precisa ser pressionado e mantido por 2.000 ms. Enquanto ele é mantido, o preenchimento avança de 0% a 100% e o texto mostra "confirmando" e a porcentagem. Soltar antes do fim zera o botão e mostra "Você soltou antes do fim. Nada foi gravado.". O gesto funciona com mouse, toque e teclado (Enter ou Espaço mantidos).

- **Duas janelas de tempo:** depois do Revisar, a interface dá 10 s para segurar; o servidor aceita até 12 s (folga para a rede). Antes disso existe o prazo absoluto de 5 min do rascunho, que não reinicia ao editar. O segurar só começa se ainda couberem 2,5 s na janela.
- **O card não some calado:** quando o prazo passa, ele fica visível com "Confirmação expirada. Nada foi gravado. Peça de novo." e os botões desativados.
- **Dois canais sempre:** cor da borda (risco) + título + texto da etapa + anúncio para leitor de tela (`aria-live`) + popup com símbolo e texto ao final (RNF07).
- **Credenciais de administrador antes do Revisar:** nas tools que pedem senha, o e-mail e a senha do admin são digitados junto com a revisão, para caberem na janela de 12 s. Se o humano clicar em **Editar**, a senha é apagada e o card volta a pedir as credenciais, também em voz alta para o leitor de tela.
- **Cancelar é neutro** (não vermelho) e cancela com um clique; o vermelho fica para a ação destrutiva.

7.4 F2: atualizar um registro

A atualização começa pelo agente ("mude o telefone do cliente X") ou pelo botão **Editar** de uma linha de tabela, que abre o formulário sem chamar a IA. O servidor localiza o registro no tenant e protege contra alteração concorrente usando a data de atualização como versão.

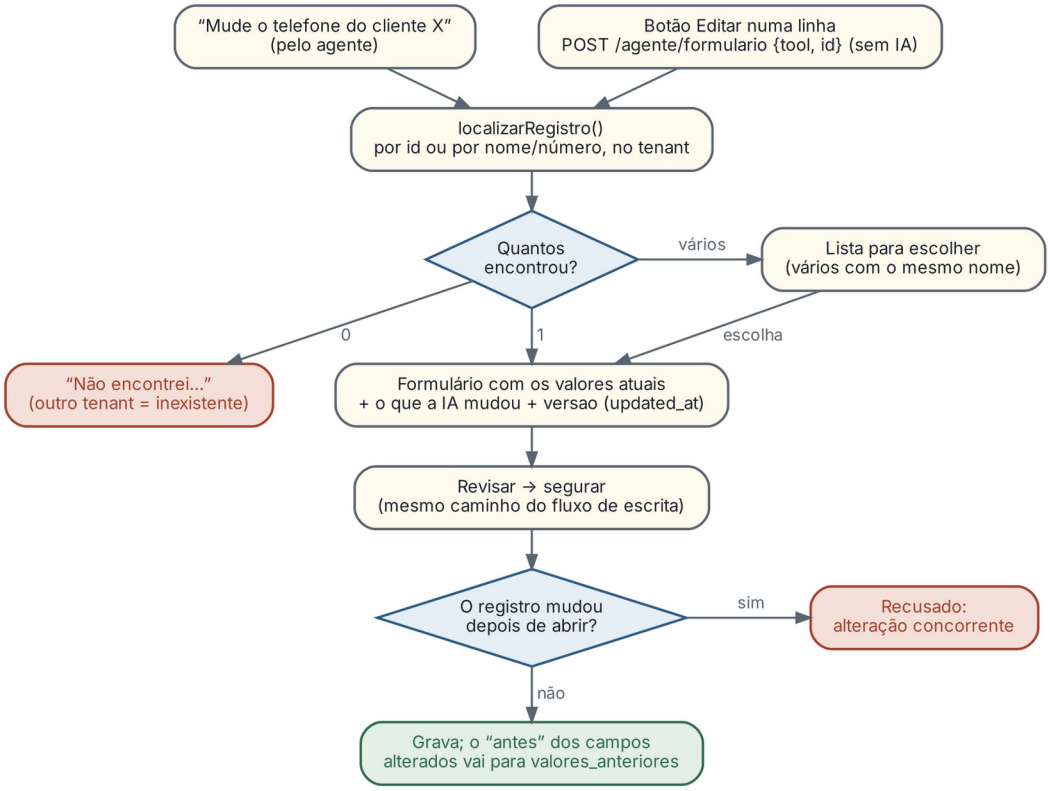


Figura 10 — F2: localizar o registro, abrir o formulário e gravar com controle de versão.

7.5 F3: lixeira, arquivamento e exclusão definitiva

Nenhum dado se perde por engano. A exclusão é um ciclo de vida com quatro estados, e só a última transição é irreversível, sempre por decisão manual de um administrador.

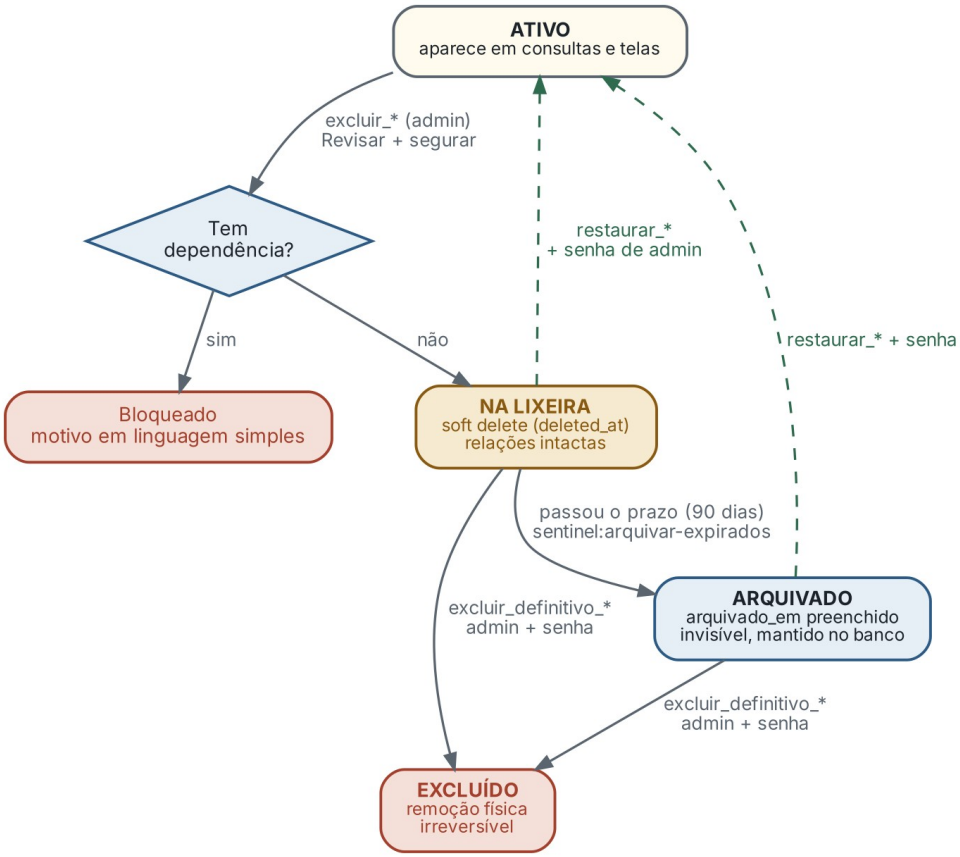


Figura 11 — Ciclo de vida de um registro: ativo, lixeira, arquivado e excluído.

- **Bloqueio por dependência (RF08):** antes de excluir, o sistema verifica as relações que impedem a exclusão e explica o motivo em linguagem simples ("Não é possível excluir: existem lançamentos vinculados a este registro."). A recusa aparece já no Revisar e é conferida de novo no segurar.
- **Lixeira (RF09):** soft delete; as relações continuam intactas. Prazo de retenção configurável (`SENTINEL_RETENCAO_LIXEIRA_DIAS`, padrão 90 dias).
- **Arquivamento (RF11):** passado o prazo, o comando agendado `sentinel:arquivar-expirados` marca o registro como arquivado. Ele some das telas, mas continua no banco, porque documentos fiscais e trabalhistas têm prazo legal de guarda de anos.
- **Restauração (RF10):** volta o registro com a data de cadastro e as relações originais; exige a senha de um admin do mesmo tenant, e a auditoria grava quem autorizou (`autorizado_por`). Restaurar um registro cujo "pai" está na lixeira é recusado, dizendo qual é o pai.
- **Exclusão definitiva (RF12):** só de registro arquivado, só por admin e com senha. Nenhuma rotina automática faz exclusão física.

A ordem das verificações na confirmação com senha foi desenhada para que o card **não sirva de verificador de senhas**: tudo o que não envolve segredo é conferido primeiro, e um usuário sem permissão recebe a mesma resposta com senha certa ou errada, sem gastar tentativa.

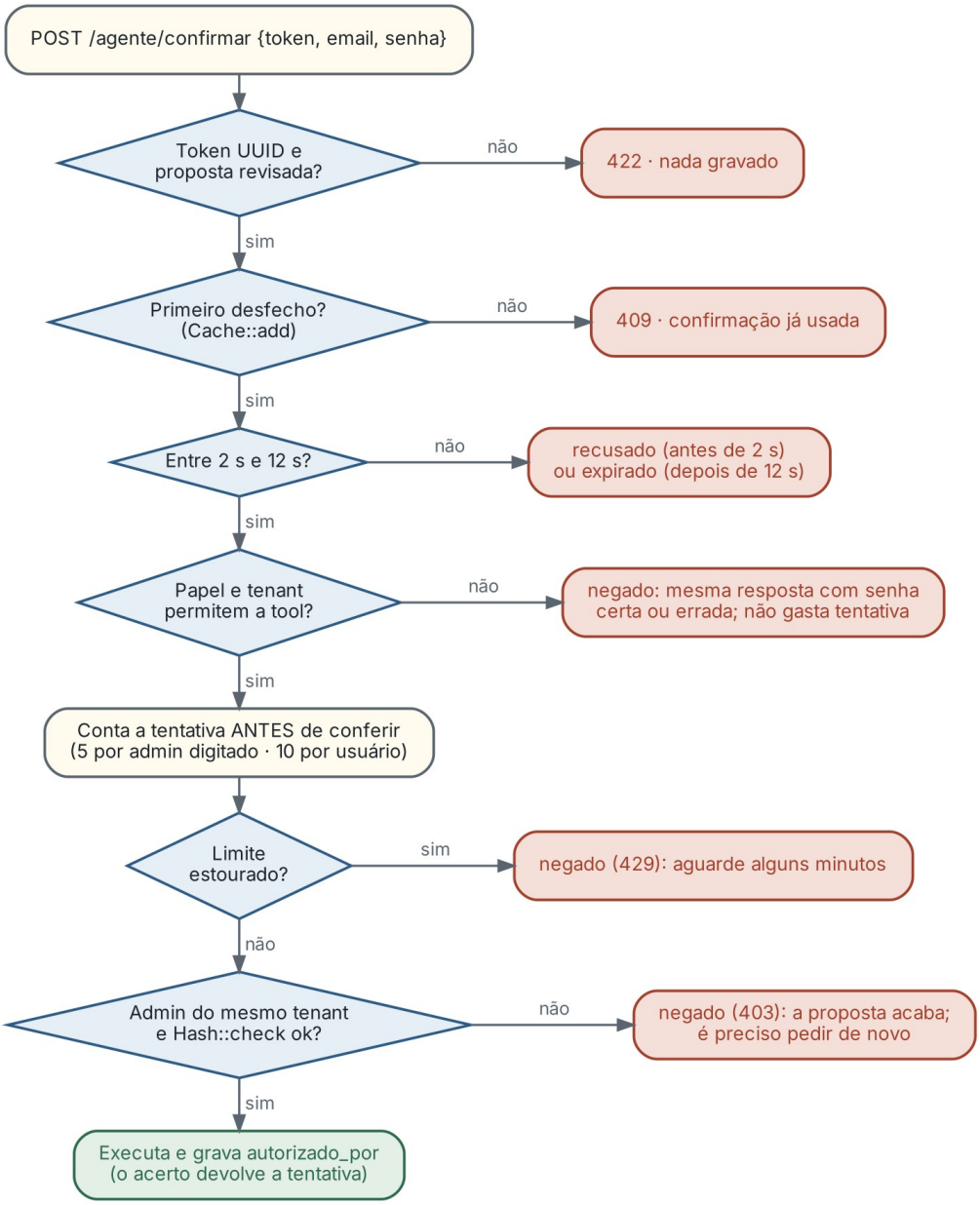


Figura 12 — Ordem das verificações em uma confirmação que exige senha de administrador.

7.6 F4: consultar e exportar

O navegador nunca diz ao servidor **o que** exportar: ele envia apenas um id. A consulta (tool e argumentos) fica guardada na sessão, e na exportação o servidor a refaz com o RBAC, o tenant e a máscara do momento. Assim, um usuário que perdeu permissão depois da consulta não consegue exportar, e um id adulterado não leva a dados de outro tenant.

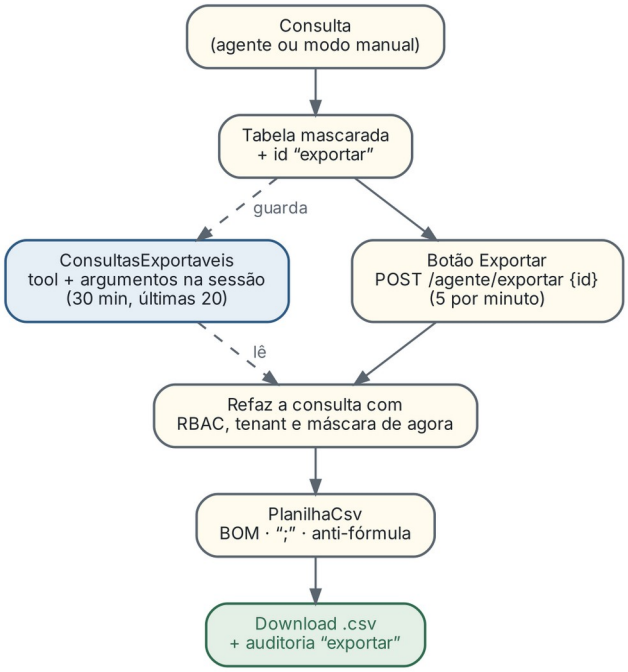


Figura 13 — F4: consulta, guarda da consulta na sessão e exportação segura em CSV.

A **auditoria** também pode ser exportada, pela tela `/auditoria`. Esse caminho é separado do agente e tem regras próprias:

- **Só admin, e o papel vem antes dos filtros:** quem não é admin recebe a mesma negação auditada, com filtro válido ou inválido. A validação nunca responde antes da permissão.
- **Mesmos filtros da tela, todas as páginas, só do tenant**; o CSV leva data, usuário, papel, ação, entidade, registro, resultado, origem e lote, mas **nunca os parâmetros**, para não reexpor dados.
- **Teto de 5.000 linhas:** acima disso o pedido é recusado (auditado como "recusado") pedindo um período menor. O arquivo nunca é cortado em silêncio.
- A própria exportação vira uma linha na auditoria ("N registros exportados").

7.7 F5: modo manual, sem IA

As telas Cadastros e Lixeira funcionam sem o agente. Elas chamam `POST /agente/consultar` com a tool e os filtros; o servidor aceita só tools de leitura do catálogo e confere os filtros contra o esquema da tool. "Novo" e "Editar" abrem o mesmo card de confirmação, sem IA. É assim que o sistema atende ao RNF05 quando a Azure está fora do ar.

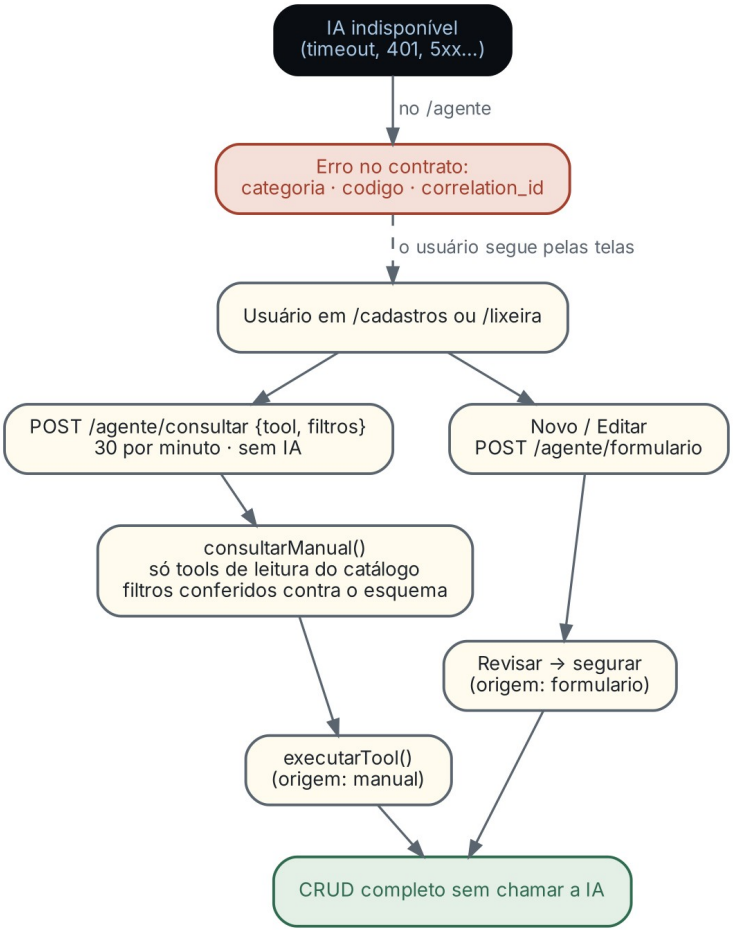


Figura 14 — F5: modo manual e degradação quando a IA está indisponível.

7.8 F6: conciliação em lote (RF03)

O RF03 pede ações em massa somente com confirmação explícita. A equipe implementou o padrão **planejar → o humano aprova → executar** em uma única tool, `conciliar_lancamentos_em_lote`, escolhida por ser a ação em massa mais comum num escritório de contabilidade: marcar como conciliados os lançamentos pendentes de um período.

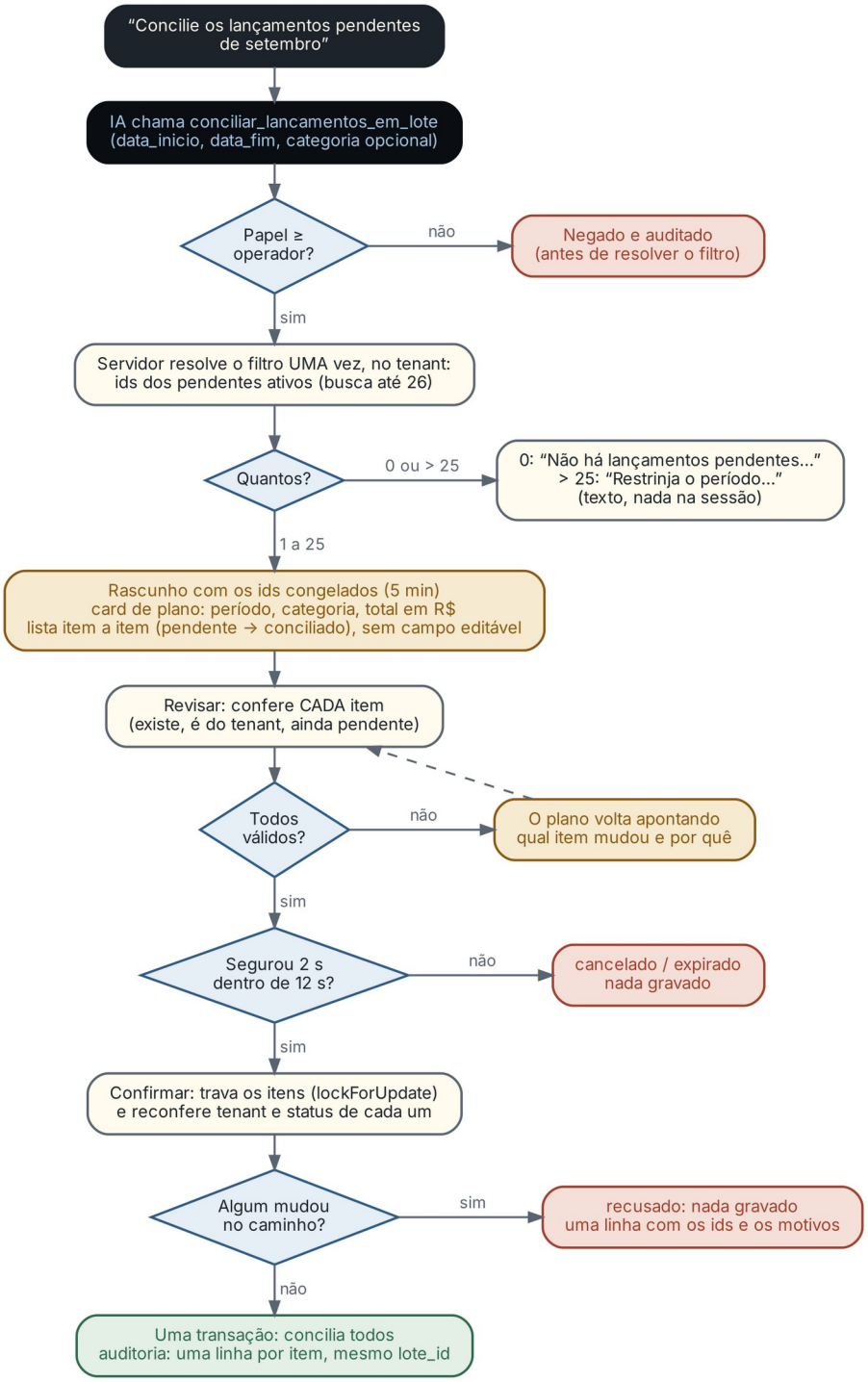


Figura 15 — F6: do pedido de conciliação ao plano, à revisão item a item e à gravação tudo ou nada.

- **O filtro é resolvido uma vez, no servidor.** A IA só informa o período (e, se quiser, a categoria). O servidor busca, no tenant, os lançamentos pendentes e ativos, até 26 deles. Nenhum: responde em texto. Mais de 25: recusa pedindo um filtro menor. **Nunca trunca** a lista.
- **Os ids ficam congelados no rascunho.** Revisar e Confirmar trabalham só com esses ids; o que o navegador mandar é ignorado. Assim, um lançamento novo que entrar no período depois do plano não é conciliado sem ter sido visto.

- **O card é um plano, não um formulário.** Mostra período, categoria, quantidade, valor total e a lista item a item (descrição, valor, data, pendente → conciliado), montada a partir do banco, sem nada vindo da IA e sem campo editável.
- **O Revisar confere cada item** (existe, é do tenant, ainda está pendente). Se um item mudou, o plano volta apontando qual e por quê.
- **O Confirmar trava os registros** (`lockForUpdate`), reconfere tenant e status de cada um e grava tudo ou nada, numa transação.
- **Auditoria:** uma linha por item, todas com o mesmo `lote_id` (UUID) e com o "antes" (pendente). Lote recusado ou com erro grava uma linha só, com os ids e os motivos.

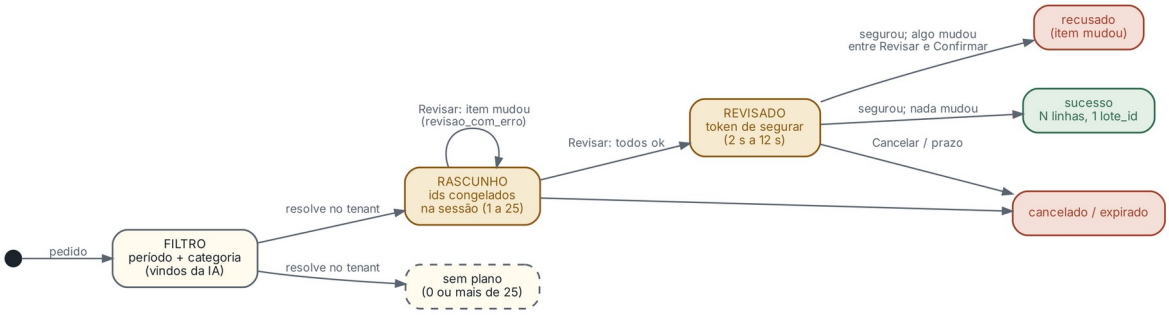


Figura 16 — Estados de um plano de conciliação em lote.

7.9 Diagramas de sequência

Os fluxogramas acima mostram as decisões; os diagramas de sequência a seguir mostram **a ordem das chamadas entre as classes** nos três fluxos mais importantes. Cada coluna é um participante (o navegador, uma classe ou um serviço externo), o tempo corre de cima para baixo, setas cheias são chamadas e setas tracejadas são retornos. As caixas coloridas são notas: vermelho para uma saída de negação, âmbar para uma ação humana e azul para uma regra que vale o tempo todo.

O primeiro diagrama cobre o pedido ao agente até a proposta chegar ao card. Repare que a `OperacaoSolicitada` é consultada **antes** do Foundry, e que o histórico guardado depois da resposta é só uma frase fixa.

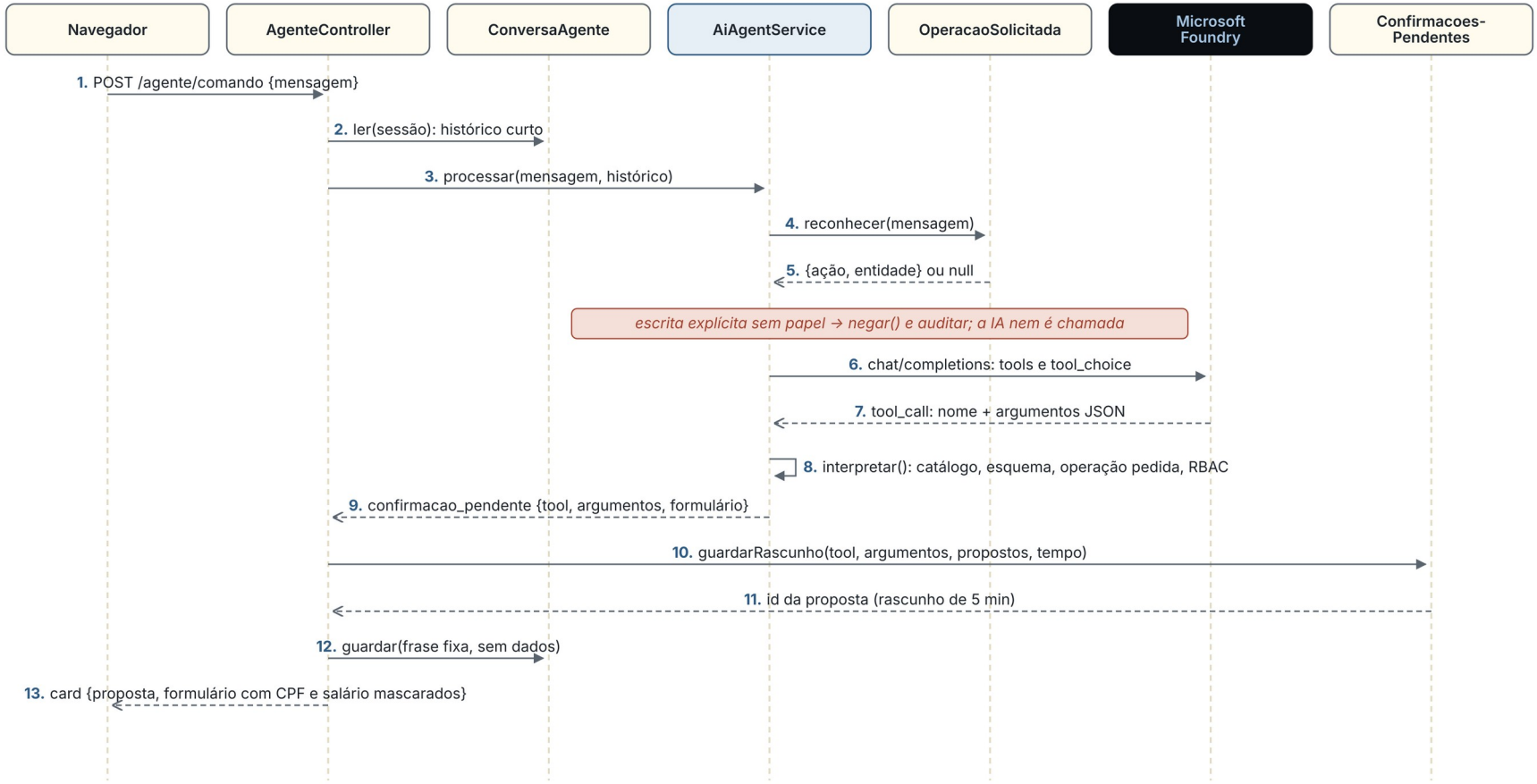


Figura 17 — Sequência do comando ao agente até o rascunho e o card de confirmação.

O segundo cobre Revisar e Confirmar de uma escrita comum. O ponto central é o final: o executor da tool e o registro de auditoria acontecem dentro da mesma transação, então as duas gravações acontecem juntas ou nenhuma acontece.

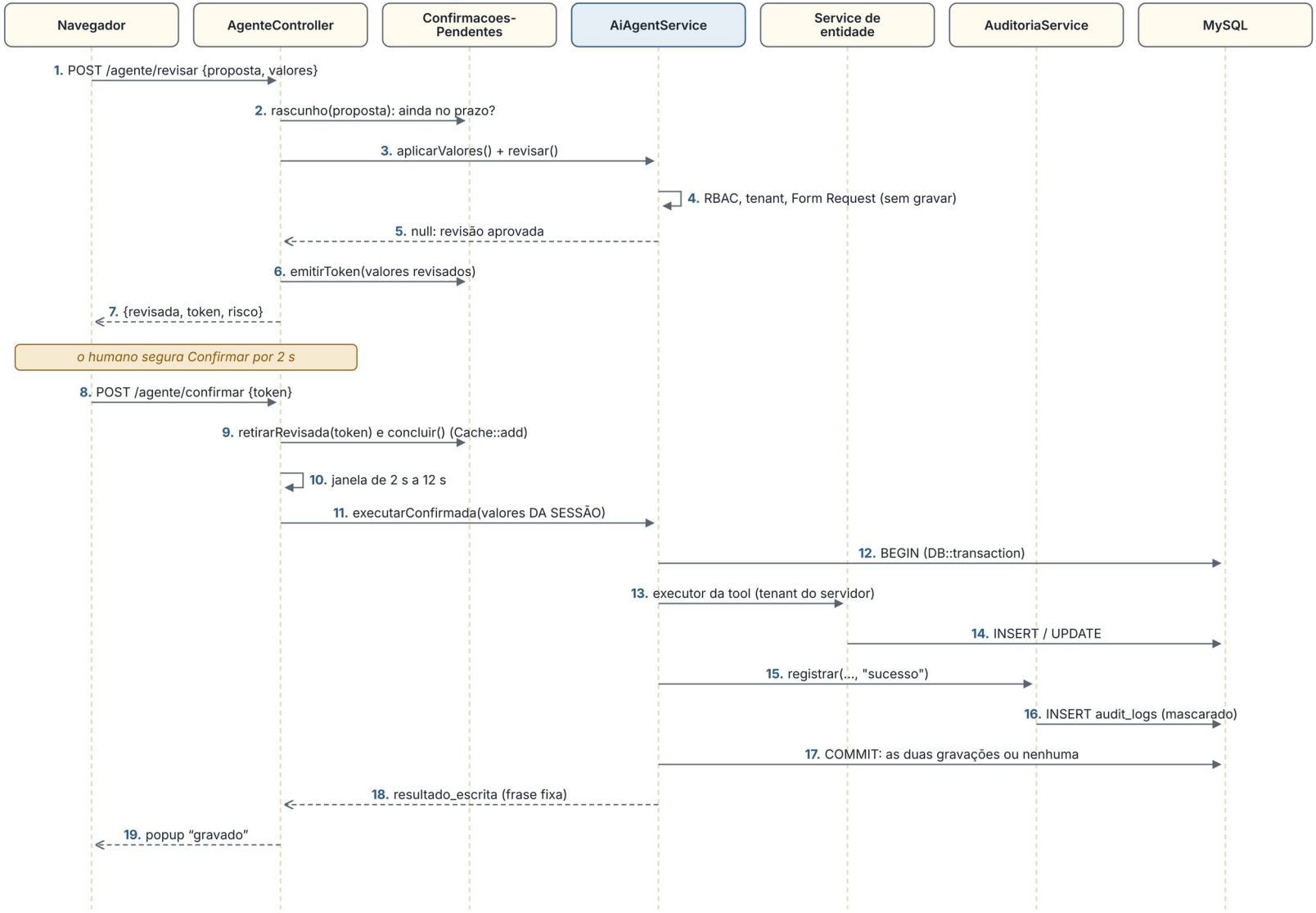


Figura 18 — Sequência de Revisar e Confirmar até a gravação com auditoria na mesma transação.

O terceiro cobre uma restauração autorizada por senha de administrador, feita por um operador. A permissão é conferida antes da senha, a tentativa é contada antes de a senha ser conferida, e a senha nunca sai desta requisição.

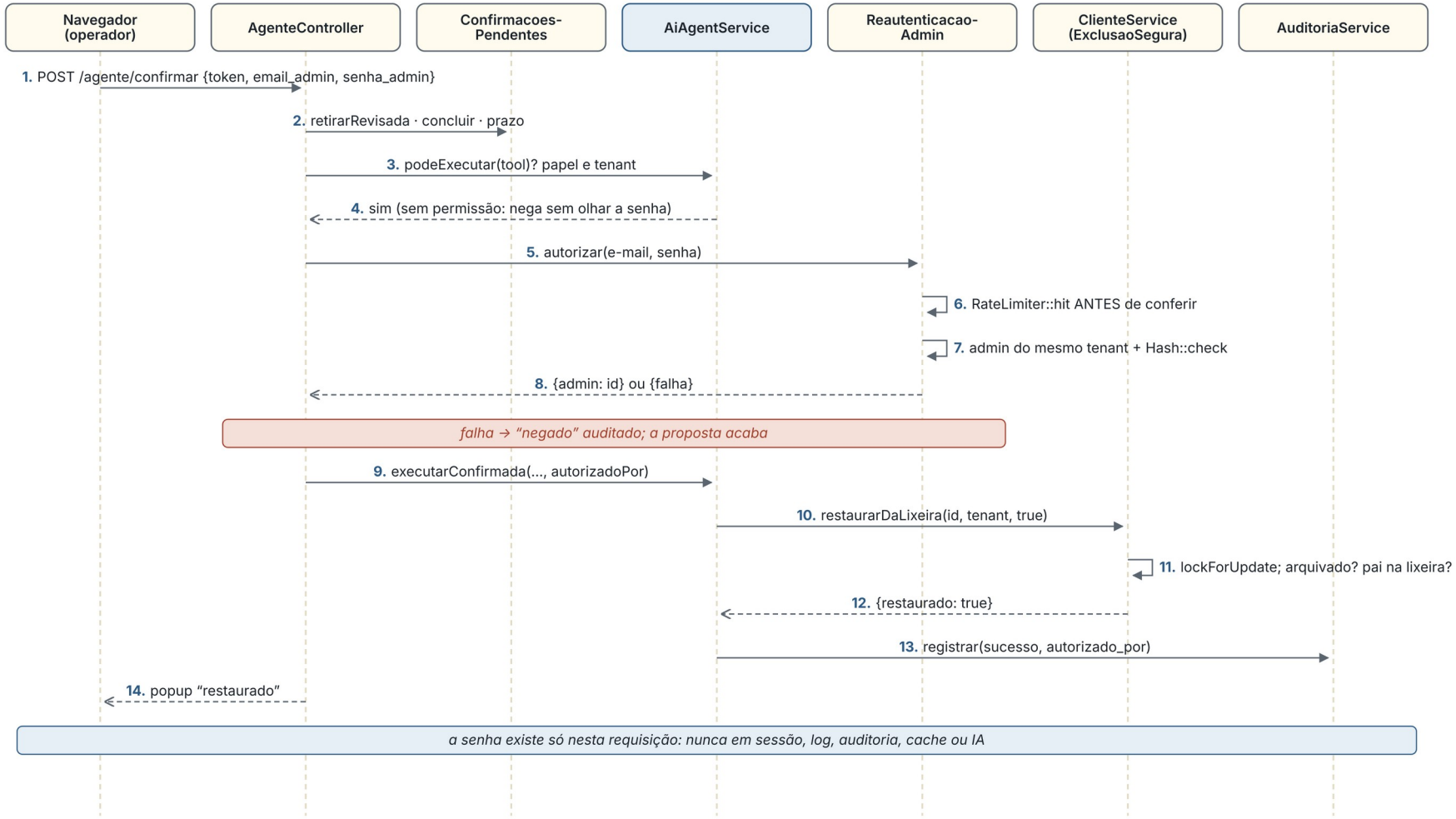


Figura 19 — Sequência de uma restauração da lixeira autorizada pela senha de um administrador.

8. Segurança

O Sentinel é um protótipo acadêmico, mas trata dados que seriam sensíveis em uma empresa real: CPF, salário e movimentação financeira. A regra do projeto é que **controle de escopo não é desculpa para segurança relapsa**: não se adiciona infraestrutura de produção (filas, 2FA, observabilidade), mas tudo o que está no escopo é feito com cuidado e coberto por teste.

8.1 Princípios

- **A IA é tratada como entrada não confiável.** O que vem do modelo passa pelas mesmas verificações que um formulário vindo de um usuário mal-intencionado, e a operação que ele devolve é comparada com a que o usuário pediu.
- **Defesa em profundidade.** O RBAC é conferido antes da IA (pedido explícito), na proposta, no Revisar e de novo na confirmação; o tenant é injetado pelo servidor em todas as camadas.
- **Falha fechada.** Sem usuário, papel ou tenant, ou com papel desconhecido, tudo é negado.
- **Permissão antes do conteúdo; sem segredo antes da hora.** O papel é conferido antes dos filtros e antes da senha.
- **Mínimo de dado em cada lugar.** O navegador, o log, a auditoria, o CSV e a IA recebem só o que precisam, com a mesma máscara.
- **Tudo auditável.** Toda decisão do sistema, inclusive as negadas e as respostas rejeitadas da IA, vira uma linha consultável.

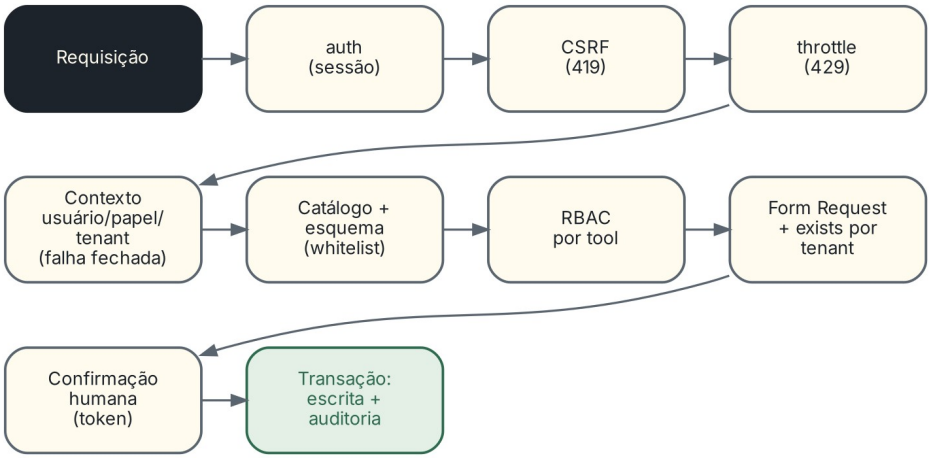


Figura 20 — Camadas que uma requisição de escrita atravessa até a gravação.

8.2 Ameaças e proteções

Tabela 15 — Ameaças, proteções e testes

Ameaça	Proteção	Onde	Teste que comprova
Injeção de prompt / tool inventada pelo modelo	Whitelist estática; argumentos conferidos contra o esquema; chave fora do esquema = resposta inválida; tool fora do catálogo negada e auditada só com nomes de argumentos	`interpretar()`, `executarTool()`	`test_tool_inventada_pelo_modelo_nao_executa_nada`; `test_argumentos_invalidos_nao_executam_tool`; `test_tool_inventada_com_cpf_e_salario_grava_so_os_nomes_dos_argumentos`
O modelo trocar a operação pedida (ex.: exclusão vira consulta)	Intenção explícita reconhecida no servidor; negação antes da IA; só a tool pedida e `tool_choice` forçado; resposta diferente = `AI_INTENT_MISMATCH`, auditada como negado	`OperacaoSolicitada`, `processar()`, `auditarDivergencia()`	`test_intencao_negada_antes_da_ia_nao_pode_virar_consulta`; `test_exclusao_definitiva_nao_vira_exclusao_logica`; `test_divergencia_audita_negado_so_com_os_nomes_dos_argumentos`
Acesso a dados de outra empresa (IDOR / tenant)	Tenant sempre do servidor; services exigem `int $tenantId`; regras `exists` reescritas para o tenant; registro de outro tenant responde como inexistente	`ContextoUsuario`, services, `escoparPorTenant()`	`test_tenant_enviado_pelo_modelo_e_ignorado`; `test_registro_de_outro_tenant_ou_na_lixeira_responde_igual_a_inexistente`; `test_id_de_outro_tenant_posto_na_sessao_e_recusado_no_service`
Escalada de papel	RBAC por tool antes da IA, na proposta, no Revisar e na confirmação; falha fechada	`papelAtualPermite()`	`test_rbac_e_rechecado_na_confirmacao`; `test_rbac_e_reaplicado_no_revisar_e_de_novo_na_confirmacao`; `test_rbac_papel_desconhecido_nega_tudo`
Força bruta de senha	Login: 5/min por e-mail + IP. Card: 5 erros por admin digitado e 10 por usuário, contados antes de conferir	`AppServiceProvider`, `ReautenticacaoAdmin`	`test_login_bloqueia_a_sexta_tentativa_no_mesmo_minuto`; `test_a_tentativa_conta_antes_de_conferir_a_senha`
Usar o card para descobrir senhas	Papel e tenant conferidos antes da senha; mesma resposta com senha certa ou errada para quem não tem permissão	`AgenteController::confirmar()`	`test_usuario_rebaixado_recebe_o_mesmo_negado_com_senha_certa_ou_errada_sem_gastar_tentativa`
Usar a validação da auditoria para sondar a tela	O papel é conferido antes dos filtros: não admin recebe a mesma negação auditada, com filtro válido ou não	`AuditoriaController::responder()`	`test_nao_admin_com_filtro_invalido_recebe_a_mesma_negacao_auditada`
XSS	Blade só com `{{ }}`; JS só `createElement`/`textContent`; a IA nunca gera HTML	views, `partials/bancada.blade.php`	`test_erros_e_email_sao_escapados_e_senha_nao_e_reexibida` (JS: revisão de código)
CSRF	Middleware padrão do grupo `web`; cabeçalho `X-CSRF-TOKEN` em todo `fetch`; 419 em português	`layouts/app.blade.php`	`test_layout_envia_credenciais_ao_login_real_com_csrf`; bloqueio conferido no teste manual

Ameaça	Proteção	Onde	Teste que comprova
Injeção de fórmula no CSV	Apóstrofo antes de `=`, `+`, `-`, `@`, também depois de separador ou aspas	`PlanilhaCsv::celula()`	`test_celula_que_comeca_com_formula_ganha_apostrofo`
Vazamento de dado pessoal	Máscara única; logs só com metadados; senha só na requisição; histórico só com frases fixas; CSV da auditoria sem parâmetros	`AuditoriaService`, `ConversaAgente`, `ConsultaAuditoria`	`AuditoriaMascaraTest` (15 testes); `test_a_senha_nao_aparece_na_resposta_log_auditoria_sessao_historico_cache_nem_na_ia`
Confirmação duplicada (corrida)	Token de uso único com `Cache::add`; um desfecho por proposta	`ConfirmacoesPendentes::concluir()`	`test_m1_confirmar_e_cancelar_simultaneos_so_o_primeiro_desfecho_vale`; `test_rnf02_token_so_vale_uma_vez`
Valor adulterado no navegador depois do Revisar	A confirmação usa os valores e os ids do lote guardados na sessão; o corpo só leva o token	`AgenteController::confirmar()`	`test_valor_alterado_no_navegador_depois_do_revisar_e_ignorado`; `test_valores_e_ids_do_navegador_sao_ignorados_no_revisar_e_no_confirmar`
Lote gravado pela metade ou com item alterado	Ids congelados; teto de 25; revalidação no Revisar; trava e reconferência no Confirmar; uma transação	`resolverLote()`, `LancamentoService::conciliarLote()`	`test_falha_no_meio_da_gravacao_desfaz_o_lote_inteiro`; `test_item_que_muda_entre_revisar_e_confirmar_desfaz_o_lote_inteiro_com_uma_linha_so`
Exclusão acidental	Revisar + segurar; bloqueio por dependência; lixeira; arquivamento; exclusão física só manual com senha	`ExclusaoSegura`, tools da lixeira	`GovernancaTest::test_rf09_*`, `test_rf11_rf12_*`; `test_excluir_e_so_para_admin`
Conta de demonstração com senha fraca; tela de amostra aberta	`DemonstracaoSeeder` recusa senha com menos de 12 caracteres ou conhecida; as telas de amostra foram removidas	`DemonstracaoSeeder`, `routes/web.php`	`test_demo_senha_curta_e_recusada_sem_gravar_nada`; `test_amostra_nao_existe_mais`
Abuso de custo e de uso	Limites de taxa por rota (seção 8.4); teto de 5.000 linhas no CSV da auditoria	rotas `web`, `ConsultaAuditoria`	`test_rnf10_a_11a_chamada_no_mesmo_minuto_recebe_429`; `test_acima_do_teto_recusa_sem_cortar_e_audita`

8.3 Dados pessoais (LGPD)

Uma só máscara, definida em `AuditoriaService`, é aplicada em todos os lugares por onde um dado pessoal poderia sair: auditoria, resposta de consulta, formulário do card, CSV e histórico da conversa.

Tabela 16 — Máscara de dados sensíveis

Campo	Regra	Exemplo de saída
`cpf`	Mostra só os 5 últimos dígitos	`***.***.789-01`
`documento` com 11 dígitos	É CPF: mesma máscara do `cpf`	`***.***.789-01`

Campo	Regra	Exemplo de saída
`documento` com 14 caracteres	É CNPJ, dado público de empresa: passa intacto	`12.345.678/0001-90`
`salario`	Nada do valor aparece	`[redigido]`
Valor que não é texto nem número	Redigido	`[redigido]`
E-mail e telefone	Não são mascarados (decisão da equipe em 28/10: são dados de contato que o usuário precisa ver para trabalhar)	como cadastrado
Senha de admin	Só existe na requisição de confirmação; nunca em sessão, log, auditoria, cache ou IA	—

O administrador vê o salário em reais **apenas no card** de cadastro ou atualização de funcionário, porque precisa conferir o valor antes de gravar; em todo o resto (tabela, histórico, auditoria e CSV) o valor continua redigido. O CSV de funcionários sai sem CPF e sem salário, e o CSV da auditoria sai sem nenhum parâmetro.

8.4 Autenticação, sessão e limites

- Login com o `Auth` nativo do Laravel: `session()->regenerate()` no login e `invalidate()` no logout. Não há cadastro público; os usuários locais vêm do comando `sentinel:preparar-acesso`, que gera e mostra uma senha aleatória de 24 caracteres só na criação, ou do `DemonstracaoSeeder`, que exige uma `DEMO_SENHA` forte.
- O logout dá desfecho às propostas abertas e limpa a memória da conversa.
- Formulário enviado com a sessão já expirada mostra a tela 419 (sessão expirada) em vez de executar algo; as rotas JSON continuam recebendo 401.
- Na máquina da apresentação: `APP_DEBUG=false` (não expor stack traces) e `SESSION_ENCRYPT=true` (sessão cifrada). O `instalar.bat` grava os dois quando cria um `.env` novo.

Tabela 17 — Limites de taxa

Limitador	Limite	Chave
`login`	5 por minuto	e-mail + IP
`agente` (chama a IA)	10 por minuto	usuário
`agente-sem-ia` (revisar, cancelar, editar, formulário, consultar)	30 por minuto	usuário
`auditoria` (lista, detalhe e tela)	60 por minuto	usuário
`exportar` (CSV de consulta e CSV da auditoria)	5 por minuto	usuário
Senha de admin no card	5 erros por admin digitado; 10 por usuário, a cada 10 min	usuário + HMAC do e-mail

8.5 Ambiente e dependências

Tabela 18 — Ambiente e dependências

Item	Situação	Ação
PHP	As falhas críticas de outubro de 2026 (CVE-2026-109618 e CVE-2026-109619) exigem 8.4.27 ou mais. O PC do laboratório usado na reconciliação de 03/11 já está em 8.4.27	Conferir com `php -v` na máquina da apresentação; o `instalar.bat` avisa quando a versão é menor
`session.upload_progress.enabled`	Ligado por padrão (CVE-2026-115560)	Desligado (`=0`) no laboratório; repetir na máquina da apresentação
`composer audit`	Limpo	`symfony/http-foundation`, `symfony/mime` e `league/commonmark` atualizados
Pacotes npm	Ondas recentes de pacotes comprometidos no registro npm; `npm ci` de 03/11 sem vulnerabilidades	Instalar sempre com `npm ci` (respeita o `package-lock.json`). O Sentinel não roda servidor Node em produção
Arquivos `.env`	Uma cópia `.env.bak` com segredos foi achada fora do `.gitignore` em 28/10	O `.gitignore` passou a cobrir `.env.*` (exceto o `.env.example`)

Fora do escopo do protótipo (registrado, não descartado)

A lista completa do que fica para quando o projeto sair de protótipo (2FA, recuperação de senha, verificação de e-mail, tabela de tenants com chaves estrangeiras, observabilidade, hardening de servidor) está no Notion, na página "Checklist final — antes de sair de protótipo acadêmico".

9. O código em detalhe

Esta seção mostra trechos reais do repositório (revisão `c641002`) que materializam as garantias descritas acima, cada um seguido do que ele faz e de por que foi escrito assim. Os trechos estão resumidos onde indicado por `// ...`; o restante é cópia fiel. O Apêndice B complementa esta seção percorrendo o código arquivo por arquivo.

9.1 Uma tool no catálogo

Código 1 PHP · app/Services/AiAgentService.php, linhas 1272–1288

```
$this->definirTool(
    nome: 'consultar_clientes',
    descricao: 'Consulta clientes cadastrados, opcionalmente filtrando por parte do nome. Ação
de leitura — não exige confirmação.',
    tipo: 'leitura',
    risco: 'leitura',
    acao: 'consultar',
    entidade: 'Cliente',
    papelMinimo: 'leitura',
    propriedades: [
        'nome' => ['type' => 'string', 'description' => 'Parte do nome do cliente.'],
    ],
    obrigatorios: [],
    executar: fn (array $a) => $this->clienteService->buscarPorFiltro($a['nome'] ?? null,
$a['tenant_id']),
    mensagemErro: 'Não consegui consultar os clientes agora. Tente novamente.',
    colunas: ['id', 'nome', 'documento', 'email', 'telefone'],
    editavelPor: 'atualizar_cliente',
),
```

O que faz. Declara, num só lugar, tudo o que define uma ferramenta: o que a IA enxerga (nome, descrição e esquema JSON em `propriedades`), quem pode usar (`papelMinimo`), a classe de risco (que vira a cor da borda do card), a ação registrada na auditoria, as colunas devolvidas à tela, qual tool edita cada linha (`editavelPor`) e o executor.

Por que assim. Uma fonte única impede divergência entre o que a IA vê, o que o RBAC permite e o que a tela mostra. Repare que o esquema **não tem** `tenant_id`: o executor recebe o tenant do servidor, então o modelo não tem como pedir dados de outra empresa.

9.2 A intenção explícita, antes da IA

Código 2 PHP · app/Services/AiAgentService.php, linhas 161–169 e 200–201 (resumido)

```
// A intenção explícita pertence ao pedido atual, não à ferramenta escolhida pela IA.
$intencao = $this->operacoes->reconhecer($mensagem);
$compativeis = $intencao === null ? [] : array_filter($this->catalogo(), fn ($d) =>
    $d['tipo'] === 'escrita' && $d['acao'] === $intencao['acao'] && $d['entidade'] ===
$intencao['entidade']);
$toolSolicitada = count($compativeis) === 1 ? array_key_first($compativeis) : null;
if ($toolSolicitada !== null && ! $this->papelAtualPermite($compativeis[$toolSolicitada]
['papel_minimo'])) {
    // Autorizar antes do provedor: nem uma consulta nem uma falha externa substitui a
negativa.
    return $this->negar($toolSolicitada, $compativeis[$toolSolicitada], []);
}
// ...
'tools' => $toolSolicitada === null ? $this->tools() : [$compativeis[$toolSolicitada]
['definicao']],
'tool_choice' => $toolSolicitada === null ? 'auto' : ['type' => 'function', 'function' =>
['name' => $toolSolicitada]],
```

O que faz. Pergunta à `OperacaoSolicitada` se a mensagem começa com um verbo de escrita e uma entidade conhecidos. Se isso aponta para exatamente uma tool e o papel não permite, nega e audita sem chamar o provedor. Se permite, manda à IA só aquela tool e obriga o modelo a usá-la.

Por que assim. Nos testes manuais, o modelo às vezes atendia "exclua o cliente X" com uma consulta, ou "excluir de vez" com uma exclusão lógica. Isso nunca gravava nada errado, mas confundia o usuário e escondia uma negação. Decidir a operação no servidor, com um vocabulário fixo, tira essa escolha do modelo nos pedidos explícitos e ainda economiza uma chamada paga quando a resposta já é "não".

9.3 A chamada ao provedor e a medição do tempo

Código 3 PHP · app/Services/AiAgentService.php, linhas 179–220 (resumido)

```
// B1 (RNF04): tempo da chamada à IA, do envio até a resposta (ou a falha) do provedor.
$inicio = hrtime(true);
$decorrido = fn () => (int) round((hrtime(true) - $inicio) / 1e6);

try {
    $resposta = Http::withHeaders(['api-key' => $apiKey])
        ->timeout(30)
        ->post("{$endpoint}/openai/v1/chat/completions", [
            'model' => $deployment,
            'messages' => [
                ['role' => 'system', 'content' => 'Papel autenticado do usuário: ... Você é o
assistente do Sentinel, ...'],
                ...$historico,
                ['role' => 'user', 'content' => $mensagem],
            ],
            'tools' => /* todas, ou só a pedida */,
            'tool_choice' => /* 'auto', ou a pedida */,
        ]);
} catch (\Throwable $e) {
    // ... classifica timeout ou falha de conexão
    return $this->falha($timeout ? 'provider_timeout' : 'provider_connection_error', /* ...
*/);
}

$tempoIaMs = $decorrido();

// Tudo o que esta resposta auditar (leitura, negado, tool fora do catálogo) leva o tempo da
IA.
return $this->detalhesAuditoria->com(['tempo_ia_ms' => $tempoIaMs],
    fn () => $this->interpretar($resposta, $tempoIaMs, $toolSolicitada));
```

O que faz. Faz a única chamada HTTP do sistema ao Microsoft Foundry: prompt de sistema, histórico curto, mensagem do usuário e a lista de tools derivada do catálogo. Cronometra a chamada com `hrtime` (relógio monotônico, imune a ajustes de hora) e coloca o tempo no escopo da auditoria.

Por que assim. Isolar a integração em uma classe atende à regra de arquitetura (trocar de provedor é mexer em um ponto) e torna o RNF04 mensurável sem espalhar código de medição pelo sistema. A chamada é stateless: nada fica guardado no provedor entre um pedido e outro.

9.4 Conferindo o que o modelo respondeu

Código 4 PHP · app/Services/AiAgentService.php, linhas 280–344 (resumido)

```
// O provedor é externo: tool_choice não substitui a verificação no servidor.
if ($toolSolicitada !== null && ! empty($toolCalls)) {
    foreach ($toolCalls as $pedido) {
        if (($pedido['function']['name'] ?? null) !== $toolSolicitada) {
            $this->auditarDivergencia($toolSolicitada, $pedido);
            return $this->falha('intent_mismatch', 'AI_INTENT_MISMATCH', 422, [], null,
                'O agente não manteve a operação solicitada. Nada foi executado. Reformule o
pedido.');
        }
    }
}
// ...
// Valida todas as chamadas antes de processar a primeira; nunca converte JSON inválido em
consulta ampla.
foreach ($toolCalls as $pedido) {
    // ...
    try {
        $objeto = json_decode($f['arguments'], false, 32, JSON_THROW_ON_ERROR);
    } catch (\JsonException) {
        return $this->respostaForaDoFormato($resposta->status());
    }
    $args = (array) $objeto;
    unset($args['tenant_id']);   // tenant é sempre removido e derivado do backend
    $schema = $this->catalogo()[$f['name']]['definicao']['function']['parameters']
['properties'] ?? null;
    if ($schema !== null && array_diff(array_keys($args), array_keys($schema)) !== []) {
        return $this->respostaForaDoFormato($resposta->status());
    }
}
```

O que faz. Antes de executar qualquer coisa, confere duas coisas: se o pedido era explícito, toda tool devolvida precisa ser a pedida (senão, divergência auditada e 422); e cada chamada precisa ter JSON válido, ser um objeto e não ter chave fora do esquema da tool. O `tenant_id` é descartado mesmo que o modelo o envie.

Por que assim. Forçar a tool no `tool_choice` reduz o erro, mas não é garantia: o provedor é externo. Um JSON quebrado nunca pode virar "consulta sem filtro" (que devolveria tudo). Chave inesperada é sinal de alucinação ou de injeção de prompt; o sistema prefere responder "não entendi" a adivinhar.

9.5 Revisar: validar sem gravar

Código 5 PHP · app/Http/Controllers/AgenteController.php, linhas 151–188 (resumido)

```
$tool = $rascunho['tool'];
$revisados = $this->aiAgentService->aplicarValores($tool, $rascunho['argumentos'], $valores);

$recusa = $this->aiAgentService->revisar($tool, $revisados,
ConfirmacoesPendentes::propostos($rascunho),
    ConfirmacoesPendentes::origem($rascunho), ConfirmacoesPendentes::tempoIa($rascunho),
    ConfirmacoesPendentes::loteIds($rascunho));
// ...
if ($recusa !== null) {
    // O rascunho guarda o que já foi editado: na próxima revisão, sensível em branco mantém o
editado.
    $this->pendentes->atualizarRascunho($request->session(), $proposta, $rascunho,
$revisados);
    return response()->json([...$recusa, 'proposta' => $proposta, 'formulario' =>
$formulario], 422);
}

return response()->json([
    'tipo' => 'revisada',
    'proposta' => $proposta,
    'token' => $this->pendentes->emitirToken($request->session(), $proposta, $rascunho,
$revisados),
    'risco' => $this->aiAgentService->risco($tool),
]);
```

O que faz. Aplica os valores que o humano editou sobre a proposta guardada na sessão (só os campos do formulário) e valida com RBAC, tenant e as regras do Form Request, **sem gravar**. No lote, confere cada item a partir dos ids guardados. Se houver erro, devolve o formulário (ou o plano) com a frase de cada campo ou item; se não houver, emite o token de segurar.

Por que assim. Separar "revisar" de "confirmar" garante que o humano veja o formulário validado antes de o botão de segurar existir, o que combate a fadiga de confirmação. E a confirmação usa exatamente os valores revisados, guardados no servidor.

9.6 Confirmar: sem segredo antes da hora

Código 6 PHP · app/Http/Controllers/AgenteController.php, linhas 207–255 (resumido)

```
$pendente = $this->pendentes->retirarRevisada($request->session(), $token); // uso único
// ...
if (! $this->pendentes->concluir($proposta)) {
    return response()->json(['tipo' => 'erro', 'mensagem' => 'Esta confirmação já foi
usada.'], 409);
}
$decorrido = $this->pendentes->decorrido($pendente);

// Fora do prazo: antes da espera mínima, ou vencido (12 s do token ou os 5 min absolutos do
rascunho).
if ($decorrido < ConfirmacoesPendentes::ESPERA_MINIMA_S || $this->pendentes-
>expirou($pendente)) {
    // ... audita "expirado" ou "recusado" e responde 422
}

// Ordem: primeiro o que não envolve segredo (proposta revisada e no prazo, acima; papel e
tenant,
// aqui). Sem permissão, a senha nem é conferida nem gasta tentativa.
if ($this->aiAgentService->exigeReautenticacao($pendente['tool'])
    && $this->aiAgentService->podeExecutar($pendente['tool'])) {
    $autorizacao = $this->reautenticar($request);
    // ...
}
```

O que faz. Retira a proposta revisada da sessão (uso único), marca o desfecho de forma atômica, confere a janela de 2 a 12 s e só então, se a tool exigir e o usuário tiver permissão, confere a senha de administrador.

Por que assim. A ordem importa. Se a senha fosse conferida primeiro, um usuário sem permissão poderia usar o card para testar senhas de administradores. Com a ordem atual, ele recebe a mesma negação com senha certa ou errada e não gasta tentativas. A contrapartida aceita: senha errada encerra a proposta, e é preciso pedir de novo.

9.7 Uso único do token

Código 7 PHP · app/Services/ConfirmacoesPendentes.php, linhas 238–241

```
public function concluir(string $proposta): bool
{
    return Cache::add("agente:token_usado:{$proposta}", true,
        now()->addMinutes(max(1, (int) config('session.lifetime'))));
}
```

O que faz. Grava uma marca no cache somente se ela ainda não existir. `Cache::add` é atômico no armazenamento, então, entre duas requisições simultâneas, só uma recebe `true`.

Por que assim. Duas requisições podem ler a mesma sessão ao mesmo tempo (por exemplo, confirmar e cancelar em sequência rápida). Sem a marca atômica, a mesma proposta poderia ter dois desfechos. A marca é por proposta, não por token: dois tokens da mesma proposta nunca geram duas gravações.

9.8 Execução e auditoria na mesma transação

Código 8 PHP · app/Services/AiAgentService.php, linhas 533–563 (resumido)

```
$argumentos['tenant_id'] = $this->contexto->tenantId();

try {
    $resultado = $this->detalhesAuditoria->com([], fn () => DB::transaction(function () use
($tool, $definicao, $argumentos) {
        $retorno = $definicao['executar']($argumentos);

        // RF03: o lote já auditou uma linha por item (com lote_id) dentro do executor.
        if ($tool === 'conciliar_lancamentos_em_lote') {
            return $retorno;
        }

        $this->auditar($tool, $definicao['acao'], $definicao['entidade'],
            $retorno instanceof Model ? (int) $retorno->getKey() : null,
            $argumentos, 'sucesso', null, true);

        return $retorno;
    }));
} catch (ToolRecusadaException $e) {
    // Regra de negócio: a mensagem já é uma frase amigável escrita para o usuário.
    $this->auditar($tool, /* ... */ 'recusado', $e->getMessage(), true);
```

O que faz. Sobrescreve o tenant com o do usuário logado e executa a tool e o registro de auditoria dentro de `DB::transaction`. Recusas de regra de negócio viram "recusado" com uma frase amigável.

Por que assim. Se gravar a auditoria falhar, a escrita é desfeita: não existe gravação sem rastro. Isso é comprovado por `test_escrita_e_auditoria_sao_atomicas_se_o_audit_falha_a_escrita_e_desfeita`. O lote faz a própria auditoria (uma linha por item) dentro da mesma transação.

9.9 RBAC com falha fechada

Código 9 PHP · app/Services/AiAgentService.php, linhas 2113–2120

```
private function papelAtualPermite(string $papelMinimo): bool
{
    $hierarquia = (array) config('sentinel.papeis', []);   // ['leitura', 'operador', 'admin']
    $atual = array_search($this->contexto->papel(), $hierarquia, true);
    $minimo = array_search($papelMinimo, $hierarquia, true);

    return $atual !== false && $minimo !== false && $atual >= $minimo;
}
```

O que faz. Compara a posição do papel do usuário com a do papel mínimo da tool na hierarquia configurada.

Por que assim. Se o papel do usuário ou o papel mínimo não estiverem na lista (erro de digitação, papel removido), `array_search` devolve `false` e a função nega. O erro de configuração resulta em "não pode", nunca em "pode tudo". A checagem roda antes da IA, na proposta, no Revisar e na confirmação.

9.10 Regras exists presas ao tenant

Código 10 PHP · app/Services/AiAgentService.php, linhas 1804–1821

```
private function escoparPorTenant(array $regras, int $tenantId): array
{
    return array_map(function ($regra) use ($tenantId) {
        // ...
        return array_map(function (string $parte) use ($tenantId) {
            if (! str_starts_with($parte, 'exists:')) {
                return $parte;
            }
            [$tabela, $coluna] = array_pad(explode(',', substr($parte, 7)), 2, 'id');

            return Rule::exists($tabela, $coluna)->where('tenant_id', $tenantId)-
>whereNull('deleted_at');
        }, explode('|', $regra));
    }, $regras);
}
```

O que faz. Reescreve, em tempo de execução, toda regra `exists:tabela,coluna` dos Form Requests para exigir o tenant atual e um registro fora da lixeira.

Por que assim. Sem isso, uma nota fiscal poderia apontar para o cliente de outra empresa só informando o id dele. A reescrita centralizada protege todos os Form Requests de uma vez.

9.11 Máscara LGPD

Código 11 PHP · app/Services/AuditoriaService.php, linhas 18 e 100–130 (resumido)

```
private const MASCARAS = ['cpf' => 'mascararCpf', 'salario' => 'redigir', 'documento' =>
'mascararDocumento'];

public static function mascararCamposSensiveis(array $parametros): array
{
    return collect($parametros)->map(function ($valor, $campo) {
        $metodo = self::MASCARAS[$campo] ?? null;
        return $metodo === null ? $valor : self::$metodo($valor);
    })->all();
}

private static function mascararDocumento(mixed $valor): mixed
{
    if ($valor === null) {
        return null;
    }
    if (! is_scalar($valor)) {
        return self::REDIGIDO;
    }
    return strlen(preg_replace('/\D/', '', (string) $valor)) === 11 ?
self::mascararCpf($valor) : $valor;
}
```

O que faz. Mapeia cada campo sensível para a sua regra de máscara. O campo `documento` decide pelo número de dígitos se é CPF (mascarado) ou CNPJ (público).

Por que assim. Ter uma fonte única evita que um lugar novo (por exemplo, o CSV) esqueça de mascarar. Tipos inesperados (listas, objetos) são redigidos em vez de passarem crus.

9.12 Higiene do histórico da conversa

Código 12 PHP · app/Services/ConversaAgente.php, linhas 78–94 (resumido)

```
private function sanitizar(string $texto): string
{
    // Credenciais não são necessárias aos fluxos de cadastro disponíveis.
    if (preg_match('/senha|password|api.?key|app.?key|csrf|cookie|session.?id|bearer/i',
$texto)) {
        return '[Mensagem com credenciais omitida do histórico]';
    }
    // ... troca a chave da API e a APP_KEY, se aparecerem, por '[credencial omitida]'
    $texto = preg_replace('/\b\d{2}\.?\d{3}\.?\d{3}\/?\d{4}-?\d{2}\b/', '[documento omitido]',
$texto);
    $texto = preg_replace('/\b\d{3}\.?\d{3}\.?\d{3}-?\d{2}\b/', '[documento omitido]',
$texto);
    $texto = preg_replace('/\b(sal[aá]rios?|remunera[cç](?:[ãa]o|[oõ]es))\b(\D{0,30}?)(?:R\$\
s*)?\d(?:[\d.,]*\d)?/iu',
        '$1$2[valor omitido]', $texto);
    return mb_substr($texto, 0, 1000);
}
```

O que faz. Antes de uma mensagem entrar no histórico que volta para a IA, remove credenciais, CNPJ, CPF e o valor que acompanha "salário" ou "remuneração", e corta em 1.000 caracteres.

Por que assim. O histórico é reenviado ao provedor a cada pedido; quanto menos dado pessoal ele carrega, menor a exposição. As limitações conhecidas (por exemplo, "ganha 4500" sem a palavra salário) estão registradas na seção 14.

9.13 Limite atômico de tentativas de senha

Código 13 PHP · app/Services/ReautenticacaoAdmin.php, linhas 72–98 (resumido)

```
// O e-mail digitado entra na chave só como HMAC: se alguém digitar a senha no campo
// do e-mail, ela não fica legível no cache.
$porUsuario = "reautenticacao:usuario:{$usuarioId}";
$porAdmin = "reautenticacao:admin:{$usuarioId}:".hash_hmac('sha256', Str::lower(trim($email)),
(string) config('app.key'));

// A tentativa conta ANTES de conferir a senha (incremento atômico no cache): pedidos em
// paralelo não passam todos pela conferência antes de o primeiro erro ser contado.
$tentativasDoUsuario = RateLimiter::hit($porUsuario, self::JANELA_S);
$tentativasNoAdmin = RateLimiter::hit($porAdmin, self::JANELA_S);

if ($tentativasDoUsuario > self::LIMITE_POR_USUARIO || $tentativasNoAdmin >
self::LIMITE_POR_ADMIN) {
    $this->devolverTentativa($porUsuario, $porAdmin);
    return ['falha' => 'limite'];
}

$admin = /* ... */ $this->adminQueConfere(trim($email), $senha);
if ($admin === null) {
    return ['falha' => 'invalido'];
}

// Acerto não conta: a tentativa volta.
$this->devolverTentativa($porUsuario, $porAdmin);
return ['admin' => (int) $admin->id];
```

O que faz. Conta a tentativa antes de conferir a senha, com dois limites (por usuário e por e-mail de admin digitado). Só um admin **do mesmo tenant** é aceito. O acerto devolve a tentativa.

Por que assim. Se a contagem viesse depois da conferência, vários pedidos em paralelo passariam todos antes de o primeiro erro ser registrado. O HMAC evita que uma senha digitada por engano no campo de e-mail fique legível no cache.

9.14 Bloqueio por dependência

Código 14 PHP · app/Services/Concerns/ExclusaoSegura.php, linhas 42–59 e 181–192

```
protected function excluirComBloqueio(Model $registro, array $relacoesQueBloqueiam): array
{
    $motivo = $this->motivoDeBloqueio($registro, $relacoesQueBloqueiam);
    if ($motivo !== null) {
        return ['bloqueado' => true, 'motivo' => $motivo];
    }
    // Uma marca de arquivado que sobrou faria o registro pular o prazo da lixeira
(RF09/RF11).
    if ($registro->arquivado_em !== null) {
        $registro->forceFill(['arquivado_em' => null])->save();
    }
    $registro->delete();   // soft delete
    return ['bloqueado' => false, 'registro' => $registro];
}

private function motivoDeBloqueio(Model $registro, array $relacoes): ?string
{
    foreach ($relacoes as $relacao) {
        if ($registro->{$relacao}()->exists()) {
            $rotulo = self::ROTULOS_RELACAO[$relacao] ?? "registros em {$relacao}";
            return "Não é possível excluir: existem {$rotulo} a este registro.";
        }
    }
    return null;
}
```

O que faz. Verifica as relações que impedem a exclusão e devolve o motivo em linguagem simples; sem dependência, faz o soft delete (lixeira).

Por que assim. O motivo é escrito para o usuário, não para o desenvolvedor, como pede o RF08. A limpeza da marca de arquivado evita um caso raro em que o registro pularia o prazo da lixeira.

9.15 Lote: tudo ou nada

Código 15 PHP · app/Services/LancamentoService.php, linhas 124–138

```php
public function conciliarLote(array $ids, int $tenantId): array
{
    $lancamentos = $this->doLote($ids, $tenantId, travar: true);   // lockForUpdate
    $motivos = $this->motivosParaNaoConciliarLote($ids, $lancamentos);

    if ($motivos !== []) {
        return ['conciliados' => [], 'motivos' => $motivos];
    }

    foreach ($ids as $id) {
        $lancamentos->get($id)->update(['status' => 'conciliado']);
    }

    return ['conciliados' => array_values($ids), 'motivos' => []];
}
```

O que faz. Trava os lançamentos do lote, reconfere cada um (existe, é do tenant, ainda está pendente) e só então concilia todos. Se qualquer um mudou, não altera nenhum e devolve o motivo de cada item.

Por que assim. O método roda dentro da transação do `executarTool()`, então a trava vale até o commit e nenhuma outra requisição altera os itens no meio do caminho. Decidir "tudo ou nada" aqui, e não item a item no controller, é o que impede um lote gravado pela metade.

9.16 CSV sem injeção de fórmula

Código 16 PHP · app/Services/PlanilhaCsv.php, linhas 53–64

```php
$texto = (string) $valor;

if (is_int($valor) || is_float($valor) || preg_match('/^-?\d+(\.\d+)?$/D', $texto) === 1) {
    return $texto;
}

// UTF-8 inválido vira "?" antes das regex (com /u, elas devolveriam null e a célula sumiria).
$texto = mb_scrub(strtr($texto, ["\r" => ' ', "\n" => ' ', "\t" => ' ']), 'UTF-8');
$texto = (string) preg_replace('/([,;][\s"\x{00A0}]*)([=+\-@])/u', "\$1'\$2", $texto);

return preg_match('/^[\s"\x{00A0}]*[=+\-@]/u', $texto) === 1 ? "'".$texto : $texto;
```

O que faz. Neutraliza células que começariam com `=`, `+`, `-` ou `@` (inclusive depois de separador, espaço ou aspas) colocando um apóstrofo na frente. Números puros passam intactos.

Por que assim. Um nome de cliente como `=HYPERLINK(...)` viraria uma fórmula executável ao abrir o CSV no Excel. O CSV também sai com BOM e separador `;`, para abrir corretamente no Excel em português. A mesma função atende o CSV das consultas e o CSV da auditoria.

9.17 Segurar para confirmar, no navegador

Código 17 JavaScript · resources/views/partials/bancada.blade.php, linhas 19–20, 796–810 e 866–880 (resumido)

```
const DURACAO_HOLD_MS = 2000;
const DURACAO_IDLE_MS = 10000;

function passoHold() {
    if (!segurando) return;
    const decorrido = performance.now() - inicioHold;
    const pct = Math.min(100, (decorrido / DURACAO_HOLD_MS) * 100);
    fill.style.width = pct + '%';
    grande.textContent = Math.floor(pct) + '%';
    if (pct >= 100) {
        segurando = false;
        confirmarAcao();
        return;
    }
    rafId = requestAnimationFrame(passoHold);
}

btnConfirmar.addEventListener('mousedown', iniciarHold);
btnConfirmar.addEventListener('touchstart', iniciarHold);
btnConfirmar.addEventListener('keydown', (e) => {
    if ((e.key === 'Enter' || e.key === ' ') && !e.repeat) { e.preventDefault();
iniciarHold(); }
});
// mouseup, mouseleave, blur, touchend, touchcancel e keyup chamam soltarHold()
```

O que faz. Anima o preenchimento a cada quadro (`requestAnimationFrame`) enquanto o botão é mantido; ao chegar a 100%, envia só o token ao servidor. Mouse, toque e teclado iniciam e encerram o gesto.

Por que assim. O gesto contínuo de 2 s evita confirmações por clique acidental, e a acessibilidade por teclado é obrigatória. A segurança, porém, **não depende** do navegador: o servidor recusa confirmações antes de 2 s ou depois de 12 s, mesmo que alguém altere este JavaScript.

10. Modelo de dados e auditoria

O banco tem seis tabelas de domínio, a tabela de usuários e a tabela de auditoria. Todas as tabelas de domínio têm `tenant_id` (a empresa), `deleted_at` (lixeira) e `arquivado_em` (arquivamento).

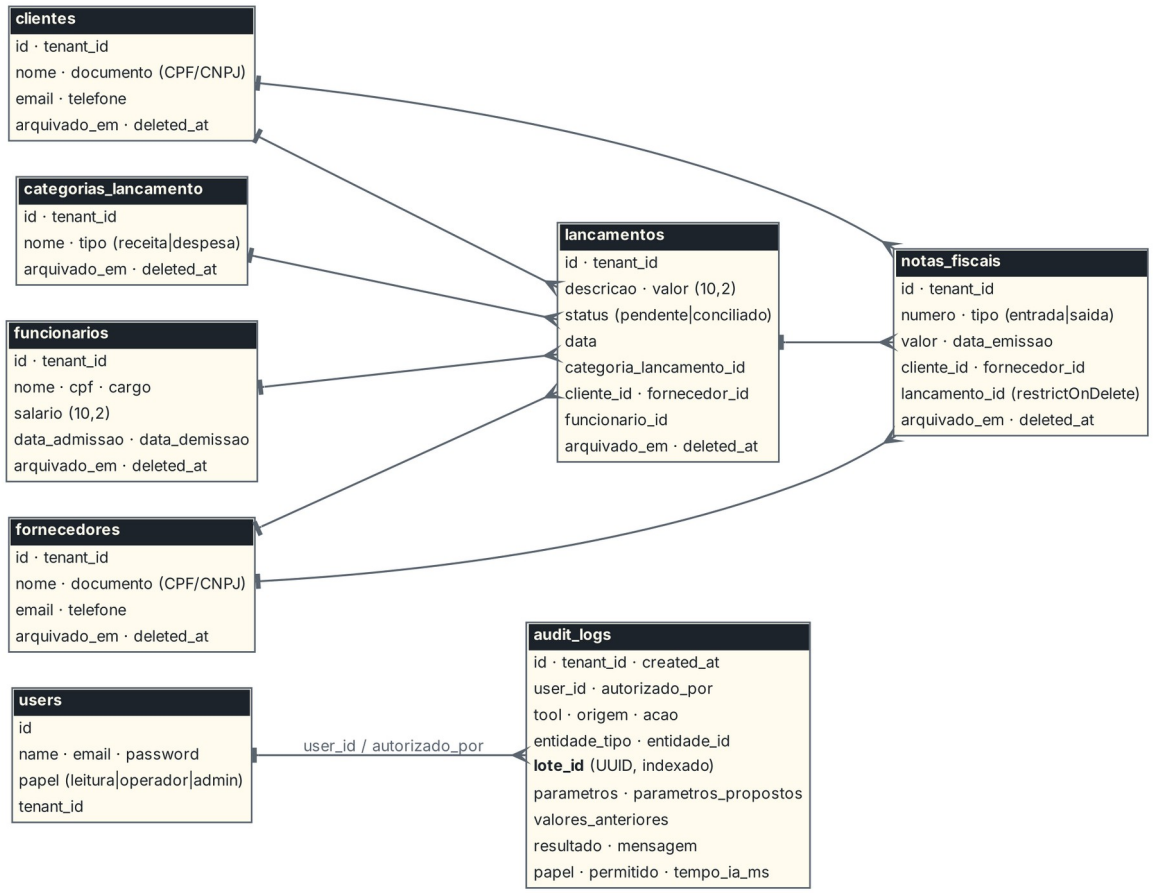


Figura 21 — Modelo de dados: entidades de domínio, usuários e auditoria.

Tabela 19 — Tabelas do domínio

Tabela	Colunas relevantes	Relações
`lancamentos`	`descricao`, `valor` (10,2), `status` (pendente/conciliado), `data`	Categoria, cliente, fornecedor e funcionário (opcionais)
`categorias_lancamento`	`nome`, `tipo` (receita/despesa)	Tem lançamentos
`clientes` e `fornecedores`	`nome`, `documento` (CPF ou CNPJ), `email`, `telefone`	Têm lançamentos e notas fiscais
`funcionarios`	`nome`, `cpf`, `cargo`, `salario` (10,2), `data_admissao`, `data_demissao`	Têm lançamentos

Tabela	Colunas relevantes	Relações
`notas_fiscais`	`numero`, `tipo` (entrada/saída), `valor`, `data_emissao`	Cliente, fornecedor e lançamento, com `restrictOnDelete`
`users`	`name`, `email`, `password` (hash), `papel`, `tenant_id`	Autor e autorizador na auditoria

10.1 A tabela de auditoria

`audit_logs` é a memória do sistema. Ela responde quem fez o quê, por qual caminho, com qual permissão, o que a IA propôs, o que o humano confirmou, o que havia antes, a que lote a linha pertence e quanto tempo a IA levou.

Tabela 20 — Colunas de audit_logs

Coluna	Conteúdo
`user_id`	Quem estava logado
`autorizado_por`	O administrador que digitou a senha (restaurar e excluir de vez)
`tool`	Nome da tool (ou `[tool redigida]` quando o modelo inventou uma; `consultar_auditoria` para a própria tela de auditoria)
`origem`	`agente`, `formulario` ou `manual`
`acao`	`consultar`, `criar`, `atualizar`, `excluir`, `restaurar`, `excluir_definitivo`, `exportar` ou `desconhecida`
`entidade_tipo`, `entidade_id`	O registro afetado
`lote_id`	UUID que liga as linhas de uma mesma conciliação em lote (RF03); nulo fora de lote
`parametros`	O que foi executado (mascarado)
`parametros_propostos`	O que a IA propôs (mascarado; vazio quando veio sem IA)
`valores_anteriores`	O "antes" dos campos alterados (mascarado)
`resultado`, `mensagem`	O desfecho (seção 7.2) e a frase mostrada ao usuário
`papel`, `permitido`	O papel no momento e se a ação foi permitida
`tempo_ia_ms`	Tempo da chamada à IA (RNF04)
`tenant_id`, `created_at`	Empresa e momento

> **Proposto × confirmado**
> A coluna `parametros_propostos` permite mostrar, na demonstração, um caso em que a IA sugeriu um valor e o humano o corrigiu antes de confirmar. É a evidência concreta do "Jarvis com freio".

10.2 Migrations

Tabela 21 — Migrations em ordem de execução

#	Migration	O que faz
1–3	`create_users_table`, `create_cache_table`, `create_jobs_table`	Estrutura padrão do Laravel (usuários, cache, sessões)
4	`2026_09_08_173654_create_lancamentos_table`	Lançamentos (Fase 0)
5–9	`2026_09_09_2222xx_create_*`	Categorias, clientes, fornecedores, funcionários e notas fiscais
10	`add_relacionamentos_e_governanca_to_lancamentos_table`	Chaves estrangeiras, soft delete e arquivamento em lançamentos
11	`create_audit_logs_table`	Tabela de auditoria
12	`add_papel_e_permitido_to_audit_logs_table`	Papel e permissão na auditoria (RNF01)
13	`add_papel_e_tenant_id_to_users_table`	Papel e tenant no usuário (RF10)
14	`add_proposto_autorizacao_tempo_e_origem_to_audit_logs_table`	Proposto, quem autorizou, tempo da IA e origem
15	`add_valores_anteriores_to_audit_logs_table`	Valor anterior dos campos alterados
16	`add_lote_id_to_audit_logs_table`	Id do lote (UUID, indexado) para a conciliação em lote

11. Interface e acessibilidade

A interface tem duas zonas com regras diferentes. A **vitrine** (antes do login) tem fundo escuro e pode ter movimento: um farol 3D que gira com a rolagem. A **bancada** (login e tudo depois dele) tem fundo creme e **nenhuma animação decorativa**, só o preenchimento do segurar e barras de progresso. As regras completas estão na skill `sentinel-visual` do repositório e na página "Direção visual" do Notion.

Tabela 22 — Tokens de cor (todas as combinações passam WCAG AA)

Token	Cor	Uso
`noite`	#0A0E13	Fundo da vitrine
`gelo`	#A8C8E6	Destaque sobre fundo escuro
`creme`	#FFFCEF	Fundo da bancada
`tinta` / `tinta-2`	#1D232A / #5B6570	Texto principal e secundário; borda de risco neutro (leitura)
`aco`	#2E5B85	Único destaque sobre creme: links, foco, rótulos
`aviso`	#8A5A12 (faixa #F6EBD0)	Borda de escrita pendente; avisos
`erro`	#A3412E (faixa #F5E0D9)	Borda de exclusão; mensagens de erro
`sucesso`	#2F6B4F (faixa #E4EFE6)	Confirmação concluída

11.1 Sinalização de risco

Todo card de ação do agente tem uma borda de 2 px na cor do risco: neutra para leitura, âmbar para escrita e vermelha para exclusão. A cor nunca está sozinha: título, texto da etapa e ícone repetem a mesma informação. Tipografia: Sora nos títulos, IBM Plex Sans no corpo e IBM Plex Mono em números, códigos e CNPJ, servidas do próprio projeto, com fallback para Segoe UI.

11.2 Área de trabalho redesenhada

Em 03/11 entrou na `main` o redesenho da área autenticada (PR #16): menu lateral com navegação por papel, cabeçalho da conversa, sugestões de pedido que só preenchem o campo (nunca enviam nem confirmam nada), tabelas como regiões roláveis e popup como `alertdialog`. A lógica do card (Revisar, segurar, prazos, senha) não foi tocada, e a suíte continuou verde (475 casos). A revisão de 04/11 apontou, porém, que parte do visual ainda não segue a skill: borda de risco só à esquerda, transições na bancada, gradientes, fontes abaixo do mínimo e menu lateral escuro. Os ajustes estão com a equipe de front-end e registrados na seção 14.

11.3 Acessibilidade

- Estado nunca comunicado só por cor ou só por animação (RNF07).

- Mensagens do agente em região `aria-live`; o leitor de tela ouve o início e o resultado do gesto de segurar, não a porcentagem. Avisos repetidos são relidos (o texto é limpo antes de ser gravado de novo).
- Segurar funciona com teclado (Enter ou Espaço mantidos); nada depende de passar o mouse por cima.
- Foco visível em tudo que é clicável; área de toque mínima de 44 × 44 px; link "pular para o conteúdo"; no celular, o menu prende o foco e fecha com Esc.
- Contorno dos campos com contraste de 5,8:1 (o mínimo exigido é 3:1).
- Funciona em 360 px de largura sem rolagem horizontal; tabelas largas rolam dentro do próprio contêiner.
- `prefers-reduced-motion` respeitado.

12. Testes do projeto

A estratégia de qualidade do Sentinel parte de uma regra simples: **toda garantia de governança tem de ser provada por um teste automatizado** (RNF09), e o que só um humano consegue verificar (a tela, o tempo real da IA, o MySQL de verdade) é coberto por testes manuais com roteiro. Esta seção descreve os tipos de teste usados, as técnicas aplicadas para escolher os casos, a infraestrutura que torna a suíte confiável e os resultados.

Tabela 23 — A qualidade em números

Indicador	Valor
Casos de teste automatizados	475 (397 métodos; os demais vêm de conjuntos de dados)
Asserções	4.335
Arquivos de teste	39 (37 de feature e 2 unitários), ≈ 7.900 linhas
Tempo da suíte inteira	≈ 30 s, sem rede e sem custo de API
Resultado em `c641002`	475/475 aprovados
Bateria manual por papel	57 casos no commit `574e359` (28/10), com reteste das correções e teste de fumaça no MySQL em 03/11

12.1 Tipos de teste realizados

Testes unitários

O que são. Testam uma peça isolada do código, sem banco, sem HTTP e sem framework em volta, para provar uma regra pequena e exata.

No Sentinel. Ficam em `tests/Unit`. O principal é o `DetalhesAuditoriaTest`, que prova o comportamento do escopo por requisição usado pela auditoria: escopos aninhados somam os campos e, ao sair, voltam ao estado anterior, inclusive quando uma exceção interrompe o meio do caminho. É uma regra pequena, mas toda a auditoria depende dela: se o escopo vazasse, o tempo da IA ou o "quem autorizou" de um pedido apareceria na linha de outro.

Testes de feature (integração)

O que são. Testam um caminho completo do sistema, do jeito que o usuário o usa: uma requisição HTTP entra pelas rotas reais, passa pelo middleware (login, CSRF, limites, varredura), pelo controller, pelos services e pelo banco, e a resposta é conferida junto com o que ficou gravado.

No Sentinel. São quase toda a suíte (37 arquivos). Um teste típico loga com um papel, envia `POST /agente/comando` com o "modelo" simulado respondendo uma tool, depois `POST /agente/revisar` e `POST /agente/confirmar`, e no fim confere três coisas: o status e o JSON da resposta, a tabela da entidade e a linha em `audit_logs`. Esse tipo de teste pega erros de integração que um teste unitário não veria, como uma rota sem o middleware certo ou um campo que o controller esquece de tirar antes de responder.

Testes de contrato

O que são. Conferem o formato combinado entre duas partes: o que um lado promete entregar e o outro espera receber.

No Sentinel. Há dois contratos. O primeiro é o **contrato de erro** que o navegador consome (`tipo`, `categoria`, `codigo`, `http_status`, `mensagem`, `correlation_id`): cada teste de falha confere esses campos, e nenhuma resposta pode trazer `exception` ou `trace`. O segundo é o **contrato com o provedor de IA**: o `AgenteTest` tem um conjunto de dados com onze respostas fora do formato (mensagem como texto, `choices` vazio, HTML com status 200, argumentos que não são texto...) e prova que todas viram erro amigável, nunca 500.

Testes de segurança (adversariais)

O que são. Testes escritos do ponto de vista de quem quer quebrar o sistema: em vez de conferir o caminho feliz, eles mandam exatamente o que um atacante, ou um modelo de IA confuso, mandaria.

No Sentinel. O "modelo" simulado é instruído a pedir uma tool inexistente, mandar o `tenant_id` de outra empresa, devolver JSON quebrado ou com chaves extras, trocar a operação pedida e pedir várias tools de uma vez. Do lado do navegador, os testes mandam token com ponto (para tentar navegar dentro da sessão), arrays no lugar de textos, um id de exportação adulterado e valores alterados depois do Revisar. Do lado da senha, mandam e-mail com injeção de SQL, senha de 5.000 caracteres e tentativas em sequência para estourar o limite. Em todos os casos, o teste confere que **nada foi gravado** e que a tentativa ficou na auditoria quando deveria.

Testes de concorrência

O que são. Simulam duas operações que chegam ao mesmo tempo, para provar que o sistema não produz dois resultados para a mesma coisa.

No Sentinel. Os testes `test_m1_*` disparam confirmar e cancelar da mesma proposta em sequência imediata, e confirmar duas vezes com o mesmo token. O esperado é exatamente um desfecho e, no máximo, uma gravação. O teste do lote também exige que os itens sejam lidos com trava (`lockForUpdate`). Como o SQLite dos testes ignora `FOR UPDATE`, a trava é provada por um teste que exige a chamada `doLote(..., travar: true)`, e não por disputa real entre processos; essa limitação é registrada na seção 14.

Testes de regressão

O que são. Garantem que o que já funcionava continua funcionando depois de uma mudança. Na prática, é rodar a suíte inteira e manter, para cada defeito corrigido, um teste que o reproduza.

No Sentinel. A suíte inteira roda antes de cada commit, e cada defeito achado vira um teste novo com o nome do comportamento. Exemplos que nasceram de defeitos encontrados nos testes manuais: `ConversaMarcadorTest` (o segundo cadastro seguido virava uma frase copiada, sem card), `RoteamentoIntencaoTest` (o modelo trocava exclusão por consulta), `SessaoExpiradaFormularioTest` (formulário enviado depois do logout) e `IsolamentoDoAmbienteTest` (com o cache de configuração ativo, os testes poderiam rodar contra o MySQL local e apagá-lo).

Testes de acessibilidade e de interface

O que são. Conferem se a tela pode ser usada por todos (teclado, leitor de tela, celular) e se mostra o que deve para cada papel.

No Sentinel. De forma automatizada, `CascaTest`, `WorkspaceTest` e `TelasTest` conferem rótulos dos campos, `aria-current` na página atual, o item de menu certo para cada papel, as telas de erro em português e o texto que o leitor de tela recebe. O comportamento visual (cores, foco, 360 px, o gesto de segurar com teclado) é conferido no teste manual, porque nenhuma asserção de HTML prova como a tela aparece.

Teste manual por roteiro (funcional e de aceitação)

O que é. Uma pessoa executa no navegador, passo a passo, um roteiro escrito com o resultado esperado de cada passo, e anota aprovado ou reprovado. Serve para aceitar o sistema do ponto de vista de quem vai usá-lo.

No Sentinel. A bateria tem 57 casos organizados por papel (leitura, operador e admin), cobrindo consulta, cadastro, atualização, lixeira, auditoria e exportação, com a Azure real. Na rodada sobre o commit `574e359` (28/10), 56 casos passaram; o T7.3 (exportar a auditoria) ficou bloqueado por ambiente, porque a máquina usava PHP 8.3.33. As correções feitas durante a bateria foram revisadas e reconciliadas em 03/11 (seção 12.4).

Teste exploratório

O que é. Uso livre e criativo do sistema, sem roteiro fechado, procurando comportamentos que ninguém previu. O valor está nas perguntas que surgem durante o uso.

No Sentinel. Durante e depois da bateria, a equipe repetiu pedidos com variações de texto ("quero excluir", "pode apagar", "exclua de vez"), alternou papéis e misturou cadastros seguidos. Foi assim que surgiram os defeitos de roteamento de intenção e de marcador interno no histórico, que depois viraram testes automatizados.

Teste de fumaça

O que é. Uma passada curta e rápida pelas funções principais, feita logo depois de uma mudança grande ou de montar um ambiente novo, só para saber se "acende": se algo básico quebrou, não vale a pena seguir para testes mais longos.

No Sentinel. Depois da reconciliação de 03/11, o roteiro de fumaça no MySQL do laboratório cobriu login, consulta com máscara, negação ao papel leitura, cadastro com segurar pelo teclado, conciliação em lote, exclusão como admin (e negação ao operador), restauração com senha, Editar depois do Revisar, auditoria filtrada pelo lote, exportação da auditoria e isolamento entre tenants. A conferência no banco olha só as linhas novas da auditoria.

Medição de desempenho

O que é. Medir quanto tempo o sistema leva para responder, em condições reais, e comparar com o requisito.

No Sentinel. O RNF04 é medido de dois jeitos: automaticamente, o `TempoIaTest` prova que o `tempo_ia_ms` é gravado em todos os desfechos (e fica nulo quando não houve IA); manualmente, a bateria de 28/10 cronometrou 77 pedidos com a Azure real (seção 6.5). O limite de 10 s ainda não é imposto.

Inspeção de código

O que é. Leitura crítica do código por alguém que não o escreveu, procurando erros que os testes não pegaram.

No Sentinel. Toda mudança de governança passou por uma ou duas passadas de um agente revisor separado (`@reviewer`), instruído a procurar falhas reais e citar arquivo e linha. Foram as revisões que acharam, entre outros, a senha conferida antes da permissão, a contagem de tentativas fora de ordem, um item de `tool_calls` fora do formato que virava 500 e o filtro da auditoria validado antes do papel.

12.2 Técnicas usadas para escolher os casos

Partição de equivalência

A ideia. Dividir as entradas possíveis em grupos que o sistema deve tratar do mesmo jeito e testar um representante de cada grupo, em vez de testar tudo.

No Sentinel. Os papéis formam três classes (leitura, operador, admin) mais a classe "papel desconhecido", que deve negar tudo. O campo `documento` tem as classes CPF (11 dígitos, mascarado), CNPJ (14, intacto) e "outro tipo" (redigido). A nota fiscal tem as classes entrada (exige fornecedor) e saída (exige cliente). O `PermissaoExclusaoLogicaTest` testa a exclusão com um representante de cada papel, num conjunto de dados.

Análise de valor-limite

A ideia. Os erros se concentram nas fronteiras. Para cada limite do sistema, testa-se o último valor aceito e o primeiro recusado.

Tabela 24 — Valores-limite testados

Limite	Último aceito	Primeiro recusado	Teste
Itens no lote	25	26 (recusa sem truncar)	`test_mais_de_25_pendentes_recusa_sem_truncar`
Espera mínima para confirmar	2 s	antes de 2 s ("recusado")	`test_rnf02_confirmar_antes_de_2s_e_recusado_e_nao_grava`
Validade do token de segurar	12 s	depois de 12 s ("expirado")	`test_rnf02_token_expirado_apos_12s_e_recusado_e_nao_grava`
Prazo do rascunho	5 min	depois de 5 min	`test_rascunho_expira_em_5_minutos_e_e_auditado`
Linhas no CSV da auditoria	5.000	5.001 (recusa auditada)	`test_acima_do_teto_recusa_sem_cortar_e_audita`
Pedidos ao agente por minuto	10	11º (429)	`test_rnf10_a_11a_chamada_no_mesmo_minuto_recebe_429`
Tentativas de login por minuto	5	6ª	`test_login_bloqueia_a_sexta_tentativa_no_mesmo_minuto`
Tamanho da mensagem ao agente	1.000 caracteres	1.001	`test_mensagem_acima_de_1000_caracteres_da_422_e_nao_chama_o_foundry`
Senha de demonstração	12 caracteres	11	`test_demo_senha_curta_e_recusada_sem_gravar_nada`

Um exemplo real, do teste do teto do lote: o teste cria 26 lançamentos pendentes no mesmo mês, pede a conciliação e exige três coisas, que a resposta seja texto, que a frase peça para restringir o filtro e que **nada** tenha ficado na sessão.

Código 18 PHP · tests/Feature/ConciliarLoteTest.php, linhas 74–84

```php
public function test_mais_de_25_pendentes_recusa_sem_truncar(): void
{
    foreach (range(1, 26) as $dia) {
        $this->lancamento(sprintf('2026-09-%02d', min($dia, 30)));
    }

    $this->pedirLote()->assertOk()->assertJsonPath('tipo', 'texto')
        ->assertJsonPath('mensagem', fn (string $m) => str_contains($m, 'Restrinja'));

    $this->assertSame([], session('confirmacoes', []));
}
```

Teste de transição de estados

A ideia. Quando o comportamento depende do estado (como numa máquina de estados), testa-se cada transição válida e tenta-se cada transição proibida.

No Sentinel. A proposta (seção 7.2) e o plano de lote (seção 7.8) são máquinas de estados, e o `ConfirmacaoDesfechoTest` e o `FormularioAgenteTest` cobrem as transições: rascunho → revisada, revisada → rascunho pelo Editar (sem desfecho), cada desfecho final e as transições proibidas, como confirmar sem revisar, revisar um rascunho vencido, cancelar depois de confirmar e "ressuscitar" com Editar uma proposta cujo token já venceu.

Tabela de decisão

A ideia. Quando várias condições se combinam para decidir o resultado, monta-se a tabela das combinações e testa-se cada linha.

No Sentinel. A confirmação com senha combina "tem permissão?", "senha certa?" e "limite estourou?". Sem permissão, a resposta precisa ser a mesma com senha certa ou errada e não gastar tentativa; com permissão e senha errada, "negado" e a tentativa conta; com o limite estourado, 429 mesmo com a senha certa; com permissão e senha certa, executa e o acerto devolve a tentativa. O `ReautenticacaoCardTest` e o `LixeiraTest` cobrem cada linha.

Testes negativos e entradas malformadas

A ideia. Além de conferir o que deve funcionar, provar que o que não deve funcionar é recusado de forma controlada: sem 500, sem vazar detalhe técnico, sem gravar nada.

No Sentinel. Todo endpoint tem testes com o corpo errado: array no lugar de texto, UUID malformado, campos a mais, número no lugar de senha. Esses casos são escritos como **conjuntos de dados** (data providers do PHPUnit): o mesmo teste roda uma vez para cada entrada, com um nome legível. O exemplo abaixo é o conjunto de corpos estranhos enviados à confirmação com senha; para cada um, o teste exige 403 ou 429, nenhum `trace` na resposta e nenhuma restauração.

Código 19 PHP · tests/Feature/ReautenticacaoCardTest.php, linhas 184–195

```
public static function corposEstranhos(): array
{
    return [
        'e-mail em lista' => [['email_admin' => [self::EMAIL], 'senha_admin' => self::SENHA]],
        'senha em lista' => [['email_admin' => self::EMAIL, 'senha_admin' => [self::SENHA]]],
        'senha número' => [['email_admin' => self::EMAIL, 'senha_admin' => 12345]],
        'e-mail nulo' => [['email_admin' => null, 'senha_admin' => self::SENHA]],
        'objeto no lugar da senha' => [['email_admin' => self::EMAIL, 'senha_admin' => ['x' =>
['y' => self::SENHA]]]],
        'textos enormes' => [['email_admin' => str_repeat('a', 300).'@x.test', 'senha_admin'
=> str_repeat('s', 5000)]],
        'e-mail inválido' => [['email_admin' => "a'; drop table users; --", 'senha_admin' =>
self::SENHA]],
    ];
}
```

Dublê do provedor de IA (stub)

A ideia. Trocar uma dependência externa, lenta, cara ou imprevisível por uma versão controlada pelo teste.

No Sentinel. O trait `SimulaAgente` configura credenciais falsas e usa `Http::fake` para que a chamada ao Foundry devolva a resposta que o teste escreveu. Assim o teste decide exatamente o que o "modelo" responde, inclusive respostas maliciosas e malformadas, e a suíte roda sem internet, sem custo e sempre com o mesmo resultado. O teste também pode conferir **o que foi enviado** ao provedor: o `ReautenticacaoCardTest` procura a senha de admin no corpo da requisição à IA e exige que ela não esteja lá.

Busca de vazamento em todos os destinos

A ideia. Para um dado que não pode sair (senha, CPF, salário), não basta conferir a resposta: é preciso procurar em todos os lugares onde ele poderia ter parado.

No Sentinel. O teste `test_a_senha_nao_aparece_na_resposta_log_auditoria_sessao_historico_cache_nem_na_ia` digita uma senha conhecida e a procura na resposta HTTP, no arquivo de log, em todas as colunas da auditoria, na sessão, no histórico da conversa, no cache e no corpo enviado à IA. O `AuditoriaMascaraTest` faz o mesmo com CPF e salário em todas as colunas de `audit_logs`, inclusive quando a gravação falha.

Mutação manual ("sabotagem")

A ideia. Um teste só tem valor se falha quando a proteção some. Para provar isso, desliga-se a proteção de propósito, roda-se o teste e confere-se que ele quebra; depois a proteção volta.

No Sentinel. Cada proteção nova foi sabotada antes de ser aceita. Alguns exemplos das rodadas de 03/11:

Tabela 25 — Exemplos de mutação manual e o teste que a detectou

Proteção sabotada	O que foi feito	Resultado
Auditoria da divergência de intenção	Tirar a chamada a `auditarDivergencia()`	6 testes falharam

Proteção sabotada	O que foi feito	Resultado
Só os nomes dos argumentos	Gravar os valores em vez dos nomes	O teste da divergência falhou
Tempo da IA na divergência	Abrir o escopo sem `tempo_ia_ms`	A asserção do tempo falhou
Item de `tool_calls` fora do formato	Voltar o tipo do parâmetro para `array`	O teste do item malformado falhou (500)
Teto do CSV da auditoria	Tirar o `limit` e o teto	O teste do teto falhou
Papel antes dos filtros na auditoria	Conferir o papel depois da validação	O teste do não admin falhou
Senha forte na demonstração	Tirar as duas checagens do seeder	Os 2 testes de recusa falharam

Rastreabilidade requisito → teste

A ideia. Cada requisito precisa apontar para os testes que o provam, e cada teste deve deixar claro o que prova.

No Sentinel. Os nomes dos testes são frases em português que dizem o comportamento esperado ("excluir e só para admin", "falha no meio da gravação desfaz o lote inteiro"), e muitos levam o código do requisito no nome (`test_rf09_*`, `test_rnf02_*`). A seção 13 usa esses nomes como evidência de cada requisito.

12.3 Infraestrutura de testes

- **PHPUnit com o Laravel:** `php artisan test` roda tudo; os testes de feature usam `RefreshDatabase`, então cada teste começa com o banco limpo.
- **Banco em memória:** SQLite `:memory:`, cache e sessão em `array`. O `phpunit.xml` força essas variáveis (`force="true"`), e o `tests/bootstrap.php` aplica a mesma configuração ao processo e ignora o cache de configuração local, para que os testes nunca rodem contra o MySQL de desenvolvimento. O `IsolamentoDoAmbienteTest` prova isso.
- **Senhas rápidas:** `BCRYPT_ROUNDS=4` nos testes, para que os de login e reautenticação não demorem.
- **Helpers compartilhados:** `tests/Concerns/SimulaAgente.php` (o provedor simulado e atalhos para comando, revisão e confirmação) e o `TestCase` base.

12.4 Testes automatizados por arquivo

Tabela 26 — Testes automatizados por arquivo (397 métodos; 475 casos com os conjuntos de dados)

Arquivo de teste	Métodos	O que cobre
`FormularioAgenteTest`	36	Formulário no card (RF04), Revisar, valores do navegador ignorados, rascunho de 5 min, salário só para admin
`AgenteTest`	26	Leitura, escrita confirmada (RNF02), limite de taxa (RNF10), contrato com o provedor (respostas fora do formato)

Arquivo de teste	Métodos	O que cobre
`LixeiraTest`	25	Excluir, restaurar, excluir de vez, dependências, papel e senha
`ConfirmacaoDesfechoTest`	23	Cancelar, expirar, varredura, logout, corrida entre confirmar e cancelar
`AtualizarTest`	19	Atualizar por id ou nome, botão Editar, alteração concorrente, outro tenant
`AutenticacaoTest`	17	Login, logout, limite de tentativas, contexto do usuário
`ConciliarLoteTest`	16	RF03: rascunho com ids, teto de 25, plano montado no servidor, revisão item a item, tudo ou nada, trava, auditoria por item
`AuditoriaConsultaTest`	16	Consulta da auditoria: só admin, por tenant, filtros, paginação, detalhe mascarado
`AuditoriaMascaraTest`	15	Máscara de CPF, salário e documento em todos os formatos e colunas
`AgenteGovernancaTest`	15	Tool inventada, tenant ignorado, RBAC repetido, atomicidade da auditoria
`ExportarCsvTest`	14	CSV das consultas: anti-fórmula, BOM, máscara, limite, tenant, auditoria
`AtualizarStatusLancamentoTest`	14	Conciliar e voltar a pendente, recusas com frase amigável
`FuncionarioNotaFiscalToolsTest`	14	Tools de funcionário (só admin) e nota fiscal (vínculos por tipo e por tenant)
`RoteamentoIntencaoTest`	13	Intenção explícita: negação antes da IA, tool forçada, divergência auditada, exclusão definitiva
`ReautenticacaoCardTest`	13	Senha de admin: ordem, limites, corpos estranhos, nunca vaza, nunca 500
`ValorAnteriorTest`	13	Valor anterior mascarado; tool inventada grava só nomes
`AgenteCorrecoesTest`	12	Argumentos inválidos, histórico isolado e higienizado, transporte com TLS e timeout
`GovernancaTest`	10	RF08, RF09, RF11, RF12, RNF03 nos services; tenant nos services; `%` e `_` literais
`DemonstracaoSeederTest`	10	Dados da demonstração: volume, isolamento do tenant 102, senha forte, idempotência
`ClienteFornecedorToolsTest`	9	Tools de cliente e fornecedor (conjuntos de dados para as duas entidades)
`TempoIaTest`	8	RNF04: `tempo_ia_ms` em cada desfecho e nulo sem IA
`CategoriaLancamentoToolsTest`	8	Tools de categoria de lançamento
`ModoManualTest`	7	RNF05: com a Azure fora do ar, lista, busca, cadastra e exporta

Arquivo de teste	Métodos	O que cobre
`CascaTest`	7	Layout autenticado, menu por papel, telas de erro em português
`ExportarAuditoriaTest`	6	CSV da auditoria: só admin, tenant, anti-fórmula, teto de 5.000, papel antes dos filtros, 429
`TelasTest` e `PrepararAcessoLocalTest`	5 + 5	Telas por papel; criação de acesso local com senha aleatória
Outros 12 arquivos	21	Workspace, vitrine, login, `DetalhesAuditoria` (unitário), marcador do histórico, sessão expirada, isolamento do ambiente, amostra removida, matriz de exclusão, criar lançamento, exemplos do Laravel

12.5 Testes manuais e homologação

A rede de segurança da construção foi a suíte automática; os testes manuais verificam o que ela não alcança. A tabela resume as rodadas.

Tabela 27 — Rodadas de teste manual

Rodada	Quando	Escopo	Resultado
Bateria por papel	28/10, commit `574e359`	57 casos com a Azure real, em SQLite (o MySQL da máquina deu erro 1045)	56 aprovados; T7.3 bloqueado por ambiente (PHP 8.3.33). Tempos HTTP anotados (seção 6.5)
Correções e reteste	29/10 a 03/11	Exportar a auditoria, roteamento de intenção, marcador do histórico, foco do card, sessão expirada	Revisado em 29/10 e reconciliado em 03/11; tudo coberto por testes automatizados novos
Reconciliação	03/11	Diferenças desde `574e359`, revisão de segurança em duas passadas e pendências	Sem falha de tenant, RBAC, LGPD ou escrita sem confirmação; 3 correções aplicadas
Fumaça no MySQL	03/11	Roteiro F1–F10 nas funções principais, com o MySQL migrado (16 migrations)	Concluído pela equipe; o Editar depois do Revisar foi reconferido

Itens que **só** o teste manual comprova: a parte visual do RNF07, o bloqueio de CSRF (o Laravel o desliga nos testes), o comportamento em 360 px, o aviso de "Salvar senha?" do navegador e os tempos reais da IA (RNF04). As evidências das rodadas estão em `documentacao/testes/`.

13. Requisitos e rastreabilidade

Os requisitos são os da versão 1.0. Para cada um, a tabela indica a situação e a evidência no código e nos testes (arquivos em `tests/Feature/`).

13.1 Requisitos funcionais

Tabela 28 — Rastreabilidade dos requisitos funcionais

ID	Requisito	Situação	Evidência
RF01	Interpretar comandos em linguagem natural e mapeá-los para operações válidas	Feito	`processar()`/`interpretar()`, `OperacaoSolicitada`, catálogo de 38 tools · `AgenteTest`, `AgenteGovernancaTest`, `RoteamentoIntencaoTest`
RF02	Consultar com filtros em linguagem natural; tabela editável e exportável	Feito	Tools `consultar_*`, `editavelPor`, `ConsultasExportaveis`, `PlanilhaCsv` · `AtualizarTest`, `ExportarCsvTest`
RF03	Executar ações em massa somente com confirmação explícita	Feito	`conciliar_lancamentos_em_lote`: plano com até 25 itens, revisão item a item, transação única, `lote_id` · `ConciliarLoteTest` (16)
RF04	Cadastro e atualização por formulário pré-preenchido, invocado pela IA	Feito	`FormularioTool`, `AgenteController::revisar()` · `FormularioAgenteTest` (36), `AtualizarTest`
RF05	Telas de CRUD tradicionais como via alternativa	Feito	`TelasController`, `/agente/consultar`, `/agente/formulario` · `ModoManualTest`, `TelasTest`
RF06	Registrar em log toda ação executada, cancelada ou negada	Feito	`AuditoriaService`, `executarTool()`, `auditarDesfecho()`, `auditarDivergencia()` · `AgenteGovernancaTest`, `ConfirmacaoDesfechoTest`, `RoteamentoIntencaoTest`
RF07	Consulta e filtragem do log por usuários autorizados	Feito	`ConsultaAuditoria`, `AuditoriaController`, tela `/auditoria`, exportação CSV · `AuditoriaConsultaTest` (16), `ExportarAuditoriaTest` (6)
RF08	Bloquear exclusão com dependências ativas, com motivo simples	Feito	`ExclusaoSegura` · `GovernancaTest::test_rf08_*`, `LixeiraTest`
RF09	Exclusão vai para lixeira com retenção configurável	Feito	SoftDeletes + `retencao_lixeira_dias` (90) · `GovernancaTest::test_rf09_*`
RF10	Restaurar da lixeira com senha de administrador, registrando na auditoria	Feito	`restaurar_*`, `ReautenticacaoAdmin`, `autorizado_por` · `LixeiraTest`, `ReautenticacaoCardTest` (13)
RF11	Após o prazo, mover para arquivado (invisível, mantido no banco)	Feito (depende do agendador do SO)	`sentinel:arquivar-expirados`, agendado diariamente; o `instalar.bat` oferece criar a tarefa no Windows · `GovernancaTest::test_rf11_rf12_*`
RF12	Exclusão física só manual, por administrador	Feito	`excluir_definitivo_*` (admin + senha) · `GovernancaTest`, `LixeiraTest`

13.2 Requisitos não funcionais

Tabela 29 — Rastreabilidade dos requisitos não funcionais

ID	Requisito	Situação	Evidência
RNF01	RBAC aplicado às tools da IA, não só às telas	Feito	`papel_minimo` por tool, `papelAtualPermite()`, negação antes da IA · `test_rbac_hierarquia_leitura_operador_admin`, `test_rbac_papel_desconhecido_nega_tudo`
RNF02	Toda escrita exige confirmação humana explícita	Feito	Proposta → Revisar → token → segurar 2 s → confirmar · `AgenteTest::test_rnf02_*` (4)
RNF03	Validação exclusivamente no backend	Feito	Form Requests via `validar()` + `escoparPorTenant()` · `test_valor_alterado_no_navegador_depois_do_revisar_e_ignorado`
RNF04	Tempo de resposta mensurável e dentro de um limite (referência: 10 s)	Em aberto	Mensurável: `tempo_ia_ms` em cada desfecho · `TempoIaTest` (8). Medição manual de 28/10: p90 de 4,05 s e máximo de 9,07 s. O limite não é imposto (timeout de 30 s); decisão depende de uma rodada gravando no banco
RNF05	Com a IA fora do ar, o CRUD continua operável	Feito	Modo manual · `test_com_a_azure_fora_do_ar_lista_busca_cadastra_e_exporta`
RNF06	Isolamento de tenant em todas as camadas, inclusive nas tools	Feito	`ContextoUsuario`, tenant sobrescrito, `exists` escopado · `test_tenant_enviado_pelo_modelo_e_ignorado`, `ExportarCsvTest`, `ExportarAuditoriaTest`
RNF07	Estado de ação pendente em pelo menos dois canais	Feito (visual a conferir)	Borda + título + texto + `aria-live` + popup · `test_confirmacao_pendente_traz_o_risco_escrita_da_tool`; parte visual no teste manual
RNF08	Expirar cancela só a ação pendente	Feito	`VarrerConfirmacoesVencidas` · `test_varredura_nao_mexe_na_proposta_ainda_valida`
RNF09	Governança coberta por testes automatizados	Feito	475 casos; tipos e técnicas na seção 12
RNF10	Limite de taxa na rota da IA	Feito	`throttle:agente` (10/min) e demais limitadores · `test_rnf10_a_11a_chamada_no_mesmo_minuto_recebe_429`

14. Limitações reconhecidas

As limitações abaixo são conhecidas e foram aceitas conscientemente. Nenhuma delas permite gravar dado sem confirmação ou acessar dado de outro tenant.

Tabela 30 — Limitações e riscos aceitos

#	Limitação	Impacto e motivo da aceitação
1	Campo sensível opcional (ex.: CPF no documento) não pode ser apagado pelo formulário: em branco mantém o valor anterior	Evita apagar dado sem querer; para limpar, é preciso outro caminho
2	Campo sensível com erro de validação volta em branco no formulário	O valor digitado não é devolvido ao navegador (LGPD)
3	No histórico da conversa, o salário só é escondido logo depois de "salário" ou "remuneração"	"ganha 4500" e o segundo número de "entre 3000 e 4500" passam; CPF e CNPJ já são omitidos
4	Duas abas gravando a sessão ao mesmo tempo podem perder uma proposta ou um id de exportação	Efeito: "proposta indisponível"; nada é gravado indevidamente
5	A página 404 de usuário logado não mostra o menu	Só navegação; o link para o agente aparece
6	A lista de escolha (vários registros com o mesmo nome) não é auditada	Não grava nada; a ação escolhida depois é auditada
7	Recusado e erro de atualização sem `entidade_id` na coluna	O id aparece nos parâmetros
8	Senha de admin errada encerra a proposta	É preciso pedir de novo; é a ordem mais segura e reduz tentativas por proposta
9	`lockForUpdate` não tem efeito no SQLite dos testes	No MySQL a trava vale; a chamada com trava é exigida por teste
10	CNPJ sem máscara e filtro "nome" sem máscara na auditoria de consulta	CNPJ é público; um CPF digitado no filtro apareceria
11	CSV: número negativo puro sem apóstrofo; telefone "+55" com apóstrofo	Decisão: `-150.00` é número; o apóstrofo no telefone é inofensivo
12	Modo manual: tool fora do catálogo recebe 422 sem linha na auditoria	Não executa nada; o limite de 30/min continua valendo
13	O lote revalida existência, tenant e status de cada item, mas não se ele ainda está no período ou na categoria	Coerente com os ids congelados: o humano aprovou aquela lista; a janela é de no máximo 5 min
14	Erros 403, 422 e 429 da exportação da auditoria aparecem como JSON no navegador (o download abre por `window.location`)	Cosmético; só admin chega ali, e o teto de 5.000 é raro na demonstração
15	O histórico pode ficar com falas do usuário seguidas (as respostas internas não voltam à IA)	Não é problema de segurança; o corte de 2 em 2 mensagens pode separar pergunta e resposta

#	Limitação	Impacto e motivo da aceitação
16	Parte do visual da área de trabalho ainda não segue a skill `sentinel-visual` (seção 11.2)	Ajustes com a equipe de front-end; a lógica do card não foi afetada
17	RNF04: o limite de 10 s é medido, não imposto, e ainda não há amostra de `tempo_ia_ms` no banco da demonstração	Decisão em aberto (seção 15)
18	RF11 depende do Agendador de Tarefas do Windows chamar `schedule:run`	Sem ele, o arquivamento só roda manualmente; o `instalar.bat` oferece criar a tarefa
19	`tenant_id` sem chave estrangeira	Ainda não existe a tabela de empresas
20	Restaurar não desfaz efeitos externos já ocorridos (ex.: uma notificação enviada)	Limitação inerente; o Sentinel não envia notificações hoje
21	Dependência de serviço de terceiros e custo por uso da API	Mitigadas pelo modo manual e pelos limites de taxa

15. Próximos passos

15.1 Antes da entrega (25/11)

- Definir a data de congelamento de funcionalidades; depois dela, só correção.
- Aplicar os ajustes visuais pendentes da área de trabalho e repetir a parte visual do roteiro manual.
- Rodada curta com a Azure real (20 pedidos ou mais, misturando leitura, escrita e lote) gravando `tempo_ia_ms`, para decidir o RNF04.
- Máquina da apresentação: PHP 8.4.27 ou mais, `session.upload_progress.enabled=0`, `APP_DEBUG=false`, `SESSION_ENCRYPT=true`, contas criadas por `sentinel:preparar-acesso` (nunca senha conhecida), `DEMO_SENHA` forte e o Agendador do Windows chamando `schedule:run` (RF11).
- Atualizar o roteiro manual: exclusão feita como admin (e negada ao operador), credenciais do admin antes do Revisar e "senha errada encerra a proposta".
- Dados de demonstração (`DemonstracaoSeeder`), plano B em vídeo e ensaio com o professor.

15.2 Depois do TCC

- Tabela de empresas (tenants) com chaves estrangeiras e migração para Supabase PostgreSQL (checklist no `CLAUDE.md`).
- Validação real de documentos (CPF, CNPJ alfanumérico, PIS) e verificação de e-mail.
- Outras ações em massa no mesmo padrão de plano (ex.: excluir vários registros), reaproveitando o mecanismo do lote.
- Camada offline (PWA ou aplicativo com banco local) para a falta total de conectividade.
- Entrada por voz, dada a familiaridade do público com mensagens de áudio.
- Camada preditiva sobre os dados estruturados e auditados que o próprio uso gera (seção 3.4).
- Itens de produção listados no Notion: 2FA, recuperação de senha, observabilidade e hardening.

Referências

MICROSOFT. Anomaly Detector: What's New / FAQ. Microsoft Learn, Azure AI Services, 2025–2026.

MICROSOFT. Azure OpenAI in Microsoft Foundry: function calling (Chat Completions). Microsoft Learn, 2026.

NIELSEN, J. AI as a Paradigm Shift in User Interfaces. Nielsen Norman Group, 2023.

OWASP. CSV Injection. OWASP Foundation.

OWASP. Top 10 for Large Language Model Applications. OWASP Foundation, 2025.

SEBRAE. Mapa de Empresas: 2º quadrimestre de 2025. Ministério do Empreendedorismo, da Microempresa e da Empresa de Pequeno Porte, 2025.

SEBRAE. Pesquisa TIC 2025: Transformação Digital nos Pequenos Negócios. Sebrae/PR, 2025.

SEBRAE. Pesquisa de Maturidade Digital dos Pequenos Negócios no Brasil. Sebrae/PR, 2024–2025.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD).

W3C. Web Content Accessibility Guidelines (WCAG) 2.2. 2023.

YAO, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629, 2022.

ISTQB. Certified Tester Foundation Level Syllabus, v4.0. International Software Testing Qualifications Board, 2023.

Apêndice A. Glossário

Tabela 31 — Termos usados neste documento

Termo	Significado no Sentinel
Agente	O modelo de linguagem somado à camada de orquestração que confere e executa o que ele decide
Tool (ferramenta)	Operação registrada no catálogo que o modelo pode pedir; ele escolhe, o servidor executa
Catálogo / whitelist	A lista fechada de tools; o que não está nela não existe para o modelo
Intenção explícita	Pedido de escrita reconhecido pelo servidor (verbo + entidade no começo da mensagem), que fixa a tool antes da IA
Tenant	A empresa dona dos dados; cada usuário pertence a um tenant e só enxerga os dados dele
RBAC	Controle de acesso por papel (leitura, operador, admin), aplicado a cada tool
Proposta / rascunho	Escrita pedida pela IA ou por um formulário, guardada na sessão e ainda não gravada
Plano (lote)	Proposta de ação em massa: a lista fechada de itens que vão mudar, revisada e confirmada de uma vez
Revisar	Etapa que valida o formulário (ou o plano) sem gravar e libera o botão de segurar
Token de segurar	Identificador de uso único emitido pelo Revisar e exigido na confirmação
Segurar para confirmar (hold-to-confirm)	Manter o botão Confirmar pressionado por 2 s para autorizar a escrita
Desfecho	O resultado final de uma proposta: sucesso, negado, recusado, cancelado, expirado ou erro
Soft delete / lixeira	Exclusão que só marca o registro (`deleted_at`), mantendo-o restaurável
Arquivado	Registro que passou do prazo da lixeira: invisível nas telas, mantido no banco
Falha fechada	Na dúvida (dado ausente ou inválido), negar em vez de permitir
Modo manual	Uso das telas de Cadastros e Lixeira sem chamar a IA
Human-in-the-loop	Modelo de supervisão em que a decisão de escrita é sempre de um humano
Dublê (stub)	Versão controlada de uma dependência externa usada nos testes; no Sentinel, o provedor de IA simulado
Mutação manual (sabotagem)	Desligar uma proteção de propósito para conferir que o teste dela falha

Apêndice B. Guia do código-fonte

Este apêndice percorre o repositório arquivo por arquivo. Ele foi escrito para quem vai ler, manter ou avaliar o código: o que cada arquivo faz, para que serve e onde estão as partes que importam. Para não virar uma listagem de centenas de páginas, o guia tem **três camadas**:

- **B.2 Arquivos centrais:** os treze arquivos que concentram as regras do sistema. Cada um tem uma ficha de navegação, a explicação do que faz e os trechos mais importantes, com o motivo de terem sido escritos assim.
- **B.3 Fichas curtas:** os demais arquivos com lógica própria (controllers menores, middleware, comandos, rotas, configuração, views e scripts), cada um em poucas linhas.
- **B.4 Arquivos de mesmo formato:** models, services de entidade, Form Requests e seeders, que seguem um padrão comum e são apresentados em tabelas. As migrations e os arquivos de teste já estão descritos nas seções 10.2 e 12.4.

Os trechos que já aparecem na seção 9 não são repetidos aqui; o guia aponta para eles. Os números de linha se referem à revisão `c641002`.

B.1 Mapa das pastas

Tabela 32 — Pastas do repositório

Pasta	O que guarda
`app/Services`	O coração do sistema: o agente, a governança das propostas, a auditoria, a máscara LGPD e os services de cada entidade
`app/Services/Concerns`	Traits reaproveitados pelos services de entidade: `ExclusaoSegura` e `FiltroPorNome`
`app/Http/Controllers`	Controllers finos: recebem a requisição, conferem o formato e delegam aos services
`app/Http/Middleware`	`VarrerConfirmacoesVencidas`, que audita as propostas esquecidas depois de cada requisição
`app/Http/Requests`	Os seis Form Requests, cujas regras validam as escritas e desenham o formulário do card
`app/Models` e `app/Models/Concerns`	Os models Eloquent e o trait `ComArquivamento` (lixeira e arquivamento)
`app/Console/Commands`	`sentinel:arquivar-expirados` e `sentinel:preparar-acesso`
`app/Exceptions`	`ToolRecusadaException`, a recusa de regra de negócio com frase para o usuário
`app/Providers`	`AppServiceProvider`: limites de taxa e o escopo por requisição da auditoria
`config/sentinel.php`	Hierarquia de papéis e prazo da lixeira
`routes`	`web.php` (todas as rotas) e `console.php` (agendamento do RF11)

Pasta	O que guarda
`database`	Migrations, seeders de desenvolvimento e o `DemonstracaoSeeder`
`resources/views`	Blade: layout autenticado, agente, telas, login, vitrine, erros e o partial da bancada
`resources/js` e `resources/css`	Scripts e estilos compilados pelo Vite: área de trabalho, login, vitrine e o farol 3D
`tests`	Suíte PHPUnit (seção 12), o helper `SimulaAgente` e o bootstrap que isola o ambiente
`instalar.bat`	Instalador para Windows/Laragon usado nas máquinas do laboratório e da apresentação

B.2 Arquivos centrais

B.2.1 AiAgentService.php

`app/Services/AiAgentService.php` 2.244 linhas	
Serve para	É a única porta para a IA e o centro da governança: guarda o catálogo de tools e decide, para cada pedido, se executa, propõe, nega ou recusa.
Depende de	Os seis services de entidade, `AuditoriaService`, `ContextoUsuario`, `DetalhesAuditoria`, `FormularioTool`, `OperacaoSolicitada` e os Form Requests (pelas regras).
É usado por	`AgenteController`, `TelasController`, `AutenticacaoController` e o middleware `VarrerConfirmacoesVencidas`.
Testes principais	`AgenteTest`, `AgenteGovernancaTest`, `RoteamentoIntencaoTest`, `FormularioAgenteTest`, `ConciliarLoteTest`, `LixeiraTest`.

O que faz. Recebe a mensagem do usuário e o histórico curto, reconhece um pedido de escrita explícito, chama a Chat Completions API e confere a resposta. Leitura é executada na hora; escrita vira uma proposta com formulário. Depois, nas outras etapas do fluxo, é o mesmo service que revisa a proposta sem gravar, executa a escrita confirmada dentro de uma transação, abre formulários sem IA, faz as consultas do modo manual e as exportações. Tudo o que ele grava na auditoria passa pelo `AuditoriaService`.

O arquivo é grande porque o **catálogo** (cerca de 900 linhas) está nele, de propósito: o catálogo é a fonte única de cada tool. Os métodos públicos são poucos e correspondem às etapas do fluxo:

Tabela 33 — Métodos públicos e etapas do AiAgentService

Método	Etapa	O que faz
`processar()`	Comando	Intenção explícita, chamada ao provedor com tempo medido; devolve texto, leitura, proposta, negação ou erro
`interpretar()` (privado)	Comando	Confere formato, operação pedida, catálogo e esquema; executa leitura ou monta a proposta
`executarTool()`	Execução	Whitelist, RBAC, senha (quando exigida), tenant do servidor, executor + auditoria na mesma transação

Método	Etapa	O que faz
`executarConfirmada()`	Confirmar	Abre o escopo da auditoria (proposto, origem, quem autorizou, tempo) e chama `executarTool()`
`revisar()`	Revisar	RBAC, tenant e regras da tool sem gravar; no lote, confere cada item
`formulario()` e `aplicarValores()`	Card	Montam o formulário (ou o plano) e aplicam os valores editados só nos campos permitidos
`abrirFormulario()`	Sem IA	O mesmo card aberto pelo botão Editar ou Novo, com o mesmo RBAC e tenant
`consultarManual()` e `exportar()`	Sem IA	Consulta com filtros conferidos contra o esquema; reexecução da consulta guardada para o CSV
`auditarDesfecho()`	Desfechos	Grava cancelado, expirado, recusado ou negado sem derrubar a resposta se a auditoria falhar

Partes mais importantes. A intenção explícita, a chamada ao provedor, a conferência da resposta, a execução em transação, o RBAC e o escopo das regras `exists` já estão nas seções 9.1 a 9.4 e 9.8 a 9.10. O trecho abaixo é o elo entre o Confirmar e a execução: ele mostra de onde vêm os ids do lote e como o contexto da auditoria é montado.

Código 20 PHP · app/Services/AiAgentService.php, linhas 815–829

```
public function executarConfirmada(string $tool, array $argumentos, array $propostos, string
$origem = 'agente',
    ?int $autorizadoPor = null, ?int $tempoIaMs = null, ?array $loteIds = null): array
{
    // RF03: os ids do lote vêm só da proposta guardada na sessão (o controller passa); nada
do navegador.
    unset($argumentos['ids']);
    if ($tool === 'conciliar_lancamentos_em_lote' && $loteIds !== null) {
        $argumentos['ids'] = $loteIds;
    }

    return $this->detalhesAuditoria->com([
        'parametros_propostos' => $propostos,
        'origem' => $origem,
        'tempo_ia_ms' => $tempoIaMs,
        ...($autorizadoPor === null ? [] : ['autorizado_por' => $autorizadoPor]),
    ], fn () => $this->executarTool($tool, $argumentos));
}
```

Primeiro o método **apaga** qualquer lista de ids que tenha vindo nos argumentos e só então coloca a lista congelada no rascunho. Assim, mesmo que um navegador adulterado mande outros ids, eles nunca chegam ao executor. Em seguida, abre um escopo da auditoria com o que a IA propôs, por onde a ação entrou, quem autorizou e o tempo da IA: tudo o que for gravado dentro de `executarTool()` herda esses campos, sem que cada executor precise conhecê-los.

B.2.2 AgenteController.php

app/Http/Controllers/AgenteController.php 490 linhas	
Serve para	Expõe o agente ao navegador: as oito rotas do fluxo de conversa, revisão, confirmação e exportação.
Depende de	`AiAgentService`, `ConversaAgente`, `ConfirmacoesPendentes`, `ReautenticacaoAdmin`, `ConsultasExportaveis`, `PlanilhaCsv`.
É usado por	As rotas `/agente/*` em `routes/web.php`.
Testes principais	`AgenteTest`, `FormularioAgenteTest`, `ConfirmacaoDesfechoTest`, `ReautenticacaoCardTest`, `ExportarCsvTest`, `ModoManualTest`.

O que faz. Cada ação valida só o formato do corpo (UUID, texto, tamanho), delega ao service e traduz o resultado em status HTTP. O controller também é quem **guarda na sessão** o que não pode ir ao navegador: a proposta crua (o navegador recebe só os valores mascarados e um id), a consulta exportável (o navegador recebe só um id) e os ids do lote. Nenhuma ação converte exceção em 500 cru: tudo cai no contrato de erro.

Tabela 34 — Ações do AgenteController

Ação	Rota	Responsabilidade
`processar`	POST /agente/comando	Valida a mensagem, chama o agente, guarda histórico, rascunho e consulta exportável
`revisar`	POST /agente/revisar	Aplica os valores editados e valida sem gravar; emite o token ou devolve o formulário com erros
`confirmar`	POST /agente/confirmar	Uso único, prazos, permissão antes da senha, execução com os valores da sessão
`editar`	POST /agente/editar	Anula o token de segurar e volta o card ao formulário, sem desfecho
`cancelar`	POST /agente/cancelar	Decide "cancelado" ou "expirado" pelo horário do servidor e audita
`formulario`	POST /agente/formulario	Abre o card de criar ou atualizar sem IA
`consultar`	POST /agente/consultar	Modo manual: consulta sem IA
`exportar`	POST /agente/exportar	Reexecuta a consulta guardada e devolve o CSV

Partes mais importantes. O Revisar e o Confirmar estão nas seções 9.5 e 9.6. O trecho abaixo, do final do `processar`, mostra a separação entre o que fica no servidor e o que vai ao navegador quando o agente propõe uma escrita.

Código 21 PHP · app/Http/Controllers/AgenteController.php, linhas 103–109

```
if (($resposta['tipo'] ?? null) === 'confirmacao_pendente') {
    // A sessão guarda a proposta crua; o front só vê CPF/salário/documento mascarados (LGPD).
    $resposta['proposta'] = $this->pendentes->guardarRascunho($request->session(),
$resposta['tool'],
        $resposta['argumentos'], $resposta['propostos'] ?? null,
        tempoIaMs: $resposta['tempo_ia_ms'] ?? null, loteIds: $resposta['lote_ids'] ?? null);
    $resposta['argumentos'] =
AuditoriaService::mascararCamposSensiveis($resposta['argumentos']);
    unset($resposta['propostos'], $resposta['tempo_ia_ms'], $resposta['lote_ids']);
}
```

Os valores reais (inclusive CPF e salário) e os ids do lote ficam só na sessão do servidor; o navegador recebe o id da proposta, os valores já mascarados e nada do tempo da IA. Como a confirmação depois usa apenas o que está na sessão, o que o navegador vê nunca precisa ser o dado completo.

B.2.3 ConfirmacoesPendentes.php

app/Services/ConfirmacoesPendentes.php 347 linhas	
Serve para	Guarda, na sessão, as propostas à espera de confirmação humana e controla os dois estados (rascunho e revisada), os prazos e o uso único.
Depende de	`Cache` (marca atômica) e a sessão do Laravel.
É usado por	`AgenteController`, `AutenticacaoController` (logout) e `VarrerConfirmacoesVencidas`.
Testes principais	`ConfirmacaoDesfechoTest`, `FormularioAgenteTest`, `AgenteTest::test_rnf02_*`.

O que faz. Uma proposta nasce como **rascunho** (o formulário em edição, com prazo de 5 minutos desde a proposta). O Revisar cria uma entrada **revisada**, com os valores conferidos e o token de segurar, válida de 2 a 12 segundos. Editar mata o token e volta ao rascunho. Qualquer desfecho apaga o rascunho e todos os tokens da proposta. A classe também devolve, para a varredura e para o logout, as propostas vencidas ou abertas, uma por proposta, para serem auditadas.

Partes mais importantes. O uso único (`concluir()`) está na seção 9.7. O trecho abaixo é a regra única de prazo, usada por todas as rotas para decidir entre "cancelado" e "expirado".

Código 22 PHP · app/Services/ConfirmacoesPendentes.php, linhas 255–265

```
public function expirou(array $pendente): bool
{
    if (($pendente['estado'] ?? 'revisada') === 'rascunho') {
        return $this->decorrido($pendente) > self::PRAZO_RASCUNHO_S;
    }

    $propostaCriadaEm = $pendente['proposta_criada_em'] ?? null;

    return $this->decorrido($pendente) > self::VALIDADE_MAXIMA_S
        || ($propostaCriadaEm !== null && self::agora() - (float) $propostaCriadaEm >
self::PRAZO_RASCUNHO_S);
}
```

O token vence pelo que acontecer primeiro: 12 s desde o Revisar **ou** 5 min desde a proposta original. Por isso o token guarda o horário em que a proposta nasceu: revisar de novo não

reinicia os 5 minutos. Os horários são guardados em segundos com milissegundos, porque, com segundos inteiros, a fronteira dos 12 s errava em até um segundo. Ter uma função só para isso garante que o confirmar, o cancelar, o logout e a varredura nunca discordem sobre o desfecho.

B.2.4 OperacaoSolicitada.php

`app/Services/OperacaoSolicitada.php` 60 linhas	
Serve para	Reconhecer, sem IA, um pedido de escrita explícito: o verbo (cadastrar, alterar, excluir, excluir de vez, restaurar) e a entidade.
Depende de	Nada além de `Str` (normalização de texto).
É usado por	`AiAgentService::processar()`.
Testes principais	`RoteamentoIntencaoTest` (13 métodos, vários com conjuntos de dados).

O que faz. Normaliza a mensagem (minúsculas, sem acento, espaços simples), tira prefixos de cortesia ("por favor", "quero", "você pode") e confere se a primeira palavra é um verbo conhecido e se o que vem depois começa com uma entidade conhecida. Devolve `{acao, entidade}` ou `null`.

Código 23 PHP · app/Services/OperacaoSolicitada.php, linhas 36–47

```
$texto = Str::lower(Str::ascii(trim($mensagem)));
$texto = preg_replace('/\s+/', ' ', $texto);
$texto = preg_replace('/^(?:por favor[, ]+)?(?:eu )?(?:(?:quero|desejo|preciso|gostaria)(?:
de)? |(?:voce )?(?:pode|poderia) )?(?:por favor[, ]+)?/', '', $texto);
[$verbo, $objeto] = array_pad(explode(' ', $texto, 2), 2, '');
$acao = self::VERBOS[$verbo] ?? null;
if ($acao === null) return null;

// Exclusão definitiva é uma operação distinta; nunca convertê-la em exclusão lógica.
if ($acao === 'excluir' && preg_match('/^(?:definitivamente|de vez)\b ?/', $objeto, $m)) {
    $acao = 'excluir_definitivo';
    $objeto = substr($objeto, strlen($m[0]));
}
```

A classe é deliberadamente **conservadora**: só reconhece o verbo no começo da frase e a entidade logo depois. "Quais clientes eu excluí?" não é um pedido de exclusão, e citações ou nomes que contenham "excluir" não disparam nada. Quando não há correspondência inequívoca, ela devolve `null` e o fluxo segue normal, com a IA escolhendo entre todas as tools. O tratamento separado de "excluir de vez" existe porque o modelo confundia as duas operações.

B.2.5 ReautenticacaoAdmin.php

`app/Services/ReautenticacaoAdmin.php` 142 linhas	
Serve para	Conferir a senha de um administrador do mesmo tenant antes de restaurar ou excluir de vez (RF10 e RF12).
Depende de	`ContextoUsuario`, `RateLimiter`, `Hash` e o model `User`.
É usado por	`AgenteController::confirmar()`.
Testes principais	`ReautenticacaoCardTest`, `LixeiraTest`, `AutenticacaoTest`.

O que faz. Recebe o e-mail e a senha digitados no card, aplica os dois limites de tentativas (seção 9.13) e procura um usuário com aquele e-mail, papel admin e o tenant de quem está logado. Devolve o id do admin ou o motivo da falha. Nunca lança exceção para o controller, exceto quando não há usuário logado.

Código 24 PHP · app/Services/ReautenticacaoAdmin.php, linhas 122–141

```php
private function adminQueConfere(string $email, string $senha): ?User
{
    $tenantId = $this->contexto->tenantId();

    $admin = User::query()
        ->where('email', $email)
        ->where('papel', 'admin')
        ->where('tenant_id', $tenantId)
        ->first();

    if ($admin === null) {
        // Confere mesmo assim, para o tempo de resposta não revelar se o
        // e-mail existe ou se é de um admin.
        Hash::check($senha, self::HASH_FICTICIO);

        return null;
    }

    return Hash::check($senha, $admin->password) ? $admin : null;
}
```

Quando o e-mail não é de um admin do tenant, a senha é conferida contra um **hash fictício** com o mesmo custo do bcrypt real. Sem isso, a resposta para "e-mail que não existe" seria bem mais rápida do que para "senha errada", e alguém poderia descobrir, só medindo o tempo, quais e-mails são de administradores. O filtro por tenant garante que um admin de outra empresa não autoriza nada aqui.

B.2.6 AuditoriaService.php e DetalhesAuditoria.php

app/Services/AuditoriaService.php · DetalhesAuditoria.php 155 + 48 linhas	
Serve para	Gravar cada linha de `audit_logs` num ponto só, com a máscara LGPD, e carregar os campos extras de cada requisição.
Depende de	`ContextoUsuario`, o model `AuditLog` e o contêiner do Laravel (`DetalhesAuditoria` é scoped).
É usado por	`AiAgentService`, `ConsultaAuditoria` e todos os lugares que mascaram dados (`AgenteController`, `FormularioTool`).
Testes principais	`AuditoriaMascaraTest`, `DetalhesAuditoriaTest` (unitário), `ValorAnteriorTest`, `TempoIaTest`.

O que fazem. O `AuditoriaService::registrar()` recebe os campos fixos (tool, ação, entidade, parâmetros, resultado, mensagem, papel, permitido), mascara os parâmetros e completa a linha com usuário e tenant do `ContextoUsuario` e com os campos extras que estiverem no escopo do `DetalhesAuditoria`: o proposto, os valores anteriores, quem autorizou, a origem, o tempo da IA e o id do lote. A máscara já foi mostrada na seção 9.11.

Código 25 PHP · app/Services/DetalhesAuditoria.php, linhas 22–32

```php
public function com(array $detalhes, Closure $acao): mixed
{
    $anteriores = $this->atuais;
    $this->atuais = $detalhes + $anteriores;

    try {
        return $acao();
    } finally {
        $this->atuais = $anteriores;
    }
}
```

Este método pequeno resolve um problema de desenho: quem sabe o tempo da IA (o `processar`) e quem grava a linha (o `AuditoriaService`) estão longe um do outro no código, e passar esses campos de mão em mão por todas as assinaturas espalharia detalhes de auditoria pelo sistema inteiro. O escopo é aberto por quem conhece o dado e vale para tudo o que for gravado dentro dele. O `finally` garante que o escopo é desfeito mesmo se uma exceção interromper a ação, então um campo de um pedido nunca vaza para a linha de outro. Como a classe é registrada como `scoped`, existe uma instância por requisição.

B.2.7 ConsultaAuditoria.php

app/Services/ConsultaAuditoria.php 213 linhas	
Serve para	Ler a auditoria para a tela: lista filtrada e paginada, detalhe com proposto × confirmado e exportação em CSV.
Depende de	`ContextoUsuario`, `AuditoriaService`, `DetalhesAuditoria` e os models `AuditLog` e `User`.
É usado por	`AuditoriaController` e `TelasController`.
Testes principais	`AuditoriaConsultaTest`, `ExportarAuditoriaTest`.

O que faz. Toda leitura começa por `autorizar()`, que deixa passar só o admin e, para os demais, grava uma linha "negado" antes de recusar. A consulta é sempre do tenant do usuário; os filtros (período, usuário, tool, resultado, grupo) só restringem. A lista devolve 25 por página, sem parâmetros. O detalhe mascara de novo o proposto, o confirmado e o anterior, em todos os níveis e sem diferenciar maiúsculas, como defesa em profundidade: o registro já foi gravado mascarado, mas a tela não depende disso. Para tool fora do catálogo, nada dos argumentos é mostrado.

Código 26 PHP · app/Services/ConsultaAuditoria.php, linhas 88–122 (resumido)

```php
public function exportar(array $filtros): ?array
{
    $this->autorizar('exportar');
    $filtrosAuditados = Arr::except($filtros, ['pagina']);
    $registros = $this->consulta($filtros)->limit(self::TETO_EXPORTACAO + 1)->get();

    if ($registros->count() > self::TETO_EXPORTACAO) {
        // ... audita "recusado" com os filtros
        return null;
    }

    $linhas = $registros->map(fn (AuditLog $log) => [
        'ID' => $log->id, 'Data' => $log->created_at?->format('Y-m-d H:i:s'),
        'Usuário' => $log->usuario?->name, 'Papel' => $log->papel, 'Ação' => $log->tool,
        'Entidade' => $log->entidade_tipo, 'Registro' => $log->entidade_id,
        'Resultado' => $log->resultado, 'Origem' => $log->origem, 'Lote' => $log->lote_id,
    ])->all();
    // ... audita "N registros exportados"
    return $linhas;
}
```

A consulta busca **um registro a mais** que o teto: se vierem 5.001, o filtro passou do limite e o pedido é recusado inteiro. Esse truque evita contar a tabela inteira e, principalmente, impede o pior comportamento possível, que seria entregar um CSV cortado parecendo completo. As colunas exportadas não incluem os parâmetros, para que o CSV nunca reexponha dados que a tela mostra só mascarados.

B.2.8 ConversaAgente.php

app/Services/ConversaAgente.php 95 linhas	
Serve para	Manter a memória curta da conversa que volta para a IA a cada pedido.
Depende de	`ContextoUsuario` e a sessão do Laravel.
É usado por	`AgenteController` (processar e confirmar) e o logout.
Testes principais	`AgenteCorrecoesTest`, `ConversaMarcadorTest`, `FormularioAgenteTest`.

O que faz. Guarda pares de mensagens (usuário e assistente) na sessão, ligados ao usuário e ao tenant, até 16 mensagens e 8.000 caracteres, sempre higienizados (seção 9.12). Respostas que não são texto viram frases fixas: uma proposta vira "Proposta pendente de confirmação humana. Nenhuma escrita foi executada." e uma consulta vira "A consulta foi concluída. Os dados não são guardados no histórico.". Uma escrita confirmada entra como a frase de sucesso do servidor.

Código 27 PHP · app/Services/ConversaAgente.php, linhas 22–41 (resumido)

```
public function ler(Session $sessao): array
{
    $dados = $sessao->get(self::CHAVE, []);
    if (($dados['identidade'] ?? null) !== $this->identidade()) {
        $sessao->forget(self::CHAVE);
        return [];
    }
    // É um marcador da interface, não uma resposta da IA. Reenviá-lo como
    // fala do assistente induz o modelo a copiá-lo no próximo cadastro,
    // sem chamar uma tool e, portanto, sem produzir um formulário.
    // ...
    return array_values(array_filter($dados['mensagens'] ?? [], fn ($m) =>
        ! ($m['interno'] ?? false)
        && ! (($m['role'] ?? null) === 'assistant' && in_array($m['content'] ?? null,
$internosLegados, true))));
}
```

Duas proteções aparecem aqui. A primeira é a **identidade**: se o usuário ou o tenant mudou, a memória é apagada em vez de ser lida, para que uma conversa nunca passe de uma pessoa para outra. A segunda é o filtro dos **marcadores internos**: nos testes manuais, o modelo passou a responder ao segundo cadastro seguido copiando a frase "Proposta pendente..." em vez de chamar a tool, e o usuário ficava sem card. As frases continuam guardadas para o contexto da conversa, mas não voltam para a IA como se fossem respostas dela.

B.2.9 ExclusaoSegura.php

`app/Services/Concerns/ExclusaoSegura.php` 193 linhas	
Serve para	Concentrar, num trait, o ciclo de vida de exclusão de todas as entidades: bloquear, mandar para a lixeira, restaurar e excluir de vez.
Depende de	Os models (relações e SoftDeletes) e o model `User` (confere o admin no excluir de vez).
É usado por	Os seis services de entidade, que declaram `DEPENDENTES` e `PAIS`.
Testes principais	`GovernancaTest`, `LixeiraTest`.

O que faz. Cada service diz quais relações impedem excluir (`DEPENDENTES`, por exemplo os lançamentos de um cliente) e quais precisam estar ativas para restaurar (`PAIS`, por exemplo a categoria de um lançamento). O trait faz o resto, sempre com motivos em linguagem simples. O bloqueio na exclusão está na seção 9.14. O trecho abaixo é a conferência da restauração.

Código 28 PHP · app/Services/Concerns/ExclusaoSegura.php, linhas 137–160

```
public function motivoParaNaoRestaurar(Model $registro): ?string
{
    if ($registro->arquivado_em !== null) {
        return 'Este registro já foi arquivado e não pode mais ser restaurado.';
    }

    foreach (self::PAIS as $relacao) {
        $pai = $registro->{$relacao}()->withTrashed()->first();

        if ($pai === null || ! $pai->trashed()) {
            continue;
        }

        $rotulo = self::ROTULOS_PAI[$relacao] ?? 'o registro vinculado';
        $nome = $pai->nome ?? $pai->descricao ?? $pai->numero ?? null;
        $quem = $rotulo.' #'.$pai->getKey().($nome !== null ? " ({$nome})" : '');

        return $pai->arquivado_em !== null
            ? "Não é possível restaurar: {$quem} está ".(str_starts_with($rotulo, 'a ') ?
'arquivada' : 'arquivado').'.'
            : "Não é possível restaurar: {$quem} está na lixeira. Restaure primeiro
{$rotulo}.";
    }

    return null;
}
```

Restaurar um lançamento cuja categoria está na lixeira traria de volta um registro apontando para algo excluído. Em vez de restaurar e deixar o vínculo quebrado, a recusa diz **qual** é o pai e o que fazer ("Restaure primeiro a categoria."). O método é público e só de leitura porque também é usado no Revisar, para que a recusa apareça antes de o humano segurar o botão; no Confirmar ele roda de novo, com o registro travado.

B.2.10 LancamentoService.php

app/Services/LancamentoService.php 309 linhas	
Serve para	Regras dos lançamentos financeiros: criar, consultar, atualizar o status, excluir, restaurar e conciliar em lote.
Depende de	O model `Lancamento` e os traits `ExclusaoSegura` e `FiltroPorNome`.
É usado por	`AiAgentService` (executores das tools de lançamento e do lote).
Testes principais	`AtualizarStatusLancamentoTest`, `ConciliarLoteTest`, `GovernancaTest`.

O que faz. Além dos métodos comuns a todos os services de entidade (B.4), tem duas famílias próprias. A **troca de status** (`atualizarStatus`) só muda o lançamento cujo status atual é o oposto do pedido e explica por que não mudou ("já está conciliado"). O **lote** tem três métodos que trabalham juntos: `idsPendentesNoPeriodo` (resolve o filtro), `doLote` (carrega os itens, com trava opcional) e `conciliarLote` (tudo ou nada, mostrado na seção 9.15).

Código 29 PHP · app/Services/LancamentoService.php, linhas 59–71

```
public function idsPendentesNoPeriodo(string $dataInicio, string $dataFim, ?int $categoriaId,
int $tenantId, int $limite): array
{
    return Lancamento::query()
        ->naSituacao('ativo')
        ->where('tenant_id', $tenantId)
        ->where('status', 'pendente')
        ->whereDate('data', '>=', $dataInicio)
        ->whereDate('data', '<=', $dataFim)
        ->when($categoriaId, fn ($query) => $query->where('categoria_lancamento_id',
$categoriaId))
        ->orderBy('data')->orderBy('id')
        ->limit($limite)
        ->pluck('id')->map(fn ($id) => (int) $id)->all();
}
```

O método devolve só **ids**, em ordem de data, já filtrados por tenant, situação ativa e status pendente. Quem chama pede 26 para saber se passou do teto de 25 sem precisar de uma contagem separada. Guardar ids (e não os registros) no rascunho é o que permite reconferir cada item no Revisar e no Confirmar com os dados do momento.

B.2.11 FormularioTool.php

app/Services/FormularioTool.php 231 linhas	
Serve para	Transformar as regras de um Form Request no formulário do card de confirmação (RF04).
Depende de	`AuditoriaService` (campos sensíveis e máscara), `DB` e `Schema` (listas de vínculo).
É usado por	`AiAgentService::formulario()`.
Testes principais	`FormularioAgenteTest`.

O que faz. Lê cada regra (`required`, `email`, `date_format`, `numeric`, `in:`, `exists:`, `required_if`, `prohibited_if`, `max`, `min`) e decide o tipo do campo (texto, e-mail, data, número, opções ou vínculo), se é obrigatório, se está faltando, os limites, as opções e as condições (por exemplo, "cliente" só aparece obrigatório numa nota de saída). Listas de vínculo trazem até 50 registros do próprio tenant, fora da lixeira, e sempre incluem o registro que a IA propôs, mesmo que ele não esteja entre os 50 mais recentes.

Código 30 PHP · app/Services/FormularioTool.php, linhas 40–71 (resumido)

```
$sensivel = in_array($nome, AuditoriaService::camposSensiveis(), true);
$mascara = match (true) {
    ! $sensivel => null,
    // Dinheiro que este usuário pode conferir: em reais, só nesta resposta ao front.
    in_array($nome, $revelados, true) && is_numeric($valor) => 'R$ '.number_format((float)
$valor, 2, ',', '.'),
    default => AuditoriaService::mascararCamposSensiveis([$nome => $valor])[$nome],
};
// ...
'sensivel' => $sensivel,
'valor' => $sensivel ? ($mascara === AuditoriaService::REDIGIDO ? null : $mascara) : $valor,
```

Um campo sensível nunca chega ao navegador com o valor real: o card mostra só a máscara (ou nada, no caso do salário), e deixar o campo em branco significa "manter o valor da

proposta". A única exceção é o salário para o admin, mostrado formatado em reais **apenas nesta resposta**, porque é ele quem precisa conferir o valor antes de gravar. Como o motor só lê regras, trocar de domínio (por exemplo, para logística) seria trocar os Form Requests do catálogo, não este arquivo.

B.2.12 partials/bancada.blade.php

`resources/views/partials/bancada.blade.php` 889 linhas	
Serve para	Todo o JavaScript da conversa com o agente: mensagens, tabelas, exportação, popups e o card de confirmação com o segurar.
Depende de	As rotas JSON do agente, o cabeçalho CSRF do layout e as classes CSS da bancada.
É usado por	A tela `/agente` e as telas de Cadastros e Lixeira (que abrem o mesmo card).
Testes principais	Asserções de HTML em `FormularioAgenteTest`, `ReautenticacaoCardTest`, `TelasTest`; o comportamento é conferido no teste manual.

O que faz. Monta, só com `createElement` e `textContent`, tudo o que o servidor devolve: texto do agente, tabelas (com Editar e Exportar), avisos e o card. O card é uma pequena máquina de estados no navegador, espelho da que existe no servidor: formulário → Revisar → segurar → confirmado, com Editar e Cancelar, e com prazos que desativam os botões em vez de sumir com o card.

Tabela 35 — Funções principais do script da bancada

Função	O que faz
`adicionarMensagem`, `adicionarTabela`, `exportarTabela`	Mostram as respostas do agente; a tabela traz Editar por linha (se o papel permite) e Exportar
`renderizarConfirmacaoPendente`	Cria o card: borda pelo risco, formulário (`montarCampo`, `aplicarCondicoes`) ou plano do lote (`montarLote`)
`revisar`	Envia os valores ao `/agente/revisar`; com erro, mostra a frase em cada campo; sem erro, trava os campos e mostra o botão de segurar
`iniciarHold`, `passoHold`, `soltarHold`, `confirmarAcao`	O gesto de segurar por 2 s e o envio do token (seção 9.17)
`voltarAoFormulario`	O Editar: anula o token no servidor, apaga a senha do admin e volta a pedir as credenciais
`agendarExpiracao`, `expirarCard`, `bloquearPorTempo`	Os prazos do lado do navegador (10 s para segurar, 5 min de rascunho)
`anunciar`, `mostrarPopup`	O que o leitor de tela ouve e o popup final com símbolo e texto (RNF07)

Código 31 JavaScript · resources/views/partials/bancada.blade.php, linhas 31–34, 546–550 e 696–702 (resumido)

```
function anunciar(texto) {
    statusAnuncio.textContent = '';
    setTimeout(() => { statusAnuncio.textContent = texto; }, 50);
}
// ...
function textoPrimeiraEtapa() {
    return formulario.reautenticar
        ? 'Confira os dados, chame um administrador para digitar o e-mail e a senha dele aqui e clique em Revisar.'
        : 'Confira os dados e clique em Revisar.';
}
// ... em voltarAoFormulario():
adminEmail.value = '';
adminSenha.value = '';
etapa.textContent = textoPrimeiraEtapa();
// A senha foi apagada: quem usa leitor de tela precisa saber que ela é pedida de novo.
if (formulario.reautenticar) anunciar('A senha do administrador foi apagada. Digite o e-mail e a senha de novo antes de Revisar.');
```

Este trecho é a correção de 03/11. Ao clicar em Editar depois do Revisar, a senha do admin é apagada (ela não pode ficar no campo esperando), e antes o card voltava a dizer só "Confira os dados e clique em Revisar.", sem pedir as credenciais de novo. Agora o texto da etapa é recalculado a partir do formulário atual (uma função, e não uma constante, porque o formulário é trocado depois de uma revisão com erro), e o leitor de tela é avisado. O `anunciar` limpa o texto antes de gravar o novo porque o leitor de tela não relê um texto que não mudou.

B.3 Fichas curtas

Os arquivos abaixo têm lógica própria, mas pequena ou concentrada num ponto. Cada ficha diz para que o arquivo serve, o que ele faz e o que vale observar ao mexer nele.

Controllers e middleware

app/Http/Controllers/AuditoriaController.php 109 linhas	
Serve para	Expor a auditoria em JSON (lista e detalhe) e em CSV para a tela `/auditoria`.
O que faz	Confere o papel **antes** de validar os filtros; valida período, usuário, tool, resultado, grupo e página; chama o `ConsultaAuditoria`; registro de outro tenant responde 404 igual a inexistente.
Observar	A validação nunca pode responder antes da permissão. O CSV usa as mesmas regras anti-fórmula do `PlanilhaCsv`.

app/Http/Controllers/TelasController.php 89 linhas	
Serve para	Montar as páginas de Cadastros, Lixeira e Auditoria.
O que faz	Declara, para cada entidade, a tool de consulta, a de cadastro e os filtros. O botão "Novo" só aparece para quem pode cadastrar; a Lixeira exige `consultar_lixeira`; a Auditoria passa pelo `autorizar()`.
Observar	Aqui o papel só **esconde** o que o usuário não pode usar; quem decide de verdade são as rotas JSON e os services.

app/Http/Controllers/AutenticacaoController.php 79 linhas	
Serve para	Login e logout com o `Auth` nativo do Laravel (RF10).
O que faz	Mesma frase para e-mail inexistente e senha errada; recusa quem não tem papel válido ou tenant mesmo com a senha certa; regenera a sessão; no logout, dá desfecho às propostas abertas antes de invalidar a sessão.
Observar	O logout nunca pode falhar: se não der para auditar (usuário sem tenant), ele segue mesmo assim.

app/Http/Middleware/VarrerConfirmacoesVencidas.php 36 linhas	
Serve para	Auditar como "expirado" as propostas esquecidas na sessão.
O que faz	Roda **depois** da ação (o confirmar e o cancelar dão o desfecho do próprio token primeiro) e audita as vencidas que sobraram.
Observar	Toda tela autenticada nova deve ficar no grupo deste middleware em `routes/web.php`, senão a sessão vive sem ser varrida.

Services menores e traits

app/Services/ContextoUsuario.php 60 linhas	
Serve para	Ser o ponto único de "quem está agindo": usuário, papel e tenant.
O que faz	Lê o usuário logado a cada chamada e lança `AuthenticationException` se faltar usuário, papel ou tenant.
Observar	Nunca adicionar um papel ou tenant padrão: a falha fechada é o que impede um comando ou job de agir em nome de ninguém.

app/Services/ConsultasExportaveis.php 57 linhas	
Serve para	Guardar no servidor as consultas que podem ser exportadas (RF02).
O que faz	Guarda tool, argumentos, origem e usuário sob um id aleatório, no máximo as últimas 20 e por 30 minutos; só devolve a consulta ao mesmo usuário.
Observar	O navegador só conhece o id; a exportação refaz a consulta com as regras do momento.

app/Services/PlanilhaCsv.php 66 linhas	
Serve para	Gerar os arquivos CSV (consultas e auditoria).
O que faz	UTF-8 com BOM, separador `;`, e a neutralização de fórmulas mostrada na seção 9.16.
Observar	Número puro (inclusive negativo) não ganha apóstrofo; telefone "+55" ganha, o que é inofensivo.

app/Services/Concerns/FiltroPorNome.php 22 linhas	
Serve para	Busca "contém" por nome em que `%`, `_` e `\` digitados são literais.
O que faz	Escapa os curingas com `!` e usa `like ? escape '!'`, sintaxe que funciona no SQLite, no MySQL e no PostgreSQL.
Observar	Sem isso, buscar "%" devolveria todos os registros do tenant. A coluna é sempre um nome fixo do código.

app/Exceptions/ToolRecusadaException.php 24 linhas	
Serve para	Representar uma recusa de regra de negócio (ex.: "já está conciliado").
O que faz	Carrega uma frase para o usuário e, opcionalmente, a frase de cada campo do formulário.
Observar	A mensagem é exibida como está: escrever sempre frase amigável, sem termo técnico. Qualquer outra exceção vira a frase genérica da tool.

app/Models/Concerns/ComArquivamento.php 43 linhas	
Serve para	O ciclo ativo → lixeira → arquivado nos models.
O que faz	Escopos `naLixeira`, `arquivados` e `naSituacao('ativo'|'lixeira'|'arquivado')`, usados pelos `localizar()` dos services.
Observar	Pressupõe que o model também usa `SoftDeletes`.

Comandos, inicialização e configuração

app/Console/Commands/ArquivarLixeiraExpirada.php 59 linhas	
Serve para	O RF11: arquivar o que passou do prazo da lixeira (`sentinel:arquivar-expirados`).
O que faz	Para cada entidade, marca `arquivado_em` nos registros na lixeira há mais de `retencao_lixeira_dias`; a condição se repete no próprio `UPDATE`.
Observar	A condição repetida impede marcar um registro que foi restaurado entre a listagem e a gravação. Nunca apaga nada.

app/Console/Commands/PrepararAcessoLocal.php 71 linhas	
Serve para	Criar os usuários locais com senha aleatória (`sentinel:preparar-acesso`).
O que faz	Só roda com `APP_ENV=local`; preserva contas existentes; cria o admin (e, com `--exemplos`, operador, leitura e o admin do tenant 2) com `Str::password(24)` e mostra a senha uma única vez no console.
Observar	A senha nunca vai para o log. É o caminho certo para criar as contas da apresentação.

app/Providers/AppServiceProvider.php 81 linhas	
Serve para	Registrar os limites de taxa e o escopo por requisição da auditoria.
O que faz	Define `login`, `agente`, `agente-sem-ia`, `auditoria` e `exportar` (seção 8.4), registra `DetalhesAuditoria` como `scoped` e impede que a senha do admin seja aparada ou devolvida à sessão num erro de validação.
Observar	O login é limitado por e-mail + IP, e não só por IP, porque no laboratório todos saem pelo mesmo IP.

bootstrap/app.php 29 linhas	
Serve para	Configurar rotas, middleware e tratamento de exceções da aplicação.
O que faz	Quem já está logado e abre `/login` vai ao agente; formulário HTML enviado depois do logout mostra a tela 419; rotas que esperam JSON recebem erro em JSON.
Observar	É onde mora a correção da sessão expirada (`SessaoExpiradaFormularioTest`).

routes/web.php · routes/console.php 51 + 13 linhas	
Serve para	Declarar todas as rotas e o agendamento diário do arquivamento.
O que faz	Vitrine e login públicos; tudo o mais atrás de `auth`; as rotas do agente, das telas e da auditoria dentro do grupo da varredura, cada uma com seu limitador.
Observar	O agendamento só roda se o sistema operacional chamar `php artisan schedule:run` (RF11).

config/sentinel.php 24 linhas	
Serve para	As duas configurações de negócio do sistema.
O que faz	Hierarquia de papéis (`leitura`, `operador`, `admin`) e prazo da lixeira (`SENTINEL_RETENCAO_LIXEIRA_DIAS`, padrão 90).
Observar	Não existe papel nem tenant padrão aqui, de propósito.

database/seeders/DemonstracaoSeeder.php 246 linhas	
Serve para	Criar os dados da apresentação final.
O que faz	Tenants próprios (101, a empresa demo, e 102, só para provar isolamento), cadastros com CPFs e CNPJs válidos sobre bases inventadas, lançamentos pendentes que cabem num lote e itens na lixeira. Idempotente; não grava auditoria.
Observar	Só roda com `DEMO_SENHA` de 12 caracteres ou mais e fora de uma lista de senhas conhecidas; a recusa acontece antes de qualquer gravação e não repete a senha.

Views e scripts

resources/views/layouts/app.blade.php 73 linhas	
Serve para	A casca de toda tela autenticada e das telas de erro.
O que faz	Meta CSRF, CSS e JS do Vite, menu lateral com os itens que o papel pode usar, papel por extenso, link "pular para o conteúdo".
Observar	Usa `rescue()` para continuar renderizando as páginas de erro mesmo sem banco ou sessão.

resources/views/agente.blade.php 149 linhas	
Serve para	A tela do agente.
O que faz	Cabeçalho, histórico em região `aria-live`, sugestões de pedido (só preenchem o campo) e o campo de comando; inclui o partial da bancada.
Observar	O chat não rouba o foco do card de confirmação aberto.

resources/views/telas/consulta.blade.php 156 linhas	
Serve para	As telas Cadastros e Lixeira (modo manual, RF05).
O que faz	Escolha da entidade, filtros montados a partir da definição do `TelasController`, consulta por `/agente/consultar` e o mesmo card da bancada para Novo, Editar, Restaurar e Excluir de vez.
Observar	Não chama a IA em nenhum momento.

`resources/views/telas/auditoria.blade.php` 288 linhas	
Serve para	A tela da auditoria (RF07).
O que faz	Filtros, lista paginada, detalhe com proposto × anterior × confirmado e o botão Exportar CSV com os filtros atuais.
Observar	O download abre por `window.location`; erros da exportação aparecem como JSON (limitação 14).

`resources/views/login.blade.php` · `resources/js/login.js` 91 + 98 linhas	
Serve para	A tela de login.
O que faz	Formulário enviado ao `POST /login` real com CSRF; o script só faz validação complementar e o farol animado do login.
Observar	A autenticação é sempre do Laravel; a senha nunca é reexibida depois de um erro.

`resources/views/welcome.blade.php` · `resources/js/landing.js` · `resources/js/lighthouse/*` 128 + 166 + 427 linhas	
Serve para	A vitrine pública com o farol 3D (Three.js).
O que faz	O farol gira conforme a rolagem; o carregador some mesmo se o WebGL falhar; respeita `prefers-reduced-motion`.
Observar	É a única zona com movimento. Os três botões levam ao login real.

`resources/js/workspace.js` · `resources/css/workspace.css` 50 + 338 linhas	
Serve para	Navegação e visual da área de trabalho.
O que faz	Menu lateral no celular com foco preso e Esc para fechar; sugestões que preenchem o campo sem enviar. O CSS define o layout, as tabelas roláveis e o card.
Observar	O script declara que operações pertencem aos scripts da bancada. Os ajustes visuais pendentes (seção 11.2) estão neste CSS.

Instalação e testes

`instalar.bat` 633 linhas	
Serve para	Preparar uma máquina Windows com Laragon do zero.
O que faz	Confere PHP, extensões e certificados; instala dependências com `composer install` e `npm ci`; cria o `.env` (com `APP_DEBUG=false` e `SESSION_ENCRYPT=true`); testa a conexão com o MySQL antes de gravar credenciais; roda as migrations e o build; oferece criar a tarefa do RF11.
Observar	Só grava root sem senha depois de testar a conexão sem senha. Exige o `package-lock.json`.

`phpunit.xml` · `tests/bootstrap.php` · `tests/TestCase.php` · `tests/Concerns/SimulaAgente.php`	
Serve para	Isolar e facilitar os testes (seção 12.3).
O que faz	Forçam SQLite em memória, cache e sessão em array e senhas rápidas; ignoram o cache de configuração local; o `SimulaAgente` simula o Foundry com `Http::fake`.
Observar	É o que impede um teste de apagar o MySQL de desenvolvimento.

B.4 Arquivos de mesmo formato

Estes arquivos seguem o mesmo padrão dentro de cada grupo, por isso aparecem em tabelas: o que muda de um para outro é a entidade e alguns detalhes.

Models

Tabela 36 — Models Eloquent

Model	Linhas	Traços	Relações
`Lancamento`	59	SoftDeletes, ComArquivamento; `valor` decimal, `data` date	pertence a categoria, cliente, fornecedor e funcionário; tem notas fiscais
`CategoriaLancamento`	30	SoftDeletes, ComArquivamento	tem lançamentos
`Cliente` e `Fornecedor`	32 e 34	SoftDeletes, ComArquivamento	têm lançamentos e notas fiscais
`Funcionario`	30	SoftDeletes, ComArquivamento	tem lançamentos
`NotaFiscal`	42	SoftDeletes, ComArquivamento	pertence a cliente, fornecedor e lançamento
`AuditLog`	27	`parametros`, `parametros_propostos` e `valores_anteriores` como array; `permitido` boolean	pertence ao usuário (`usuario`)
`User`	33	Autenticável; `papel` e `tenant_id`	autor e autorizador na auditoria

Todos usam o atributo `#[Fillable]` com a lista explícita de campos graváveis, e nenhum declara relação com uma tabela de tenants, porque ela ainda não existe (limitação 19).

Services de entidade

Tabela 37 — Services de entidade

Service	Linhas	Bloqueiam excluir (DEPENDENTES)	Precisam estar ativos (PAIS)
`LancamentoService`	309	notas fiscais	categoria, cliente, fornecedor, funcionário
`ClienteService`	102	lançamentos, notas fiscais	—
`FornecedorService`	102	lançamentos, notas fiscais	—
`FuncionarioService`	104	lançamentos	—
`CategoriaLancamentoService`	100	lançamentos	—
`NotaFiscalService`	107	—	cliente, fornecedor, lançamento

Todos têm o mesmo contrato: `criar`, `atualizar`, `buscarPorFiltro`, `localizar` (por id ou por nome, até 6 resultados, na situação pedida), `excluir`, `restaurarDaLixeira` e `excluirDefinitivo`, sempre com `int $tenantId` obrigatório. Registro de outro tenant ou na lixeira é tratado como inexistente (`findOrFail` dentro do tenant), e restaurar e excluir de vez leem o registro com `lockForUpdate`. A mecânica de exclusão vem do trait `ExclusaoSegura` (B.2.9) e a busca por nome do `FiltroPorNome` (B.3).

Form Requests

Tabela 38 — Form Requests

Form Request	Campos e regras principais
`LancamentoRequest`	`descricao` obrigatória (até 255); `valor` maior que zero e até 99.999.999,99; `data` no formato AAAA-MM-DD
`CategoriaLancamentoRequest`	`nome` obrigatório; `tipo` receita ou despesa
`ClienteRequest` e `FornecedorRequest`	`nome` obrigatório; `documento`, `email` (válido) e `telefone` opcionais
`FuncionarioRequest`	`nome`, `cpf`, `salario` (até o teto de decimal 10,2) e `data_admissao` obrigatórios; `data_demissao` não pode ser anterior à admissão
`NotaFiscalRequest`	`numero`, `tipo`, `valor` e `data_emissao` obrigatórios; cliente obrigatório na saída e proibido na entrada, fornecedor ao contrário; vínculos por `exists`

Os Form Requests não são injetados em controllers: o `AiAgentService` usa as regras deles para validar (com as regras `exists` reescritas para o tenant, seção 9.10) e o `FormularioTool` usa as mesmas regras para desenhar o card. As datas usam `date_format:Y-m-d` em vez de `date` porque "10/09" seria lido como 9 de outubro.

Seeders de desenvolvimento

`DatabaseSeeder` chama os seeders de cada entidade (`CategoriaLancamentoSeeder`, `ClienteSeeder`, `FornecedorSeeder`, `FuncionarioSeeder`, `LancamentoSeeder` e `NotaFiscalSeeder`, de 28 a 46 linhas cada), que criam alguns registros nos tenants 1 e 2 para o desenvolvimento. Os dados da apresentação vêm do `DemonstracaoSeeder` (B.3), que é separado e só roda sob comando explícito.